

UNCERTAINTY-AWARE PROBABILISTIC FORECASTING OF NON-COOPERATIVE
DYNAMIC OBSTACLES FOR SAFE AUTONOMOUS AERIAL NAVIGATION

A Dissertation

by

SYED IZZAT ULLAH

BS, Balochistan University of Info. Tech., Engineering and Management Sciences, 2016
MS, Lahore University of Management Sciences, 2020

Submitted in Partial Fulfillment of the Requirements for the Degree of

DOCTOR OF PHILOSOPHY

in

GEOSPATIAL COMPUTER SCIENCE

Texas A&M University-Corpus Christi
Corpus Christi, Texas

August 2026

© Syed Izzat Ullah
All Rights Reserved
August 2026

UNCERTAINTY-AWARE PROBABILISTIC FORECASTING OF NON-COOPERATIVE
DYNAMIC OBSTACLES FOR SAFE AUTONOMOUS AERIAL NAVIGATION

A Dissertation

by

SYED IZZAT ULLAH

This dissertation meets the standards for scope and quality of
Texas A&M University-Corpus Christi and is hereby approved.

José Baca, PhD
Chair

Pablo Rangel, PhD
Committee Member

Tianxing Chu, PhD
Committee Member

Carlos Rubio-Medrano, PhD
Committee Member

Susan Elwood, PhD
Graduate Faculty Representative

August 2026

ABSTRACT

Unmanned Aerial Vehicles (UAVs) are increasingly deployed in complex, dynamic environments for applications such as urban air mobility, surveillance, emergency response, and package delivery. Safe autonomous navigation in airspace shared with non-cooperative dynamic obstacles, such as birds, unregistered drones, and other uninformed aerial agents, remains a fundamental challenge in aerial robotics. Existing motion planning methods are constrained by significant limitations that undermine their effectiveness in real-world scenarios. First, they often rely on linear or constant-velocity motion models that do not capture the nonlinear and context-dependent behavior of dynamic obstacles, which can lead to delayed or inadequate avoidance maneuvers. Second, most trajectory planning frameworks lack modularity, treating obstacle prediction and planning as separate processes, which hinders proactive decision-making and limits the integration of advanced prediction models. Third, the absence of standardized benchmarking infrastructure to characterize the platform-level uncertainties such as actuator nonlinearities, estimator drift, and battery-dependent thrust degradation on resource-constrained aerial systems, that affect the perception-prediction-planning pipeline.

This dissertation addresses these limitations through four contributions within a modular framework for uncertainty-aware UAV trajectory planning. First, POF+MADER integrates a decentralized Kalman filter-based Probabilistic Obstacle Filter with optimization-based trajectory planners, replacing the assumption of perfect obstacle trajectory knowledge with real-time uncertainty-aware forecasts embedded as dynamic constraints. Evaluation across 800 simulation runs yields a 38.75% improvement in success rate with a 0.8% increase in navigation time, and hardware experiments on Crazyflie 2.1 UAVs show a 25% reduction in collision rate under realistic sensing conditions. Second, SynTraG introduces a parametric trajectory generator that produces configurable three-dimensional corpora of non-cooperative aerial obstacle motion using randomized kinematic primitives with heteroscedastic noise, and releases 47,894 trajectories to address the lack of public training data in this domain. Building on this data foundation, the third contribution, AeroCast,

combines a Pre-LN Transformer encoder with a Mixture Density Network output head to predict per-timestep Gaussian mixture distributions over future three-dimensional displacements. On a hybrid real-and-synthetic corpus of 90,116 sequences spanning nine motion categories, AeroCast reduces Average Displacement Error and Final Displacement Error by approximately 50% relative to the strongest recurrent baseline, achieves the lowest negative log-likelihood among all compared methods, and completes inference in 0.1 ms per sample. Finally, NanoBench introduces a multi-task benchmark for the Crazyflie 2.1 nano-quadrotor comprising over 170 flight trajectories with synchronized Vicon ground truth, raw IMU measurements, onboard EKF estimates, PID controller internals, and motor PWM commands at 100 Hz. NanoBench defines standardized protocols for system identification, controller benchmarking, and state estimation, and establishes the first open dataset to jointly expose actuator-level and estimator-level signals with millimeter-accurate external ground truth on a commercially available nano-scale platform.

These contributions show that uncertainty in autonomous UAV navigation must be addressed across the full pipeline rather than within any single module. In this dissertation, uncertainty is treated at the planning level through probabilistic trajectory generation, at the data level through synthetic corpus construction with realistic localization noise, at the prediction level through calibrated multi-modal forecasting, and at the platform level through benchmarked characterization of actuation and estimation effects on resource-constrained hardware.

DEDICATION

To my incredible wife, Zaibo, and to our children, Afia and Tamim. Without your love and your unwavering support, I could never have completed this journey alone.

To Zaibo, you carried our family while I carried this work. You never once let me worry about our children, and you cared for them with a patience and devotion I could never match. You held everything together so that I could reach this point. I am so proud of you, and I owe this accomplishment to you as much as to myself.

To my daughter, Afia, you asked me so many times, “Baba, can you play with me? Can we go to H-E-B?” and far too often I had to say, “not now.” I am deeply sorry for the time I could not give you. You are my whole world, and I love you more than anything I have in this life.

To my son, Tamim, you remind me each day what all of this is truly for. I dedicate this work to you with all my love.

ACKNOWLEDGEMENTS

All praise and gratitude are due to Allah, the Most Gracious, the Most Merciful, who gave me the strength, patience, and perseverance to begin and complete this journey. Nothing of this would have been possible without His countless blessings.

I would like to express my deepest and sincere gratitude to my advisor, Dr. José Baca, for his constant support, guidance, mentorship, and trust throughout my PhD journey. His mentorship shaped not only this dissertation but also the way I think, write, and approach research. I am sincerely grateful for the trust he placed in me and for the countless hours he devoted to my growth as a researcher.

I am equally thankful to my dissertation committee, Dr. Pablo Rangel, Dr. Carlos Rubio-Medrano, Dr. Tianxing Chu, and Dr. Susan Elwood, for their valuable feedback, insightful discussions, and continuous support. I deeply appreciate the expertise and perspective each of them brought to my committee.

I would also like to thank Dr. Starek, Menda, and Juanita, for all the administrative and logistics support, and for keeping the wheels turning behind the scenes.

To my labmates, Aleena, Chris, Mariah, Arjun, Kim, and Brittany, thank you for all the moments we shared together. Research can be challenging at times, but having such supportive colleagues made the journey much more enjoyable.

I am grateful to my friends Ibrar, Sohail, Abdullah, Fazal, Elshalkany, Azim, Ruben, Josh, Jacob, Ahsan, Efren, Farhan, and Fahad. Thank you for being my support system outside of research. You were my source of energy, motivation, laughter, and balance during both the good times and the difficult ones. Whenever research became overwhelming, you reminded me that there is life beyond deadlines, papers, and experiments. This journey would not have been the same without you.

Finally, my deepest thanks to my parents, Amma Jan and Abba Jan, and my siblings, especially my beloved late sister, Aqeela. Not a single day has passed without you all in my thoughts. Your

love, your prayers, and your sacrifices are the foundation of everything I have become. I miss you and love you all, always.

I gratefully acknowledge the funding that made this research possible. This work was supported in part by the National Science Foundation (NSF) under Grant No. 2131263, which supported me for three years during my PhD, and by the Geospatial Computer Science Program at Texas A&M University-Corpus Christi, which supported the remaining tenure of my PhD. I also gratefully acknowledge the support of the Department of Defense (DoD) and the Army Research Office. Research was sponsored by the Army Research Office and was accomplished under Grant Number W911NF-23-1-0186. The views and conclusions contained in this document are those of the author and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Office or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

TABLE OF CONTENTS

	Page
ABSTRACT	iv
DEDICATION	vi
ACKNOWLEDGEMENTS	vii
TABLE OF CONTENTS	ix
LIST OF FIGURES	xvi
LIST OF TABLES	xviii
CHAPTER I: INTRODUCTION	1
1.1 Overview	1
1.2 Research Objectives	5
1.3 Mapping of Objectives to Contributions	6
1.4 Computational and Hardware Requirements	7
1.5 Assumptions, Scope, and Limitations	8
1.6 Thesis Structure	9
1.7 Related Publications	11
CHAPTER II: BACKGROUND AND LITERATURE REVIEW	12
2.1 Optimization-Based Trajectory Planning for UAVs	12
2.1.1 Global and Local Planning Paradigms	12
2.1.2 Polynomial Trajectory Optimization	12
2.1.3 Multi-Agent Coordination and Collision Avoidance	13
2.2 Probabilistic State Estimation and Filtering	14
2.2.1 Bayesian Filtering Framework	14
2.2.2 The Kalman Filter	14
2.2.3 Extended Kalman Filter	15
2.2.4 Multi-Step Prediction and Uncertainty Growth	15
2.3 Deep Learning for Sequential Data	16
2.3.1 Recurrent Neural Networks	16

2.3.2	Self-Attention and the Transformer Architecture	17
2.3.3	Pre-Layer Normalization	17
2.3.4	Positional Encoding	18
2.4	Probabilistic Prediction and Mixture Density Networks	18
2.4.1	Gaussian Mixture Models	19
2.4.2	Mixture Density Networks	19
2.4.3	Training with Proper Scoring Rules	19
2.4.4	Calibration and Expected Calibration Error	20
2.5	Trajectory Prediction for Dynamic Obstacles	21
2.5.1	Classical and Model-Based Prediction	21
2.5.2	Learning-Based Trajectory Forecasting	22
2.5.3	Multi-Modal and Probabilistic Prediction	22
2.5.4	Prediction in the Aerial Domain	23
2.6	Integration of Prediction with Planning	23
2.6.1	Reactive versus Anticipatory Planning	23
2.6.2	Constraint-Based Integration	24
2.6.3	Chance-Constrained and Robust Formulations	24
2.6.4	The Freezing Robot Problem	25
2.7	Quadrotor Dynamics, Control, and Nano-Scale Considerations	25
2.7.1	Rigid-Body Dynamics	26
2.7.2	Motor Models and Thrust Mapping	26
2.7.3	Cascaded PID Control	27
2.7.4	Geometric Control on $SE(3)$	27
2.7.5	Onboard State Estimation	27
2.7.6	Nano-Scale Platform Constraints	28
CHAPTER III: POF+MADER: TRAJECTORY PLANNER IN DYNAMIC ENVIRON-		
MENTS WITH IMPROVED COLLISION AVOIDANCE		29

3.1	Introduction	29
3.2	Literature Review	32
3.3	Problem Formulation	35
3.3.1	Dynamic Obstacles and Partial Observability	36
3.3.2	Probabilistic Forecasting and Trajectory Planning	36
3.3.2.1	Forecasting Objective	36
3.3.2.2	Planning Objective	36
3.3.2.3	Safety Constraints	37
3.4	Methodology and Integration	37
3.4.1	Dynamic Obstacle Definition and Model	37
3.4.2	Probabilistic Obstacle Filtering	39
3.4.3	POF+MADER Integration	43
3.4.4	Localization Uncertainty in Obstacle Prediction	47
3.5	Experimentation and Results	49
3.5.1	Experimental Scenarios	49
3.5.2	Evaluation Metrics	51
3.5.3	Simulation Setup	51
3.5.4	Simulation Results	52
3.5.4.1	Comparison between MADER and POF+MADER	54
3.5.4.2	Comparison between RMADER and POF+RMADER	54
3.5.4.3	Comparison between POF-based Methods and Ego-Swarm	54
3.5.4.4	Performance under Localization Uncertainty	55
3.5.5	Real-World Experiments	57
3.5.6	Real-World Results	58
3.6	Discussion	59
3.6.1	The Advantage of Proactive Planning Under Uncertainty	59
3.6.2	State Estimation as a Foundation for Robust Planning	59

3.6.3	Limitations and Extensions	60
3.6.3.1	Current Limitations	60
3.6.3.2	Multi-Agent Coordination Extensions	61
3.6.3.3	Heterogeneous Obstacle Types Handling	61
3.6.3.4	Outdoor Environment Adaptation	62
3.7	Conclusions and Future Work	62
CHAPTER IV: SYNTRAG: A SYNTHETIC TRAJECTORY GENERATOR FOR NON-		
COOPERATIVE DYNAMIC OBSTACLES IN UAV NAVIGATION		64
4.1	Introduction	64
4.2	Related Work	66
4.2.1	Trajectory Forecasting	66
4.2.2	Existing Datasets for UAV Navigation	67
4.2.3	Synthetic Dataset for Motion Prediction	67
4.3	The SynTraG Framework	67
4.3.1	Mathematical Formulation	69
4.3.1.1	Forecasting task	69
4.3.1.2	Generator Objective	69
4.3.1.3	Mixture Prior over Motion Primitives	69
4.3.1.4	Primitive-Specific Parameter Priors	70
4.3.1.5	State Transition with Optional Uncertainty	70
4.3.1.6	Generative Program for One Sample	71
4.3.1.7	Induced Mixture over Data Pairs	71
4.3.2	Core Kinematic Motion Models	71
4.3.2.1	Straight-Line (Constant Velocity) Motion	72
4.3.2.2	Sinusoidal Motion	72
4.3.2.3	Trefoil-Knot Motion	73
4.3.2.4	Up-and-Down Motion	73

4.4	Reference Instantiation and Dataset Corpus	73
4.4.1	Data Generation Process	74
4.4.2	Dataset Composition and Visualization	74
4.4.3	Statistical Analysis	75
4.4.4	Accessibility and Usage	76
4.5	Conclusion	76
CHAPTER V: AEROCAS: PROBABILISTIC 3D TRAJECTORY PREDICTION FOR NON-COOPERATIVE DYNAMIC OBSTACLES		77
5.1	Introduction	77
5.2	Related Work	80
5.2.1	Classical Estimation and Recurrent Forecasting for Aerial Trajectories	80
5.2.2	Transformer-Based Trajectory Forecasting	81
5.2.3	Probabilistic Prediction and Calibration	82
5.2.4	Datasets for Non-Cooperative Aerial Prediction	83
5.3	Problem Formulation	83
5.3.1	Temporal Specifications	84
5.3.2	Consecutive Displacement Encoding	84
5.3.3	Multi-Modal Distribution Representation	85
5.4	Methodology	86
5.4.1	Architecture Overview	86
5.4.2	Input Embedding with Positional Encoding	86
5.4.3	Transformer Encoder	87
5.4.4	GMM Output Heads	88
5.4.5	Training Objective	88
5.4.5.1	Curriculum Sequence-Length Scheduling	89
5.5	Dataset	89
5.5.1	Physical Data Collection	90

5.5.1.1	Hardware Platform	90
5.5.1.2	Trajectory Categories	90
5.5.2	Synthetic Data Augmentation	90
5.5.3	Preprocessing and Sliding-Window Sampling	91
5.5.4	Dataset Statistics	91
5.6	Experimental Setup	92
5.6.1	Baseline Methods	92
5.6.2	Evaluation Metrics	93
5.6.3	Training Protocol	94
5.6.4	Implementation Details	94
5.7	Results and Analysis	94
5.7.1	Quantitative Comparison	94
5.7.2	Prediction Horizon Analysis	96
5.7.3	Per-Trajectory-Type Analysis	97
5.7.4	Ablation Study	99
5.7.4.1	Mixture Cardinality	100
5.7.4.2	Model Capacity	101
5.7.4.3	Positional Encoding	101
5.7.4.4	Input Features	102
5.7.5	Inference Latency	102
5.7.6	Discussion	102
5.8	Conclusion	104
CHAPTER VI: NANO BENCH: A MULTI-TASK BENCHMARK DATASET FOR NANO-QUADROTOR SYSTEM IDENTIFICATION, CONTROL, AND STATE ESTIMA- TION		105
6.1	Introduction	105
6.2	Related Work	107

6.2.1	Aerial Robot Datasets and Benchmarks	107
6.2.2	System Identification for Aerial Vehicles	108
6.2.3	Control of Nano-Quadrotors	109
6.2.4	State Estimation for Micro Aerial Vehicles	109
6.3	Methodology	110
6.3.1	Hardware Platform	110
6.3.2	Data Acquisition Pipeline	110
6.3.3	Time Synchronization	112
6.3.4	Trajectory Design	113
6.3.5	Benchmark Task Formulation	113
6.4	Experimental Validation and Results	115
6.4.1	Dataset Statistics	115
6.4.2	Task 1: System Identification Baselines	116
6.4.2.1	Baselines	116
6.4.2.2	Results and Analysis	117
6.4.3	Task 2: Controller Benchmarking	120
6.4.3.1	Baselines	120
6.4.3.2	Results and Analysis	121
6.4.4	Task 3: State Estimation	123
6.4.4.1	Baselines	123
6.4.4.2	Results and Analysis	123
6.5	Conclusion	124
CHAPTER VII: CONCLUSION		126
7.1	Summary of Contributions	126
7.2	Limitations	128
7.3	Future Work	129
REFERENCES		131

LIST OF FIGURES

	Page
1.1 Categories of the dynamic obstacles. Typically, UAVs encounter non-cooperative non-interactive dynamic obstacle in their missions, which is the focus of this dissertation.	2
1.2 FAA Operational context of UTM services [1]. Different layers of airspace and the typical activities in each of them according to the FAA guidelines.	4
3.3 A typical multi-UAVs multi-dynamic obstacles scenario where the UAVs detect and estimate the future trajectories of dynamic obstacles.	30
3.4 The POF+MADER system architecture; integrating decentralized probabilistic obstacle filter with optimization-based planners such as MADER [2].	44
3.5 Visualization of localization uncertainty and 3D Crazyflie drone trajectory.	47
3.6 Real-world experimental arena equipped with VICON motion capture system.	50
3.7 Success rate comparison of our approach (i.e., POF+MADER and POF+RMADER) and the baselines (i.e., MADER, RMADER, and Ego-Swarm) algorithms in a simulated environment.	52
3.8 Success rate comparison of our approach (i.e., POF+MADER and POF+RMADER) and baselines (i.e., MADER [2], RMADER [3], and Ego-Swarm [4], under localization uncertainty in simulation.	56
3.9 Performance evaluation of MADER and POF+MADER in a real-world environment. . .	58
4.10 Illustration of the UAV navigation problem among non-cooperative dynamic obstacles. .	65
4.11 SynTraG generative pipeline architecture.	68
4.12 Representative 3D trajectories for each motion primitive with uncertainty quantification.	75
5.13 A typical scenario where an ego-UAV encounters a non-cooperative dynamic obstacle (an uncontrolled drone).	80
5.14 AeroCast overall system architecture.	86
5.15 AeroCast position error at increasing prediction horizons over the full validation split. .	96
5.16 Validation and Training Loss Training Loss convergence over the epochs.	97

5.17 Qualitative comparison of AeroCast with the baseline models on various trajectory categories.	99
5.18 Qualitative comparison of AeroCast with the baseline models on a Trefoil trajectory. . .	100
6.19 NanoBench trajectory types visualized from recorded Vicon ground-truth data.	111
6.20 Task 1: Position MAE vs. prediction horizon.	119
6.21 Task 1: Predicted (colored) vs. Vicon ground-truth (black) trajectories for each baseline on a held-out Category B (Helix trajectory) sequence, along each axis.	120
6.22 Task 2: Closed-loop position (xyz, m) for each baseline vs. Ground Truth. Real-flight (PID, Mellinger) and offline (BC-MLP, BC-LSTM, MPPI) trajectories.	121
6.23 Task 2: Motor PWM outputs over time for each controller baseline (normalized). . . .	122
6.24 Task 3: EKF position error (mean $\pm$ std, all runs pooled by speed) on trefoil trajectories.	124

LIST OF TABLES

	Page
1.1 Mapping of dissertation objectives to contributions	7
1.2 Approximate hardware and computational requirements per contribution	8
1.3 Core Dissertation Publications	11
1.4 Additional Publications During the Doctoral Program	11
3.5 Performance differentials of our approach (i.e., POF+MADER and POF+RMADER) and the baselines (i.e., MADER, RMADER, and Ego-Swarm) algorithms, in a simulated environment.	53
3.6 Comparison of POF-based methods and the baselines (under localization uncertainty) .	55
3.7 Real-world Comparison of POF+MADER with MADER	58
4.8 Dataset statistics per motion type.	76
5.9 Positioning of AeroCast relative to representative trajectory prediction methods.	83
5.10 Dataset summary by trajectory category.	92
5.11 Quantitative comparison on 13,500 held-out validation sequences (5-second, 50-step horizon).	95
5.12 Per-trajectory-type ADE / FDE (m) on the held-out validation split (5-second, 50-step horizon).	98
5.13 Ablation study on the held-out validation split.	101
5.14 Comparison of model complexity and inference latency (with batch size 1 over 50 runs after warm-up)	103
6.15 Comparison of aerial robotics datasets.	109
6.16 Recorded signals and nominal sampling rates.	110
6.17 Category B: Tracking Trajectory Parameters.	112
6.18 Trajectory categories in the NanoBench dataset.	116
6.19 Numerical performance of cumulative simulation error at $h = 1, 10, 50$	118
6.20 Task 2: Controller Benchmarking Results	121

6.21 Task 3: EKF state estimation accuracy on trefoil trajectories. Mean $\pm$ std across all runs (pooled).	125
--	-----

CHAPTER I: INTRODUCTION

1.1 Overview

Unmanned Aerial Vehicles (UAVs) are transforming a wide range of industries with autonomous missions in urban air mobility[5], surveillance[6], search-and-rescue[7], and package delivery[8]. In these applications, UAVs operate in complex and dynamic environments where they must navigate safely around both static and dynamic obstacles, including vehicles, pedestrians, other drones, and even birds. These UAVs are therefore required to generate and continuously adapt trajectories in real time so that collisions are avoided and mission objectives are met.

A major challenge in autonomous UAVs navigation is dealing with dynamic obstacles whose motion cannot be known in advance. Such obstacles have time-varying, often non-linear, and context-dependent motion behaviors that are difficult to model [9]. As shown in Figure 1.1, dynamic obstacles faced by UAVs are typically categorized along two axes; first, cooperative versus non-cooperative, whether the obstacle shares intent or trajectory information with the UAV, and second, interactive versus non-interactive, whether the obstacle changes its behavior according to the UAV's presence [10]. In real-world applications, UAVs are mostly confronted by non-cooperative and non-interactive obstacles, such as pedestrians, unregistered drones, or birds, whose future trajectories are not known in advance and cannot be affected by the UAV's maneuvers. The uncertainty associated with these unknown dynamic agents poses a significant risk in safety-critical missions, where mid-air collisions or crashes can result in harm to people and property. In order to broaden the practical deployment of UAVs in real-world settings, it is crucial to develop robust, adaptive, and collision-aware motion-planning algorithms that ensure safe operation, especially in the presence of non-cooperative and non-interactive dynamic obstacles [11, 12, 13].

Designing reliable UAV trajectories in uncertain and dynamic environments remains a difficult problem in robotics. Static obstacles can usually be represented as fixed constraints once the environment is known, but dynamic obstacles require the planner to reason about how the scene may evolve over time. That requirement becomes particularly demanding when pedestrians, birds,

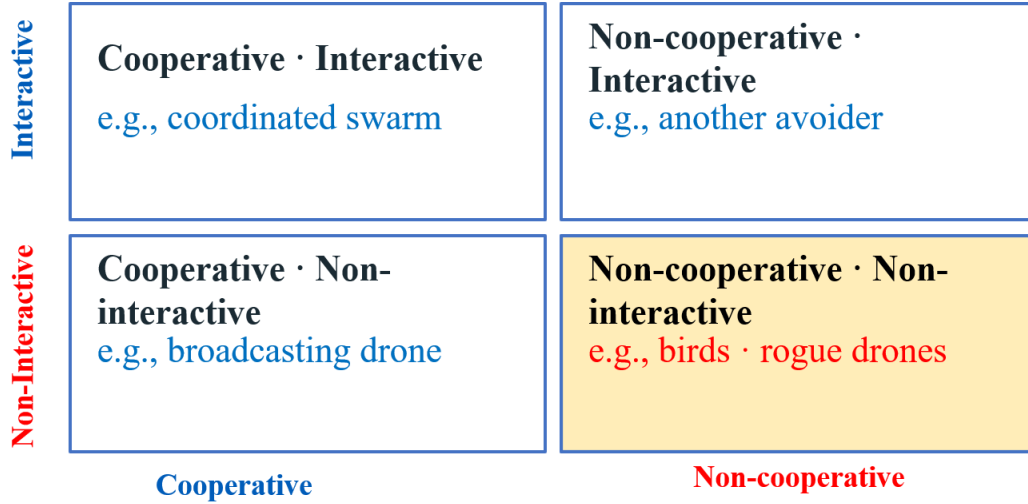


Figure 1.1
Categories of the dynamic obstacles. Typically, UAVs encounter non-cooperative non-interactive dynamic obstacle in their missions, which is the focus of this dissertation.

or other non-cooperative aerial agents change speed or direction abruptly, and even cooperative vehicles execute maneuvers that are not adequately captured by simple constant-velocity or linear-motion models [9]. Under these conditions, effective planning cannot rely solely on the current obstacle state. It must also estimate the obstacle’s likely future position over the next few seconds. Neglecting that predictive component can be hazardous, especially for UAVs, where high flight speeds leave little time for corrective action.

A large portion of the existing planning literature addresses this problem reactively, using only the obstacle’s current position or velocity to guide avoidance decisions [14, 15]. Such strategies may be adequate in slower or sparsely populated environments, but their reliability degrades when the time available to prevent a collision is limited [16]. In these settings, the vehicle may oscillate, adopt overly conservative behavior, or fail to avoid the obstacle altogether because likely future motion is not incorporated into the planning process [17].

These limitations have made anticipatory planning increasingly important. When predicted obstacle trajectories are incorporated into the planning loop, the UAV can initiate avoidance maneuvers earlier and produce smoother decisions across the planning horizon. Prior work indicates that prediction-aware planning can reduce collision risk and improve trajectory efficiency relative to

purely reactive baselines [18, 19, 20]. Coupling learning-based predictors with optimization-based planners has demonstrated significant navigation improvements in structured scenarios [20]. For instance, human motion forecasting reduces pedestrian collisions by up to 43 percent in urban drone operations [21, 22]. However, the majority of these approaches operate under idealized assumptions such as perfect obstacle state estimation, disturbance-free environments, and cooperative agent behavior. Real-world deployment introduces three critical gaps: 1) sensor degradation in perception systems (e.g., occlusions, motion blur, weather artifacts, etc.), 2) environmental disturbances (e.g., wind gusts, atmospheric turbulence, etc.) distorting obstacle kinematics, and 3) uncertainty propagation from perception to prediction pipelines [23]. These challenges remain largely unaddressed in current literature, and limits practical UAV deployment in complex scenarios.

The practical need for this research is tied directly to the regulatory trajectory of routine UAV operations in shared airspace. In June 2025, Executive Order 14307 directed the Federal Aviation Administration (FAA) to establish a rulemaking pathway for routine Beyond Visual Line of Sight (BVLOS) operations [24], and the resulting Notice of Proposed Rulemaking, commonly referred to as Part 108, was issued in August 2025 [25]. A central provision of that proposal requires unmanned aircraft operating in higher-density airspace or over populated areas to detect and avoid non-cooperative aircraft, defined as aircraft that do not broadcast their position through ADS-B Out or an approved electronic conspicuity device [25]. This requirement places the avoidance of non-cooperative agents, the problem studied in this dissertation, at the center of the safety case that will govern large-scale UAV deployment.

The broader significance of the problem is reinforced by the operational concepts developed by NASA and the FAA for managing low-altitude traffic, as shown in Figure 1.2. NASA’s UAS Traffic Management (UTM) concept of operations identifies the surveillance of non-cooperative vehicles as one of the inherent limitations that any scalable low-altitude system must accommodate [26], and the FAA UTM concept of operations similarly requires the ability to detect both cooperative and non-cooperative traffic in controlled airspace [1]. Applications anticipated under these frameworks, including package delivery, infrastructure inspection, emergency response, and urban air

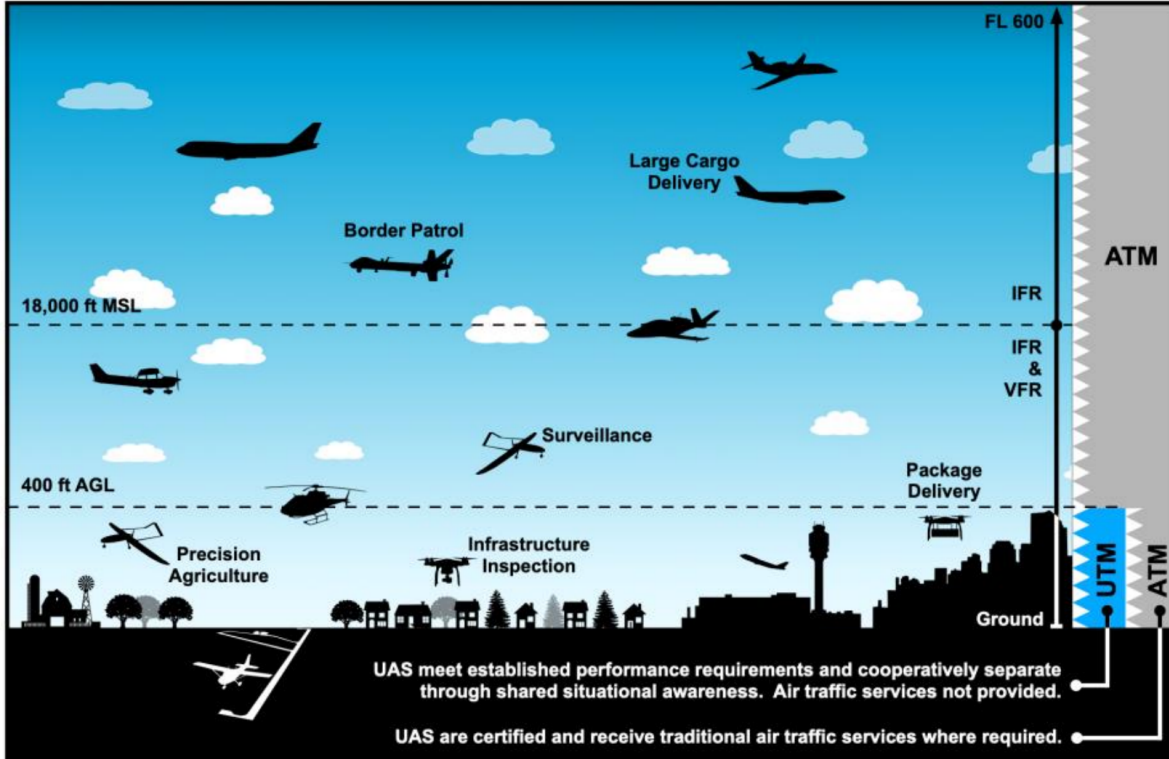


Figure 1.2
FAA Operational context of UTM services [1]. Different layers of airspace and the typical activities in each of them according to the FAA guidelines.

mobility [27], depend on a UAV's ability to reason about agents whose intent and future motion are not shared in advance. Because the residual risk in these settings is dominated by what the vehicle cannot observe or predict with certainty, methods that quantify and propagate uncertainty across estimation, prediction, and platform behavior are a prerequisite for meeting the safety expectations embedded in the emerging regulatory framework rather than an incremental refinement of existing planners.

This dissertation addresses the above challenges through four research contributions that collectively constitute a modular framework [28, 29] for uncertainty-aware UAV navigation in dynamic environments. The first contribution introduces a lightweight probabilistic obstacle filter, integrated with optimization-based multi-agent trajectory planners, that replaces the assumption of perfect a priori obstacle trajectory knowledge with time-varying, uncertainty-encoded forecasts formulated as dynamic constraints within the planning optimization. The second contribution develops a

parametric synthetic trajectory generator that produces configurable, kinematically diverse corpora of non-cooperative aerial obstacle motion, thereby addressing the absence of publicly available training and evaluation data for learning-based prediction in the three-dimensional aerial domain. Building on that data infrastructure, the third contribution presents a probabilistic Transformer-based prediction architecture that models future obstacle motion as per-timestep Gaussian mixture distributions, providing downstream planners with calibrated, multi-modal uncertainty estimates in place of deterministic point forecasts. The fourth contribution provides an open multi-task benchmark for nano-scale quadrotor dynamics, control, and state estimation, establishing the empirical foundation required to characterize the platform-level uncertainties that propagate through the perception-prediction-planning pipeline on resource-constrained aerial vehicles.

1.2 Research Objectives

The primary goal of this dissertation is to advance motion planning algorithms for UAVs operating in complex, dynamic environments characterized by non-cooperative moving obstacles, with particular attention to the role of uncertainty at the estimation, prediction, and platform levels. Three principal objectives structure the research.

Objective 1: Real-Time Probabilistic Collision Avoidance. The first objective is to develop and validate a real-time probabilistic obstacle filter that extends optimization-based trajectory planners with dynamic obstacle forecasting. The aim is to remove the assumption that obstacle trajectories are known *a priori* while preserving compatibility with existing planning frameworks such as MADER and RMADER. The proposed filter must remain lightweight enough for deployment on standard UAV hardware, account for sensor noise and localization uncertainty, and integrate with existing planners without altering their underlying optimization formulation. Its effectiveness is evaluated through large-scale simulation studies against multiple baselines as well as hardware experiments on physical UAV platforms.

Objective 2: Learning-Based Probabilistic Trajectory Prediction. The second objective is to develop the data resources and prediction models needed for multi-step probabilistic forecasting of non-cooperative aerial obstacle motion. This objective includes two closely connected components.

The first is a parametric synthetic trajectory generator that produces configurable three-dimensional trajectory corpora designed to capture the kinematic diversity of non-cooperative aerial obstacles, thereby addressing the lack of publicly available datasets for this domain. The second is a probabilistic prediction architecture that maps a finite observation history to a calibrated distribution over future obstacle motion, enabling risk-aware downstream planning while maintaining inference speeds compatible with onboard deployment.

Objective 3: Uncertainty-Aware Evaluation and Platform Benchmarking. The third objective is to address uncertainty at the platform level through the development of a benchmark dataset and evaluation framework for nano-scale quadrotors. The benchmark is intended to support reproducible comparison of system identification, control, and state estimation methods under the aerodynamic and actuation constraints specific to sub-50 g aerial vehicles. In this way, the objective connects algorithm development with physical deployment by providing a structured basis for measuring and analyzing the hardware-level uncertainties that influence the full perception-prediction-planning pipeline on resource-constrained UAV systems.

1.3 Mapping of Objectives to Contributions

The three research objectives of Section 1.2 are achieved through the four contributions summarized in Section 1.1. Table 1.1 shows the mapping of objectives to contributions, with an additional column that records the secondary role each contribution plays in the platform-level uncertainty (objective 3).

POF+MADER directly serves Objective 1 by extending optimization-based planners with real-time probabilistic obstacle forecasts, and it also produces the perception-uncertainty characterization that informs Objective 3. SynTraG and AeroCast jointly serve Objective 2. SynTraG supplies the configurable three-dimensional training corpus that the domain otherwise lacks, and AeroCast converts a finite observation history into a calibrated, multi-modal forecast suitable for risk-aware planning. Both contribute to Objective 3 by exposing, respectively, the behavioral stochasticity of non-cooperative motion and the calibration quality of learned predictors. NanoBench is the primary contribution to Objective 3, establishing the standardized platform-level characterization

Table 1.1
Mapping of dissertation objectives to contributions

Objective	Primary Contribution(s)	Chapter(s)	Secondary Support to Platform-Level (Objective 3)
Objective 1: Real-Time Probabilistic Collision Avoidance	POF+MADER	Ch. 3	Provides perception-uncertainty characterization for Objective 3
Objective 2: Learning-Based Probabilistic Trajectory Prediction	SynTraG; AeroCast	Ch. 4; Ch. 5	Exposes behavioral stochasticity and predictor calibration for Objective 3
Objective 3: Uncertainty-Aware Evaluation and Platform Benchmarking	NanoBench	Ch. 6	—

of actuation, estimation, and battery-coupled effects on a resource-constrained vehicle that any uncertainty-aware deployment must account for.

1.4 Computational and Hardware Requirements

This subsection clarify the resources required to reproduce and deploy each contribution, Table 1.2 summarizes the hardware used at the most demanding stage of each method together with the corresponding compute footprint. The figures are reported for the configurations evaluated in Chapters 3 through 6 and are intended as practical guidance rather than as formal complexity bounds. Two observations follow. The learning-based predictor is the only contribution that requires a GPU, and even there a single workstation-class accelerator with 8 GB of memory is sufficient. The remaining contributions, namely the planner, the data generator, and the benchmark acquisition pipeline, run on a single commodity laptop CPU, which keeps the full pipeline accessible without specialized infrastructure.

Onboard execution on the nano-scale platform itself is restricted to the firmware control and estimation loop running on the Cortex-M4 processor described in Chapter 6, so the planning and prediction components are evaluated on host hardware in line with the platform constraints discussed in Section 1.5.

Table 1.2

Approximate hardware and computational requirements per contribution

Contribution	Stage	Hardware Platform	Approximate Requirement
POF+MADER (Ch. 3)	Planning (Execute)	CPU only, 12th Gen Intel Core i7-12700H laptop, Ubuntu 20.04 / ROS Noetic; no GPU	Real-time planning in simulation and on a host PC for hardware flights; onboard nano-scale compute is insufficient
SynTraG (Ch. 4)	Data Generation	CPU only, i7-12700H, single-threaded; no GPU, no training	~47,894 sequences in ~10 s
AeroCast (Ch. 5)	Training	Single NVIDIA RTX A2000 (8 GB); PyTorch	~8 h for 300 epochs; 13.15 M parameters; 54.0 MB
AeroCast (Ch. 5)	Inference (Execute)	Single NVIDIA RTX A2000 (8 GB), batch size 1	0.1 ms per sample, well within a 100 Hz prediction loop
NanoBench (Ch. 6)	Data Acquisition	Crazyflie 2.1 (STM32F405, 168 MHz, 192 kB) with i7-12700H host running ROS 1 and a 12-camera Vicon system	Synchronized logging at 100 Hz; no GPU; typical desktop-class host

1.5 Assumptions, Scope, and Limitations

The contributions in this dissertation are developed under a common set of assumptions that define the scope of the work and the conditions under which the reported results are expected to hold. First, the dynamic obstacles considered throughout are non-cooperative and non-interactive, meaning that they neither share intent or trajectory information with the UAV nor alter their behavior in response to the UAV’s presence. Cooperative coordination protocols and interaction-aware obstacle models are therefore outside the scope of this study. Second, physical validation is conducted in a controlled indoor environment instrumented with a Vicon motion capture system, which supplies external six-degree-of-freedom pose at millimeter accuracy. The detection and perception front-end is consequently assumed to be available, and the dissertation concentrates on the uncertainty that arises in estimation, prediction, and platform behavior downstream of detection rather than on the detection problem itself. Outdoor flight, GPS-based estimation, and vision-based perception are not treated here and are identified as future extensions.

Third, all hardware experiments use a single nano-scale platform, the Crazyflie 2.1, and tra-

jectory planning is executed off-board on a host computer because the onboard processor cannot accommodate the planning workload in real time. Results obtained at the nano scale, particularly those concerning actuation nonlinearity, battery-coupled thrust, and estimator behavior, are specific to sub-50 g vehicles and are not assumed to transfer without modification to larger platforms. Fourth, the prediction and planning components are evaluated under single-obstacle conditions, and multi-agent settings that introduce inter-agent dependencies are left to future work. Within POF+MADER, obstacle motion is modeled with a constant-velocity Kalman formulation and Gaussian noise, an assumption appropriate for short-horizon forecasting but not for highly nonlinear or evasive maneuvers.

A further point of scope concerns integration. This dissertation treats uncertainty as a first-class quantity across the data, prediction, planning, and platform levels, and it delivers each of these components as a validated contribution. The unified, end-to-end onboard planner that fuses all four sources of uncertainty into a single system is not claimed as a delivered result. It is presented as the principal direction for future work in Section 7.3, and NanoBench is positioned as the platform-level characterization that such an integration would require. The detailed, contribution-specific limitations are stated within the respective chapters and consolidated in Section 7.2.

1.6 Thesis Structure

This dissertation is organized into the following seven chapters:

- *Chapter 1: Introduction* presents the motivation, specific technical challenges, research objectives, contributions, and the mapping between contributions and objectives.
- *Chapter 2: Background* reviews the theoretical foundations relevant to this dissertation: optimization-based multi-agent trajectory planning, Kalman filtering and probabilistic state estimation, deep learning sequence models (Transformers, RNNs, MDNs), uncertainty quantification in prediction, and UAV dynamics and control.
- *Chapter 3: POF-MADER* presents the Probabilistic Obstacle Filter and its integration with MADER and RMADER, addressing Objective 1 and the perception-uncertainty component

of Objective 3. Chapter 3 corresponds to the paper: “*POF-MADER: Trajectory Planner in Dynamic Environments with Improved Collision Avoidance*,” published in *IEEE Access*, vol. 13, 2025.

- *Chapter 4: SynTraG* presents the synthetic trajectory generator for non-cooperative dynamic obstacles, supporting Objective 2 and the behavioral-stochasticity component of Objective 3. Chapter 4 corresponds to the paper: “*SynTraG: A Synthetic Trajectory Generator for Non-Cooperative Dynamic Obstacles in UAV Navigation*,” published in *Advances in Automation and Robotics Research*, Springer, 2026.
- *Chapter 5: AeroCast* presents the Transformer-MDN probabilistic trajectory predictor, fulfilling Objective 2 and contributing calibrated behavioral uncertainty modeling for Objective 3. Chapter 5 corresponds to the paper: “*AeroCast: Probabilistic 3D Trajectory Prediction for Non-Cooperative Aerial Obstacles via Transformer-MDN Architecture*,” submitted to *IEEE Transactions on Aerospace and Electronic Systems*, 2026.
- *Chapter 6: NanoBench* presents the multi-task nano-quadrotor benchmark, addressing the platform-level uncertainty component of Objective 3. Chapter 6 corresponds to the paper: “*NanoBench: A Multi-Task Benchmark for Nano-Scale Quadrotors*,” submitted to *IEEE RA-L*, 2026.
- *Chapter 7: Conclusion* summarizes the key findings, states the limitations of each contribution, and outlines future research directions, including full end-to-end integration of perception, environmental, and behavioral uncertainties into a unified onboard uncertainty-aware planner.

Each of Chapters 3 through 6 is based on a peer-reviewed publication and is largely self-contained. Minor adaptations have been made to notation and cross-referencing for consistency within the dissertation. All datasets, code, and evaluation scripts are publicly available at the respective repository links cited in each chapter.

1.7 Related Publications

The peer-reviewed publications that form the core contributions of this dissertation are listed in Table 1.3. Additional publications completed during the doctoral program are listed in Table 1.4.

Table 1.3

Core Dissertation Publications

#	Title	Venue	Year	Status	Relation to Thesis
1	POF+MADER: Trajectory Planner in Dynamic Environments with Improved Collision Avoidance	IEEE Access, vol. 13	2025	Published	Chapter 3; Objective 1
2	SynTraG: A Synthetic Trajectory Generator for Non-Cooperative Dynamic Obstacles in UAV Navigation	Advances in Automation and Robotics Research, Springer	2026	Published	Chapter 4; Objective 2
3	AeroCast: Probabilistic 3D Trajectory Prediction for Non-Cooperative Aerial Obstacles via Transformer-MDN Architecture	IEEE Transactions on Aerospace and Electronic Systems	2026	Under Review	Chapter 5; Objective 2
4	NanoBench: A Multi-Task Benchmark for Nano-Scale Quadrotors	IEEE RA-L	2026	Under Review	Chapter 6; Objective 3

Table 1.4

Additional Publications During the Doctoral Program

#	Title	Venue	Year	Status
5	Coaxial Modular Aerial System and the Reconfiguration Applications	Proc. IEEE International Conference on Robotics and Automation (ICRA)	2023	Published
6	Autonomous Navigation and Mapping of Water Channels in a Simulated Environment Using Micro-Aerial Vehicles	Proc. International Conference on Robotics and Automation in Industry (ICRAI)	2023	Published
7	Autonomous Navigation and Mapping of Snake Robot for Urban Search and Rescue	Proc. International Conference on Robotics and Automation in Industry (ICRAI)	2023	Published

CHAPTER II: BACKGROUND AND LITERATURE REVIEW

2.1 Optimization-Based Trajectory Planning for UAVs

This section reviews the trajectory planning methods that form the algorithmic foundation for the collision avoidance framework developed in Chapter 3. The discussion proceeds from global and local planning paradigms through polynomial trajectory optimization to multi-agent coordination, establishing the technical context against which the contributions of this dissertation are positioned.

2.1.1 Global and Local Planning Paradigms

UAV path planning methods can be broadly divided into global and local approaches. Global planners such as A^* [30] and Rapidly-exploring Random Trees (RRT^*)[31] compute obstacle-free trajectories over a known map and can provide optimality or asymptotic optimality guarantees. However, these methods assume complete prior knowledge of the environment and can be computationally expensive in high-dimensional configuration spaces, making them less suitable for environments that change during execution [32].

Local reactive planners address this limitation by leveraging real-time sensor data to adjust trajectories online. The Dynamic Window Approach (DWA) searches the space of feasible velocity commands to select collision-free motions at each control cycle [32]. Artificial potential field methods generate repulsive forces around obstacles and attractive forces toward the goal, producing smooth reactive behavior at low computational cost. These methods respond quickly to environmental changes but operate with a limited look-ahead horizon, which can lead to myopic decisions, oscillatory behavior, or entrapment in local minima when navigating through dense or fast-moving obstacle fields [32].

2.1.2 Polynomial Trajectory Optimization

For UAVs that satisfy the differential flatness property, trajectory planning can be formulated as a constrained optimization problem over polynomial curve representations. In this setting, the objective is typically defined in terms of a smoothness functional, such as the integrated squared jerk or snap, subject to kinematic bounds on velocity and acceleration, boundary conditions imposed by

the initial and final states, and spatial constraints that restrict the trajectory to collision-free regions of the workspace [33]. By representing the trajectory as a sequence of piecewise polynomials, commonly parameterized using B-spline or Bernstein bases, the original continuous-time problem is transformed into a finite-dimensional optimization over the locations of the control points [34]. This formulation is well suited to solution by nonlinear programming methods and yields smooth, dynamically feasible trajectories that can be executed in real time on UAV platforms.

2.1.3 Multi-Agent Coordination and Collision Avoidance

In multi-agent settings, each UAV must plan trajectories that avoid not only static and dynamic obstacles but also the trajectories of other agents. Centralized approaches jointly optimize all agent trajectories but scale poorly with the number of agents. Decentralized methods allow each agent to plan independently while coordinating through shared trajectory commitments or velocity constraints. The Optimal Reciprocal Collision Avoidance (ORCA) framework [35], for instance, computes velocity-space half-planes that guarantee collision-free motion for each agent under the assumption that all agents share the same avoidance protocol.

Optimization-based decentralized planners such as MADER [2] and its communication-robust extension Robust-MADER [3] achieve collision avoidance through separating hyperplane constraints between the convex hulls of planned trajectory segments. Each agent broadcasts committed trajectory segments and incorporates the commitments of neighboring agents as constraints in its local optimization. EGO-Swarm [4] offers an alternative decentralized approach using gradient-based B-spline optimization with onboard sensing. These frameworks have demonstrated strong performance in cooperative multi-agent scenarios. However, they either assume perfect knowledge of dynamic obstacle trajectories or treat dynamic obstacles as quasi-static, reacting to their current positions without forecasting future motion. This limitation motivates the predictive extension developed in Chapter 3, which integrates probabilistic obstacle forecasts into the optimization formulation as time-varying constraints.

2.2 Probabilistic State Estimation and Filtering

Trajectory planning in dynamic environments requires accurate and timely estimates of obstacle states, including position and velocity, from noisy and incomplete sensor measurements. This section reviews the Bayesian filtering framework and its Kalman Filter instantiation, which underpin the obstacle tracking module in Chapter 3 and the onboard state estimation evaluated in Chapter 6.

2.2.1 Bayesian Filtering Framework

Bayesian filtering is a framework for estimating the state of a system when both the dynamics and the measurements are uncertain [36]. At each timestep, the filter alternates between two stages. In the prediction stage, the current state estimate is propagated forward through the process model to obtain a prior belief about the next state. In the update stage, that prior is corrected using the latest sensor observation through Bayes' rule to produce a posterior estimate. Repeating this predict-update cycle allows the filter to maintain a probabilistic description of the system state while accounting for both process noise and measurement noise.

2.2.2 The Kalman Filter

For linear systems with Gaussian noise, Kalman filter provides the minimum mean-squared error state estimate [37]. The filter represents the belief over the system state as a Gaussian distribution characterized by a mean vector and a covariance matrix. In the prediction stage, the mean is propagated forward through the state transition model, and the covariance is updated to account for the accumulation of process uncertainty. In the update stage, the prior estimate is corrected using the innovation, that is, the discrepancy between the actual measurement and the predicted measurement. The Kalman gain determines how strongly the correction should weight the observation relative to the model prediction, based on their respective uncertainties. After this update, the posterior covariance is generally smaller than the prior covariance which indicates the reduction in uncertainty gained from the new measurement.

The computational cost of the KF scales cubically with the state dimension as a consequence of the matrix inversions that are needed to compute the Kalman gain, but for the low-dimensional state

spaces such as for obstacle tracking (six-dimensional position and velocity), this cost of matrix inversion is negligible. In UAV applications, the tracked state often includes only position and velocity, making the filter computationally lightweight enough for real-time deployment. This is particularly useful when a vehicle must track several obstacles at once under onboard resource limits. In Chapter 3, this idea is used in a decentralized manner, where each agent runs a separate Kalman filter for every obstacle detected within its sensing range.

2.2.3 Extended Kalman Filter

The standard KF is based on linear dynamics and observation models which are not true for many practical systems. The Extended Kalman Filter (EKF) is the extension of the Kalman Filter’s framework in the case of nonlinear systems by linearizing the process and measurement models about the current state estimate using a first-order Taylor expansion [36]. The resulting Jacobian matrices take the place of the constant transition and observation matrices of the linear KF, and the predict-update recursion is identical to that of the linear KF. This linearization introduces approximation errors that may affect the performance in highly nonlinear systems but, for these simple reasons, the EKF is the most popular estimator in aerial robotics. The Crazyflie 2.1 onboard state estimator evaluated in Chapter 6 is based on an EKF with external position references on a 168 MHz Cortex-M4 processor at 100 Hz.

2.2.4 Multi-Step Prediction and Uncertainty Growth

Bayesian filters are not limited to one-step estimation; they can also be used to generate short-horizon forecasts by repeatedly applying the prediction step without incorporating new measurements. As the estimate is propagated forward in open loop, the predicted mean evolves according to the motion model and the covariance increases with each step. This growth in covariance reflects the intuitive fact that predictions become less certain as the horizon extends.

For motion planning, that uncertainty information is as important as the predicted trajectory itself. A planner that receives both the forecasted obstacle positions and their associated covariance can adapt its behavior according to confidence level. When the prediction is sharp, the planner can remain less conservative; when uncertainty grows, the planner can enlarge safety margins

accordingly. Chapter 3 uses this idea by embedding covariance-inflated safety constraints directly into the trajectory optimization process.

2.3 Deep Learning for Sequential Data

Trajectory forecasting depends on models that can learn temporal structure from ordered observations and use that structure to infer future motion. This section reviews the main deep learning architectures for sequential data that form the basis of the prediction framework presented in Chapter 5.

2.3.1 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are designed for sequence modeling by maintaining a hidden state that is updated over time as each new observation is processed [38]. Because the hidden state carries information from previous timesteps, the model can in principle use the full observation history when making predictions. This makes RNNs a natural starting point for trajectory modeling, where temporal dependencies are central.

In practice, however, standard RNNs are difficult to train over long sequences because of the vanishing and exploding gradient problem [39]. During backpropagation through time, gradients are repeatedly multiplied by the recurrent weights, which can cause them to decay toward zero or grow without bound as the sequence length increases. As a result, standard RNNs often struggle to capture dependencies that span long temporal horizons.

To address this issue, gated recurrent architectures were introduced. The Long Short-Term Memory (LSTM) network [40] augments the recurrent state with a cell state and three gates: input, forget, and output. These gates regulate what information is written, retained, or exposed, making it easier for the model to preserve useful context across many timesteps. The Gated Recurrent Unit (GRU) follows a similar idea with a simpler design based on reset and update gates. Since it merges the cell state and hidden state into a single representation, it uses fewer parameters than the LSTM while often achieving comparable performance [41].

Even with these improvements, recurrent models remain inherently sequential. The hidden state at time t depends on the hidden state at time $t - 1$, which prevents full parallelization across

the sequence. In addition, the effective path through which information must travel between distant timesteps still grows with their separation, which makes long-range temporal reasoning more difficult than desired [42].

2.3.2 Self-Attention and the Transformer Architecture

The Transformer architecture [42] was introduced to address these limitations by removing recurrence altogether and replacing it with self-attention. Instead of processing the sequence one element at a time, self-attention allows every position in the sequence to interact directly with every other position in a single layer. This makes it possible to model long-range dependencies without the sequential bottleneck of recurrent networks.

Given an input sequence of T embeddings, the self-attention mechanism maps each element into three learned representations: a query, a key, and a value. Attention scores are computed from the similarity between queries and keys, and these scores are then used to form weighted combinations of the value vectors. In this way, each timestep can gather information from the most relevant parts of the sequence, regardless of their distance in time. This formulation provides every token with direct access to every other token in the sequence, reducing the maximum information path length from $O(T)$ in recurrent models to $O(1)$ [42].

Multi-head attention extends this mechanism by running multiple parallel attention operations, each with independently learned projection matrices, and concatenating the results. This allows different heads to attend to different types of relationships, such as short-range kinematic continuity and longer-range maneuver precursors, within the same layer [42]. A standard Transformer encoder layer consists of a multi-head self-attention sublayer followed by a position-wise feedforward network, with residual connections and layer normalization applied around each sublayer. Stacking multiple such layers produces a deep encoder that can represent increasingly abstract temporal features.

2.3.3 Pre-Layer Normalization

The original Transformer applies layer normalization after each residual addition (Post-LN), which can lead to training instability in deep encoder stacks due to large variance in the residual

stream [43]. The Pre-LN variant relocates normalization to the interior of the residual branch, applying it to the input of each sublayer before the attention or feedforward computation. This reordering ensures that the residual connection carries an unnormalized signal, which stabilizes gradient flow and reduces sensitivity to learning rate and initialization choices [43]. Pre-LN has become the preferred configuration for training deep Transformer encoders without extensive hyperparameter tuning and is adopted in the prediction architecture described in Chapter 5.

2.3.4 Positional Encoding

Self-attention is permutation-equivariant: without additional information, the attention mechanism produces the same output regardless of the order in which tokens are presented. To encode temporal order, Vaswani et al. [42] introduced sinusoidal positional encodings, fixed vectors added to the input embeddings whose values are determined by sine and cosine functions of the position index at different frequencies. These encodings allow the model to distinguish temporal positions without introducing learnable parameters and have been shown to generalize to sequence lengths not observed during training [42]. Positional encoding is the sole mechanism by which a Transformer encoder preserves the temporal structure of the input, and its removal has been shown to significantly degrade prediction accuracy at longer horizons, as demonstrated empirically in Chapter 5.

2.4 Probabilistic Prediction and Mixture Density Networks

Deterministic trajectory forecasts collapse the distribution of possible future obstacle positions into a single predicted path. For safety-critical planning, this representation is insufficient: a planner that receives only a point forecast cannot distinguish between situations where the predicted trajectory is highly certain and situations where the obstacle may follow one of several plausible futures. This section reviews the probabilistic output representations and training methodologies that enable a prediction model to produce calibrated uncertainty estimates alongside its forecasts.

2.4.1 Gaussian Mixture Models

A Gaussian Mixture Model (GMM) represents a distribution as a weighted combination of K Gaussian components [44]. Each component is defined by its own mean, covariance, and mixture weight, with the weights constrained to be non-negative and to sum to one. This representation is useful because it can capture distributions that are not well described by a single Gaussian, including cases where probability mass is concentrated around several distinct regions. For trajectory prediction, those regions can be interpreted as different plausible future behaviors. In that setting, each mixture component corresponds to one possible motion mode, the covariance describes the uncertainty around that mode, and the mixture weight reflects its relative likelihood. Compared with a single confidence value, this gives a planner a more informative description of future uncertainty.

2.4.2 Mixture Density Networks

A Mixture Density Network (MDN) extends this idea by using a neural network to predict the parameters of a GMM instead of outputting a single deterministic estimate [45]. Given a learned feature representation, the output layer produces the mean, covariance-related terms, and mixture weights for each prediction step. In practice, the means are usually generated through a linear projection, the standard deviations are constrained to be positive through an exponential or similar activation, and the mixture weights are normalized with a softmax so that they form a valid distribution. One advantage of the MDN formulation is that it does not depend on a specific encoder. Any model that produces a compact representation of the observation history can be coupled with an MDN head, which makes this framework especially suitable for multimodal trajectory forecasting [46, 47].

2.4.3 Training with Proper Scoring Rules

MDNs are typically trained by minimizing the negative log-likelihood (NLL) of the observed target under the predicted mixture distribution. When averaged over samples and prediction steps, this objective encourages the model to assign high probability to the trajectories that actually

occur. NLL belongs to the class of strictly proper scoring rules, meaning that its expected value is minimized when the predicted distribution matches the true data-generating distribution [48]. This is an important property because it encourages the model to learn a full predictive distribution rather than collapsing toward a single average outcome, as often happens with mean-squared error objectives [48].

At the same time, optimizing NLL alone can lead to undesirable behavior in multi-component mixtures. A model may learn to spread several components broadly across the output space so that the target is covered, even if none of the components aligns closely with the actual trajectory. In practice, this can produce overly diffuse predictions that are statistically acceptable under NLL but less useful for downstream planning. To reduce this effect, Chapter 5 augments the main NLL objective with an auxiliary term that encourages the dominant component to remain closer to the ground-truth trajectory.

2.4.4 Calibration and Expected Calibration Error

In addition to being accurate, a probabilistic model should also be well calibrated. Calibration refers to the agreement between predicted confidence and observed frequency. For example, if a model assigns 80% confidence to a set of predictions, then roughly 80% of the corresponding ground-truth outcomes should fall within the stated confidence region [49]. This is different from pointwise accuracy. A model may predict the mean trajectory well and still produce uncertainty bounds that are consistently too narrow or too wide. Prior work has shown that modern deep networks are often overconfident, assigning stronger confidence to predictions than their empirical reliability justifies [49].

Expected Calibration Error (ECE) is a common scalar metric used to measure this mismatch [49]. It is computed by grouping predictions into confidence bins, comparing the predicted confidence in each bin with the observed frequency of correct outcomes, and then averaging the discrepancy across bins. An ECE of zero indicates perfect calibration. In trajectory prediction, this idea is adapted by checking whether the proportion of ground-truth displacements that fall inside predicted confidence ellipsoids matches the nominal probability assigned to those ellipsoids. For

that reason, ECE complements trajectory error metrics such as Average Displacement Error (ADE). A model may achieve a low ADE while still being poorly calibrated, in which case its uncertainty estimates may be unreliable for planning and safety-critical decision-making.

2.5 Trajectory Prediction for Dynamic Obstacles

Predicting the motion of dynamic obstacles is a key requirement for anticipatory planning. A planner cannot behave proactively unless it has some estimate of where surrounding agents are likely to move over the next few seconds. This section reviews trajectory prediction methods ranging from classical physics-based approaches to modern learning-based models, and then highlights the gaps in the aerial setting that motivate Chapters 4 and 5.

2.5.1 Classical and Model-Based Prediction

One common approach to obstacle prediction is to assume a simple motion model, such as constant velocity or constant acceleration, and propagate the current state forward in time accordingly [50]. When this formulation is combined with a Kalman Filter, it produces both a nominal future trajectory and an associated covariance that grows over the prediction horizon [36]. Related approaches fit low-order polynomials to recent observations and extrapolate them forward. These methods are attractive because they are simple, computationally efficient, and often adequate for short-horizon prediction on resource-constrained systems.

Their main weakness, however, is that they rely on fixed assumptions about how an obstacle will move. Constant-velocity models are poorly suited for behaviors such as turning, climbing, descending, or abrupt evasive motion, and polynomial extrapolation typically becomes unreliable once the prediction horizon extends beyond the data used for fitting. As obstacle behavior becomes more nonlinear or context dependent, the quality of these forecasts deteriorates quickly. This limitation is one of the main reasons data-driven prediction methods have become increasingly important.

2.5.2 Learning-Based Trajectory Forecasting

Over the last decade, learning-based trajectory prediction has progressed rapidly, largely due to work in autonomous driving and pedestrian behavior modeling. Early approaches often relied on recurrent models such as LSTMs, which learned temporal motion patterns directly from observed trajectory data [51]. Social-LSTM extended this idea by introducing pooling mechanisms that allow the model to account for nearby agents during prediction [52]. Later work pushed this further through graph-based models that explicitly represent pairwise or group interactions among agents [53, 54].

Transformer-based models have more recently demonstrated strong performance on trajectory prediction benchmarks by leveraging self-attention to capture long-range temporal dependencies without the sequential bottleneck of recurrent models [55]. Trajectron++ [46] combined graph-structured recurrence with conditional variational autoencoders to produce dynamically feasible and interaction-aware forecasts. Generative adversarial networks have also been employed to produce diverse trajectory samples that cover multiple plausible futures [56, 57]. These methods have achieved strong results in structured ground-domain settings such as pedestrian walkways, road intersections, and highway merging scenarios.

2.5.3 Multi-Modal and Probabilistic Prediction

A central requirement for safety-critical planning is that the predictor represent the multi-modal nature of obstacle behavior. An obstacle approaching a decision point may follow any of several plausible paths, and a predictor that collapses this distribution into a single trajectory discards information critical to safe planning. Probabilistic prediction methods address this by outputting distributions over future trajectories rather than point estimates. Conditional variational autoencoders [58, 59, 60] sample from a learned latent space to generate diverse trajectory hypotheses, and Mixture Density Networks parameterize per-timestep Gaussian mixture distributions that explicitly represent multiple modes with associated probabilities, as discussed in Section 2.4.

A recurring challenge in multi-modal prediction is calibration. Models that overpredict unlikely behaviors can cause planners to overreact to low-probability events, while models that underestimate

tail risks leave the planner exposed to rare but dangerous maneuvers [46]. End-to-end trajectory distribution prediction on occupancy grids and calibration-focused training objectives represent recent efforts to align predicted probabilities with empirical outcome frequencies. The calibration-oriented training protocol developed in Chapter 5 builds directly on these findings.

2.5.4 Prediction in the Aerial Domain

The majority of learning-based trajectory prediction research has focused on ground-domain scenarios, with datasets such as nuScenes [61] and Waymo Open Motion [62] providing large-scale training corpora for vehicle and pedestrian forecasting. These datasets constrain motion to planar or structured environments with lane geometry and social conventions that regularize behavior. The aerial domain presents fundamentally different challenges. Non-cooperative airborne obstacles such as birds, erratic drones, or unregistered vehicles move in three dimensions without lane structure or social norms to constrain their trajectories. Collecting real-world data for these scenarios is difficult due to the rarity of collision-prone encounters, ethical constraints on wildlife interaction, and safety hazards during flight operations. To the best of our knowledge, no widely adopted public dataset of non-interactive, non-cooperative three-dimensional aerial obstacle trajectories existed prior to this dissertation. This data gap limits the training and evaluation of learning-based predictors for UAV navigation and directly motivates the development of SynTraG (Chapter 4) and the hybrid real-and-synthetic training corpus used by AeroCast (Chapter 5).

2.6 Integration of Prediction with Planning

Accurate trajectory prediction alone is insufficient for safe navigation; the predicted information must be effectively consumed by the planning algorithm. This section reviews how obstacle forecasts are integrated into trajectory planners and the challenges that arise from this coupling.

2.6.1 Reactive versus Anticipatory Planning

Reactive planning methods respond to the currently observed state of dynamic obstacles without explicitly reasoning about how those obstacles will move in the near future. Their main advantage is simplicity: they are typically easy to implement and computationally efficient enough for fast

replanning. The drawback is that they leave very little temporal margin for avoidance. When obstacle motion is fast relative to the replanning rate, this can lead to abrupt maneuvers, oscillatory behavior, or even collisions [17].

Anticipatory planners address this limitation by incorporating predicted obstacle motion into the planning process. Instead of reacting only when a collision is about to occur, the vehicle can begin adjusting its trajectory earlier based on how the scene is expected to evolve. Prior work has shown that this predictive capability can substantially reduce collision rates relative to purely reactive baselines, with reported improvements ranging from 37 to 63 percent [18, 19]. In addition to improving safety, the smoother trajectories associated with anticipatory planning can also lead to better energy efficiency.

2.6.2 Constraint-Based Integration

One common way to combine prediction and planning is to convert predicted obstacle motion into constraints within a trajectory optimization problem. Under this strategy, the planner does not treat prediction as a separate module; instead, forecasted obstacle positions directly shape the feasible region of the optimization. Recent energy-based neural prediction methods [63], for example, generate multimodal forecasts over a lookahead horizon and translate them into constraint sets for Model Predictive Control (MPC), enabling proactive navigation in cluttered scenes.

A related line of work uses intent-aware prediction to provide the planner with several candidate future trajectories rather than a single forecast. In such approaches, vision-based tracking is combined with intent estimation so that the planner receives multiple hypotheses together with their relative likelihoods [9]. The resulting optimization can then favor trajectories that remain safe under the most plausible obstacle behaviors rather than committing to only one deterministic prediction.

2.6.3 Chance-Constrained and Robust Formulations

When uncertainty is modeled probabilistically, it can be integrated into planning more directly through chance-constrained formulations [64, 65]. Instead of requiring strict separation under every possible realization of obstacle motion, these methods enforce that the probability of collision

remains below a specified threshold. This gives the planner a more flexible mechanism for reasoning about risk than worst-case approaches, which often become overly conservative.

Another widely used strategy is scenario-based optimization [66]. In this setting, a finite set of obstacle trajectories is sampled from the predictive distribution, and collision-avoidance constraints are imposed for each sampled scenario. The original probabilistic problem is thus recast as a deterministic optimization over multiple realizations of possible obstacle behavior, after which standard optimization methods can be applied.

2.6.4 The Freezing Robot Problem

A known failure mode of prediction-aware planning is the freezing robot problem, where the planner becomes so conservative that it stops the vehicle rather than selecting a feasible motion [67]. This often occurs when multimodal predictions are enforced too rigidly. If every possible obstacle mode is treated as a hard exclusion region, the union of those regions can occupy much of the workspace and leave the planner with no viable path.

One way to reduce this effect is to weight collision cost by occupancy likelihood rather than treating all predicted modes equally [67]. Under this approach, the planner can tolerate small risks associated with unlikely modes while still strongly avoiding high-probability collision regions. Designing planners that remain both safe and sufficiently non-conservative is still an active research problem, and it is one of the key motivations for the probabilistic prediction-planning integration explored in this dissertation.

2.7 Quadrotor Dynamics, Control, and Nano-Scale Considerations

This section reviews the dynamic modeling, control structure, and state estimation foundations relevant to quadrotor UAVs, with particular emphasis on the additional constraints introduced at the nano scale. These topics provide the background needed for the hardware studies in Chapter 3 and for the benchmark design in Chapter 6.

2.7.1 Rigid-Body Dynamics

A quadrotor is commonly modeled as a rigid body with six degrees of freedom evolving on $SE(3)$. Its translational dynamics describe how the combined thrust generated by the four rotors, together with gravity and aerodynamic drag, determines the vehicle's linear acceleration in the world frame [68]. The thrust vector is produced in the body frame and mapped to the inertial frame through the rotation matrix, whereas drag is often modeled as a velocity-dependent term. The rotational dynamics follow Euler's rigid-body equation, which relates the torques induced by differential rotor thrust to angular acceleration and accounts for gyroscopic coupling through the interaction between angular velocity and angular momentum [68, 69]. This six-degree-of-freedom model underlies most work on quadrotor simulation, control design, and system identification.

2.7.2 Motor Models and Thrust Mapping

Each motor-propeller unit converts an input (e.g., an electrical signal) into aerodynamic thrust and reaction torque. The standard model approximates thrust as a quadratic function of rotor speed [70]. For platforms that provide pulse-width modulation (PWM) commands rather than direct rotor speed measurements, the mapping from PWM to thrust is usually obtained through calibration using static load cell data. The torque generated by each rotor is then expressed as proportional to its thrust, with proportionality constant determined by the propeller geometry. The relationship between individual motor thrusts and the resulting total body-frame force and torque is computed by a mixing matrix specified by the arm length and rotor configuration [68].

For nano-quadrotors powered by single-cell lithium polymer batteries, this motor model becomes more complicated because thrust degrades as a function of battery voltage. Over the course of a flight, the battery voltage drops from approximately 4.2 to 3.0 volts, and the maximum achievable thrust decreases nonlinearly over that interval. As a consequence, constant-parameter models identified at nominal voltage become less accurate as the battery discharges [71]. Chapter 6 explains this behavior through battery-drain hover trajectories that record synchronized voltage and kinematic measurements across the full discharge curve.

2.7.3 Cascaded PID Control

The most widely used control architecture for quadrotors is the cascaded proportional-integral-derivative (PID) controller [72]. In this architecture, the outer loop uses position and velocity errors to generate desired thrust and attitude commands, and the inner loop tracks those attitude commands using orientation and angular-rate feedback. This decomposition is effective because the attitude dynamics evolve on a faster timescale than the translational dynamics. The inner loop can therefore stabilize the vehicle rapidly, after which the outer loop regulates trajectory tracking. Because of its lightweight implementation, cascaded PID control is well suited to embedded platforms with limited onboard resources [73].

2.7.4 Geometric Control on SE(3)

Geometric control offers an alternative formulation that operates directly on $SO(3)$ and therefore avoids the singularities and ambiguities associated with Euler-angle representations [33]. In the Mellinger controller, the desired thrust direction is first determined from the position tracking objective, and a desired rotation is then constructed from that direction and tracked using a feedback law defined on the attitude manifold. This formulation is particularly attractive for aggressive maneuvers, where large rotations can violate the small-angle assumptions that are often implicit in PID-based controllers. In Chapter 6, both cascaded PID control and the Mellinger controller are evaluated on the Crazyflie 2.1 under matched experimental conditions in order to compare their tracking behavior.

2.7.5 Onboard State Estimation

Reliable state estimation is essential for stable closed-loop flight, but nano-scale aerial vehicles impose strict limits on estimator complexity. Nano-scale quadrotors such as Crazyflie 2.1, onboard state estimation is performed by a lightweight Extended Kalman Filter running on a 168 MHz Cortex-M4 processor with 192 kB of SRAM [73]. The estimator fuses measurements from a six-axis IMU with external position updates, which are typically provided by a motion-capture system, to estimate position, velocity, and attitude at 100 Hz. More computationally intensive

approaches, such as visual-inertial odometry pipelines used on larger UAVs, are generally not feasible on hardware in this class [74].

A meaningful evaluation of onboard estimation performance requires access to both the internal filter outputs and accurate external ground truth at synchronized timestamps. Without this pairing, it is difficult to separate estimator error from measurement noise or to compare estimator behavior across different configurations.

2.7.6 Nano-Scale Platform Constraints

Aerial vehicles with a takeoff mass below 50 g, are defined as Nano-quadrotors. These vehicles operate under physical constraints that differ in important ways from those of larger UAVs. First, their coreless DC motors exhibit nonlinear electromechanical behavior, including sensitivity to back-EMF and wear effects, that is not encountered in the brushless motors commonly used on heavier platforms [71]. Second, the aerodynamic regime associated with very small propellers introduces low-Reynolds-number effects, such as laminar separation and tip-vortex interactions, that are often neglected in standard quadrotor models [75]. Third, because these vehicles rely on a single-cell LiPo battery, the supply voltage declines continuously during flight, directly coupling battery state to thrust generation and violating the constant-parameter assumptions used in many simulation models [76].

These constraints mean that models, controllers, and estimators developed and validated on platforms weighing hundreds of grams to several kilograms cannot be assumed to transfer to the nano scale without re-characterization. The NanoBench dataset introduced in Chapter 6 is designed to support this re-characterization by providing the actuator-level, controller-level, and estimator-level data required to study nano-quadrotor dynamics under realistic operating conditions.

CHAPTER III: POF+MADER: TRAJECTORY PLANNER IN DYNAMIC ENVIRONMENTS WITH IMPROVED COLLISION AVOIDANCE

Abstract

Trajectory planning in dynamic environments is crucial for safe and efficient navigation of UAVs, particularly in scenarios involving unknown dynamic obstacles. The Trajectory Planner in Multiagent and Dynamic Environments (MADER) algorithm has been successfully used for this purpose; however, it assumes perfect knowledge of the future trajectories of the dynamic obstacles. In practical scenarios, the future trajectory of the obstacles may not be known in advance. To address this limitation, we propose POF+MADER, an enhanced version that integrates a probabilistic obstacle filter with adaptive prediction horizons based on obstacle velocity. Our approach integrates a decentralized real-time Kalman Filter-based trajectory estimation with MADER’s planning core, eliminating the need for perfect a priori knowledge. We validate our approach through simulations with sensor noise ($\sigma = 0.2 - 0.5m$) to simulate real-world localization uncertainty, with performance benchmarked against the MADER, Robust-MADER, and Ego-Swarm planner baselines. To further demonstrate its practical applicability, we conducted real-world experiments on the Crazyflie UAV platform, comparing its performance to the original MADER algorithm. Results show a 38.75% collision rate reduction in simulation and a 25% reduction in real-world experiments, without a significant increase in average navigation time. Furthermore, even in the presence of real-time sensor-induced noise in obstacle position measurements, our approach consistently outperforms the baseline planners, demonstrating its robustness in trajectory planning under perception uncertainty.

3.1 Introduction

Unmanned Aerial Vehicles (UAVs) have become increasingly prevalent in various applications where autonomous navigation in complex and dynamic environments is essential, such as surveillance, agriculture, package delivery, and disaster response [11, 12, 13]. A crucial aspect of UAV autonomous operation is trajectory planning, which ensures collision-free navigation through static obstacles (e.g., buildings, towers, tree canopies, and wires) and dynamic obstacles (e.g., pedes-

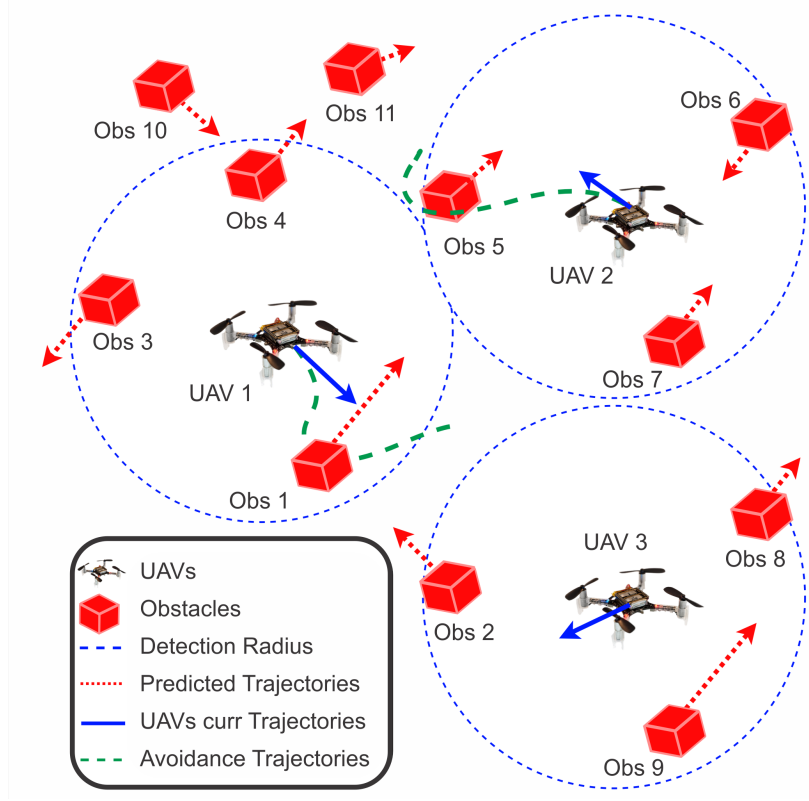


Figure 3.3

A typical multi-UAVs multi-dynamic obstacles scenario where the UAVs detect and estimate the future trajectories of dynamic obstacles.

trains, ground vehicles, balloons, other drones, or even birds). Trajectory planning is especially challenging when the future motion of obstacles is unknown or uncertain.

Dynamic obstacles encountered by UAVs can be broadly categorized as cooperative (sharing trajectory or intent information) or non-cooperative (offering no such information), and as interactive (adapting to the UAV's behavior) or non-interactive (moving independently) [10]. In practical scenarios, UAVs often face non-cooperative, non-interactive obstacles, such as pedestrians, vehicles, unregistered drones, or even birds, whose future trajectories are not available a priori and may be affected by sensor noise and environmental uncertainty.

Various approaches in the literature have been proposed to provide UAV trajectories that handle static obstacles, dynamic obstacles, and interactions with other agents[4, 77, 78]. Among these, MADER[2] and its communication-relaxed extension, Robust-MADER (RMADER) [3], have

demonstrated strong performance in handling multiagent systems to avoid dynamic obstacles. However, both approaches inherently assume that the future trajectories of dynamic obstacles are perfectly known to the planner. This assumption is rarely satisfied in real-world deployments. Therefore, during trajectory planning, considering the uncertainties of the dynamic obstacle’s maneuvers is important for efficiently avoiding collisions. The collision avoidance component is especially important when flying in dynamic environments, to consistently detect the position of dynamic obstacles and estimate their future trajectories, as depicted in Fig. 3.3.

To alleviate this limitation of MADER [2] and RMADER [3], in this work, we propose a Probabilistic Obstacle Filter (POF) coupled with the MADER and RMADER algorithms, based on the Kalman Filter (KF)[79] to estimate the future trajectories of non-cooperative, non-interactive dynamic obstacles in real-time. We call this extension POF+MADER. Unlike prior approaches that either require perfect knowledge [2, 3] or rely on computationally intensive learning-based predictors [80], POF+MADER adaptively predicts obstacle motion based on observed velocities and uncertainties, and enable robust, real-time trajectory planning on resource-constrained UAV platforms.

The main contributions of this chapter can be summarized as follows:

- *Real-time Dynamic Obstacle Prediction:* We introduce a decentralized lightweight probabilistic obstacle filter to track moving obstacles and extrapolate their future trajectories. This relaxes the restrictive assumption of perfect future knowledge in optimization-based trajectory planners such as MADER and RMADER. Our filter dynamically adjusts the prediction horizon based on observed obstacle velocities and incorporates uncertainty into the planning process.
- *Robust Collision Avoidance in Dynamic Environments:* We integrate the probabilistic obstacle filter with MADER’s trajectory optimizer to develop POF + MADER (or POF+RMADER if integrated with RMADER), which formulates the predicted obstacle states and their associated uncertainties as dynamic, time-varying constraints within the MADER optimization problem. POF-integrated methods transform the planners from being purely reactive to

proactively generating collision-free trajectories that account for most probable future trajectories of the dynamic obstacles. To comprehensively evaluate their robustness, these planners are tested against a representative suite of four motion models: straight-line for predictable paths, sinusoidal for oscillatory maneuvers, vertical wave for altitude variations, and trefoil-knot for complex, looped evasions. These canonical motion models serve as parametric approximations of challenging real-world behaviors, such as the flight patterns of birds or other non-cooperative drones. Across these scenarios, POF-integrated methods demonstrated significantly lower collision rates compared to the baselines.

- *Experimental Validation and Benchmarking:* We provide a comprehensive benchmark of our hybrid architecture in simulation, against the MADER, RMADER, and the state-of-the-art EGO-Swarm [4] baselines. Our analysis also considers sensor noise ($\sigma = 0.2 - 0.5m$) to quantify the system’s resilience to real-world perception uncertainty. Furthermore, we demonstrate the practical viability of our lightweight approach through successful hardware implementation on resource-constrained Crazyflie 2.1 UAVs, which shows its applicability beyond simulation.

The remainder of the chapter is organized as follows: Section 3.2 provides a literature review on multiagent trajectory planning considering obstacle avoidance. Section 3.3 provides problem formulation, including the assumptions and constraints associated with dynamic obstacles trajectory prediction. Section 3.4 describes our proposed method, focusing on the integration of our probabilistic filter with optimization-based planners to create a proactive trajectory planner. Section 3.5 presents the evaluation and results of our experiments, including a comparison with the baselines. Section 3.6 discusses the performance, robustness, and limitations of the proposed method. Lastly, Section 3.7 discusses the conclusion and future work.

3.2 Literature Review

Trajectory planning in dynamic environments remains a critical challenge for unmanned aerial vehicles (UAVs) [81], particularly when navigating alongside non-cooperative dynamic obstacles

with uncertain motion [82]. One way to avoid mid-air collisions with dynamic obstacles is to predict their short-term future trajectories [10]. Recent studies have presented several prediction techniques, including model-based [83, 84, 85], learning-based [86, 87], biomimetic [88, 89], and polynomial fitting [90, 91] methods. Model-based methods such as Kalman Filters (KF) estimate future states from past observations and provide fast, low-latency predictions [83, 92]. Learning-based methods, such as reinforcement learning, employ neural networks to capture complex and non-linear obstacle motions, but they require substantial training data and computational resources [93, 94]. Biomimetic methods replicate biological vision, offering quick responses using minimal visual information but usually require significant parameter tuning and lack predictive depth [95]. Polynomial fitting method utilizes polynomial calculations for prediction but is constrained to paths featuring regular and smooth curves [81]. Most of these approaches relied heavily on deterministic assumptions about obstacle trajectories, such as constant-velocity [96], linear acceleration, or an a priori knowledge of obstacles' future trajectories [2, 3], which limit their real-world applicability. For instance, the MADER[2] algorithm provides collision-free multi-agent planning using polyhedral obstacle representations, but assumes perfect a priori knowledge of the dynamic obstacle future trajectory. Non-cooperative obstacles, such as human-operated vehicles, pedestrians, or even birds, do not provide trajectory information to UAVs. Even cooperative obstacles may experience communication delays or failures that make their intended trajectories temporarily unavailable. Following this, RMADER [3], which is an extension of the original MADER algorithm, incorporates delay compensation mechanisms to achieve improved collision avoidance performance even when communication with other UAVs and agents is intermittent. The algorithm maintains computational efficiency while providing theoretical guarantees for collision avoidance under realistic communication constraints. However, it still assumes eventual access to obstacle future trajectories information.

Collision avoidance in multiagent systems often differs whether the method can avoid collision with static obstacles [97, 98], dynamic obstacles [99, 87], or among the agents only [100, 101]. Charbel et al. [102] have proposed a decentralized Model Predictive Control-based trajectory

planner that has better performance than MADER in terms of static obstacle avoidance and computation time, however, this approach does not explicitly incorporate dynamic obstacles as part of the collision avoidance scheme. Ego-Swarm [4] is another approach, an online decentralized and asynchronous trajectory planner implemented on both simulation and hardware, and includes capabilities for dynamic obstacle avoidance. In this chapter, we have considered it as a baseline for comparison. The Chance Constrained Rapidly-exploring Random Trees (CC-RRT) algorithm[103] is another method that avoids dynamic obstacles by conservatively constraining the likelihood of collisions, accounting for Gaussian noise inherent in both linear system dynamics and the translational predictions of dynamic obstacles, and then plans obstacle-free trajectories.

Trajectory planner for dynamic environments usually relies on sensing and detection technologies [104] that could provide real-time information about dynamic obstacles. Modern UAVs employ a variety of sensing modalities, each with distinct advantages and limitations for dynamic obstacle detection and tracking. Approaches can be broadly divided into vision-based detection [105] and map-based tracking [83]. Vision-based methods use cameras (RGB[105] or depth[106]) or other optical sensors to directly detect obstacles in the UAV's field of view. Some lightweight techniques[106][83] extract geometric cues (e.g., disparity from stereo or depth cameras) to locate obstacles and track them in real-time. For example, Liu et al. [107] has shown that lightweight object detection algorithms, can operate on UAV platforms to identify and track dynamic obstacles in real-time. Their approach combines visual object detection with Kalman filtering to estimate obstacle states and predict short-term trajectories. The limitation of purely vision methods is that they often cannot distinguish moving obstacles from static ones without temporal consistency checks [108]. Learning-based vision approaches (for example, neural network detectors)[109] are also used to classify and track obstacles, but these tend to be computationally heavy for small-scale UAV onboard use. The second category is map-based approaches[110][111][112], which integrate sensor data over time into a world model. For example, a voxel grid map where each cell is marked as occupied (obstacle) or free; by updating this map at high frequency, one can infer which cells have changed and thus detect moving objects[113]. Such methods separate static and dynamic

elements, but a drawback is that a voxel map alone does not provide an easy way to predict future motion of dynamic obstacles [83].

Even with good tracking, the future trajectories of dynamic obstacles remain uncertain [10]. The planner must therefore account for uncertainty to ensure robust collision avoidance. Amutha et al. [114] have presented a probabilistic framework that explicitly models uncertainty in obstacle motion and UAV dynamics to generate trajectories with probabilistic safety guarantees. Chance-constrained planning formalizes this idea by treating obstacle states as random variables and enforcing that the probability of collision remains below a specified threshold [115, 116]. Another approach is to use online learning [117] or adaptive modeling [118] to tighten the uncertainty bounds. Instead of assuming worst-case behavior (which can be overly conservative), the planner can learn typical obstacle behavior on the fly. For example, Zhou et al. [117] estimate intended control inputs of dynamic obstacles and compute less-conservative forward reachable sets, enabling a robust MPC to keep the ego trajectory outside those sets over the horizon.

The proposed POF+MADER approach addresses the limitations of the existing methods by providing a lightweight, theoretically grounded solution that combines the computational efficiency of Kalman filter-based prediction with the proven collision avoidance capabilities of established optimization methods (e.g., MADER, RMADER). Unlike existing approaches that require perfect a priori knowledge or extensive computational resources, our method enables practical deployment on standard UAV platforms while maintaining formal safety under realistic operational conditions with uncertain dynamic obstacles and limited computational resources.

3.3 Problem Formulation

Let $\mathcal{I} = \{1, \dots, N\}$ denote the set of N agents, with $\mathbf{p}_i(t) \in \mathbb{R}^3$ representing the position of agent $i \in \mathcal{I}$ at time t . Each agent is modeled as a rigid sphere $S_i \subset \mathbb{R}^3$ of radius $r_i > 0$, enclosing all possible orientations such that the collision geometry remains invariant to rotation. The perception range of agent i is defined as a sphere $S_{pi} \subset \mathbb{R}^3$ of radius $r_{pi} > 0$, centered at $\mathbf{p}_i(t)$, within which the agent can detect obstacles using onboard sensors.

3.3.1 Dynamic Obstacles and Partial Observability

Let $\mathcal{M} = \{1, \dots, M\}$ denote the set of M dynamic obstacles, with state vector $\mathbf{x}_m(t) = [\mathbf{q}_m(t)^\top, \dot{\mathbf{q}}_m(t)^\top]^\top \in \mathbb{R}^6$ for obstacle $m \in \mathcal{M}$, where $\mathbf{q}_m(t) \in \mathbb{R}^3$ is the true position and $\dot{\mathbf{q}}_m(t) \in \mathbb{R}^3$ is the velocity at time t . Obstacle positions are not globally known; instead, when an obstacle is within agent i 's perception range, the agent receives a noisy measurement of its position $\mathbf{z}_m(t)$, i.e., when

$$\|\mathbf{p}_i(t) - \mathbf{q}_m(t)\| \leq r_{pi} \quad \forall i \in \mathcal{I}, \forall m \in \mathcal{M}, \forall t \in [t_{\text{in}}, t_{\text{f}}],$$

where $[t_{\text{in}}, t_{\text{f}}]$ is the planning horizon, and

$$\mathbf{z}_m(t) = \mathbf{q}_m(t) + \mathbf{v}_m(t),$$

where $\mathbf{v}_m(t) \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ is zero-mean Gaussian sensor noise with covariance $\mathbf{R}$.

3.3.2 Probabilistic Forecasting and Trajectory Planning

Given this partial and uncertain information, the overall problem is to: (1) predict the obstacle's future trajectory and (2) plan a safe and efficient path for the agent.

3.3.2.1 Forecasting Objective

Upon detection, each agent i employs a probabilistic filter to estimate the obstacle's current state $\hat{\mathbf{x}}_m(t|t)$ and predict the future trajectory $\hat{\pi}_m(t|t) = \hat{\mathbf{q}}_m(t+k|t)_{k=1}^L$ over a horizon L . Let $\pi_m(t)$ represent the actual unobserved trajectory of obstacle m over $[t_{\text{in}}, t_{\text{f}}]$. The primary objective of the filter is to minimize the aggregate prediction error across all obstacles:

$$\min_{\{\hat{\pi}_m\}_{m=1}^M} \sum_{m=1}^M \int_{t_{\text{in}}}^{t_{\text{f}}} \|\pi_m(t) - \hat{\pi}_m(t)\|^2 dt \quad (3.1)$$

3.3.2.2 Planning Objective

The primary objective for each agent i is to compute a smooth, dynamically feasible trajectory $\tau_i(t)$ that reaches a goal while ensuring safety. This is typically formulated as a constrained optimization problem that minimizes control effort (e.g., integrated squared jerk):

$$\min_{\tau_i} \int_{t_{\text{in}}}^{t_{\text{f}}} \|\mathbf{j}_i(t)\|^2 dt$$

This optimization is subject to kinematic constraints (e.g., maximum velocity and acceleration) and the following collision avoidance constraints based on $\hat{\pi}_m(t)$.

3.3.2.3 Safety Constraints

To ensure safe navigation, the agent's planned trajectory $\tau_i(t)$ must remain separate from the predicted paths of other agents and obstacles.

- **Inter-Agent Collision Avoidance:** For any two distinct agents, their planned trajectories must be disjoint by a safety margin d_{agent} :

$$\bigcup_{i \in \mathcal{I}} \bigcup_{\substack{j \in \mathcal{I} \\ j \neq i}} (\tau_i(t) \oplus d_{\text{agent}}) \cap (\tau_j(t) \oplus d_{\text{agent}}) = \emptyset \quad \forall t \in [t_{\text{in}}, t_{\text{f}}] \quad (3.2)$$

where $d_{\text{agent}} > 0$ is the minimum inter-agent separation (typically $2r_i + \epsilon$).

- **Agent-Obstacle Collision Avoidance:** The agent must avoid the predicted occupancy of each dynamic obstacle. In other words, the agent's trajectory $\tau_i(t)$ must not intersect with the predicted obstacle trajectory $\hat{\pi}_m(t)$, inflated by a safety margin d_{safe} :

$$\bigcup_{i \in \mathcal{I}} \bigcup_{m \in \mathcal{M}} (\tau_i(t) \oplus d_{\text{safe}}) \cap \hat{\pi}_m(t) = \emptyset \quad \forall t \in [t_{\text{in}}, t_{\text{f}}] \quad (3.3)$$

where $d_{\text{safe}} > 0$ is the minimum safe separation (sum of agent radius r_i , obstacle radius r_m , and margin $\epsilon > 0$ for uncertainty), and $\oplus d_{\text{safe}}$ denotes the Minkowski sum inflating the trajectory by a sphere of radius d_{safe} .

3.4 Methodology and Integration

3.4.1 Dynamic Obstacle Definition and Model

In our framework, dynamic obstacles are defined as non-cooperative and non-interactive moving entities that may intersect with the agent's trajectory, and may pose collision risks. These obstacles do not share intent or trajectory information with the agents and do not alter their motion in response to the agents' behaviors. To evaluate the robustness of our approach across diverse real-world like scenarios, we model the obstacles' motions using four distinct parametric trajectories,

each representing varying levels of predictability and complexity. These models are implemented in both simulation and hardware experiments, with obstacle velocities scaled to challenge the prediction and avoidance capabilities.

The position of obstacle m at time t , denoted by $\mathbf{q}_m(t) \in \mathbb{R}^3$, is governed by one of the following equations, selected per experiment:

- ***Straight line***: This model represents linear, highly predictable trajectories common in open-field navigation or uniform motion. The position evolves as:

$$\mathbf{q}_m(t) = \mathbf{q}_m(t-1) + \dot{\mathbf{q}}_m(t-1) \cdot \Delta t \quad (3.4)$$

where $\dot{\mathbf{q}}_m(t-1) \in \mathbb{R}^3$ is the velocity at the previous timestep, and $\Delta t > 0$ is the discrete time step.

- ***Sinusoid wave***: This model captures oscillatory dynamics, mimicking evasive maneuvers or environmental perturbations (e.g., wind effects). The position at time t is defined by:

$$\mathbf{q}_m(t) = \begin{bmatrix} Bt \\ C \\ A \sin(\omega t + \phi) \end{bmatrix} \quad (3.5)$$

where $A > 0$ is the amplitude, $\omega > 0$ is the angular frequency, ϕ is the phase shift, B is a constant velocity component along the x -axis, and C is a constant offset along the y -axis.

- ***Trefoil-knot***: Inspired by parametric curves for looped, non-linear trajectories, simulating complex, repeating patterns. The position of the obstacle at time t is given by:

$$\mathbf{q}_m(t) = \begin{bmatrix} \sin(t) + 2 \sin(2t) \\ \cos(t) - 2 \cos(2t) \\ -\sin(3t) \end{bmatrix} \quad (3.6)$$

- ***Vertical wave***: This models vertical oscillations, relevant for altitude-varying obstacles like ascending/descending UAVs. The position of the obstacle at time t is given by:

$$\mathbf{q}_m(t) = \begin{bmatrix} x_0 \\ y_0 \\ A \sin(\omega t + \phi) \end{bmatrix} \quad (3.7)$$

where x_0 and y_0 are constant positions in the x and y directions, respectively, and A , ω , and ϕ are the amplitude, angular frequency, and phase shift of the sinusoidal motion in the z-direction.

3.4.2 Probabilistic Obstacle Filtering

We employ a decentralized Kalman Filter (KF) for real-time trajectory prediction of dynamic obstacles. Each agent runs an independent KF for every obstacle that enters its perception sphere S_{pi} of radius r_{pi} . This agent-centric approach scales with the number of agents and obstacles, as each agent independently manage predictions for obstacles in its local vicinity.

Algorithm 1 describes the implementation of the probabilistic obstacle filter, which takes the agent position $\mathbf{p}_i(t)$, obstacle position measurement $\mathbf{q}_m(t)$, the perception radius of the agent r_{pi} , and a minimum prediction length L_{min} . Once an obstacle m is detected within this predefined perception radius r_{pi} (i.e., $\|\mathbf{p}_i(t) - \mathbf{q}_m(t)\| \leq r_{pi}$), the tracking system initializes a dedicated KF specifically tailored to the detected obstacle's trajectory prediction (i.e., $\hat{\pi}_m(t) = \{\hat{\mathbf{q}}_m(t+1|t), \dots, \hat{\mathbf{q}}_m(t+L|t)\}$).

For each detected obstacle m at time t , POF maintains a state vector $\mathbf{x}_m(t) = [\mathbf{q}_m(t)^\top, \dot{\mathbf{q}}_m(t)^\top]^\top \in \mathbb{R}^6$, comprising the 3D position $\mathbf{q}_m(t) \in \mathbb{R}^3$ and velocity $\dot{\mathbf{q}}_m(t) \in \mathbb{R}^3$. We adopt a constant-velocity motion model, which is well-suited for short-term predictions in the absence of intent information from non-cooperative obstacles, and is commonly used in tracking applications. The discrete-time dynamics are given by:

$$\mathbf{x}_m(t+1) = \mathbf{A}\mathbf{x}_m(t) + \mathbf{w}_m(t) \quad (3.8)$$

where $\mathbf{w}_m(t) \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ is zero-mean Gaussian process noise with covariance matrix $\mathbf{Q} \in \mathbb{R}^{6 \times 6}$ that captures the unmodeled accelerations or environmental disturbances. The state transition matrix $\mathbf{A}$

Algorithm 1: Probabilistic Obstacle Filter

Data: $\bigcup_{i=1}^N \mathbf{p}_i(t)$, $\bigcup_{m=1}^M \mathbf{q}_m(t)$, perception range r_{pi} , minimum prediction length $L_{\min}$
Result: Predicted trajectories $\hat{\pi}_m(t)$ with covariances for all detected obstacles m

```

1 begin
2   foreach agent  $i \in \{1, \dots, N\}$  do
3      $D_i(t) \leftarrow \emptyset$ ;
4     foreach obstacle  $m \in \{1, \dots, M\}$  do
5       if  $\|\mathbf{p}_i(t) - \mathbf{q}_m(t)\| \leq r_{pi}$  then
6          $D_i(t) \leftarrow D_i(t) \cup \{m\}$ ;
7         Compute  $\dot{\mathbf{q}}_m(t)$  from  $\mathbf{q}_m(t)$ ;
8         Initialize KF with:  $\hat{\mathbf{x}}_m(t_{\text{in}}|t_{\text{in}}) = [\mathbf{q}_m(t)^\top, \dot{\mathbf{q}}_m(t)^\top]^\top$ ,  $\mathbf{P}_m(t_{\text{in}}|t_{\text{in}}) = \mathbf{P}_0$ ;
9       end
10    end
11    while  $|D_i(t)| > 0$  do
12      foreach obstacle  $m \in D_i(t)$  do
13        Perform KF prediction step:
          
$$\hat{\mathbf{x}}_m(t|t-1) = \mathbf{A}\hat{\mathbf{x}}_m(t-1|t-1),$$

          
$$\mathbf{P}_m(t|t-1) = \mathbf{A}\mathbf{P}_m(t-1|t-1)\mathbf{A}^\top + \mathbf{Q};$$

          Update with measurement  $\mathbf{z}_m(t) = \mathbf{q}_m(t)$  :
          
$$\mathbf{y}_m(t) = \mathbf{z}_m(t) - \mathbf{H}\hat{\mathbf{x}}_m(t|t-1),$$

          
$$\mathbf{S}_m(t) = \mathbf{H}\mathbf{P}_m(t|t-1)\mathbf{H}^\top + \mathbf{R},$$

          
$$\mathbf{K}_m(t) = \mathbf{P}_m(t|t-1)\mathbf{H}^\top\mathbf{S}_m(t)^{-1},$$

          
$$\hat{\mathbf{x}}_m(t|t) = \hat{\mathbf{x}}_m(t|t-1) + \mathbf{K}_m(t)\mathbf{y}_m(t),$$

          
$$\mathbf{P}_m(t|t) = (\mathbf{I}_6 - \mathbf{K}_m(t)\mathbf{H})\mathbf{P}_m(t|t-1);$$

          Compute adaptive horizon:
          
$$L \leftarrow \max\left(L_{\min}, \left\lceil \frac{\|\dot{\mathbf{q}}_m(t|t)\| \cdot T_{\text{plan}}}{d_{\text{step}}} \right\rceil\right),$$

          for  $k = 1$  to  $L$  do
14            Perform open-loop prediction:
              
$$\hat{\mathbf{x}}_m(t+k|t) = \mathbf{A}\hat{\mathbf{x}}_m(t+k-1|t),$$

              
$$\mathbf{P}_m(t+k|t) = \mathbf{A}\mathbf{P}_m(t+k-1|t)\mathbf{A}^\top + \mathbf{Q},$$

              
$$\hat{\pi}_m(t) \leftarrow \hat{\pi}_m(t) \cup \{\hat{\mathbf{q}}_m(t+k|t)\};$$

            end
          end
15        end
16      end
17      foreach obstacle  $m \in D_i(t)$  do
18        if  $\|\mathbf{p}_i(t) - \mathbf{q}_m(t)\| > r_{pi}$  then
19           $D_i(t) \leftarrow D_i(t) \setminus \{m\}$ ;
20          Terminate KF for obstacle  $m$ ;
21        end
22      end
23      Increment  $t$  to next timestep;
24    end
25  end
26 end

```

is:

$$\mathbf{A} = \begin{bmatrix} \mathbf{I}_3 & \Delta t \cdot \mathbf{I}_3 \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_3 \end{bmatrix}$$

where Δt represents the sampling interval, $\mathbf{I}_3$ the 3×3 identity matrix, and $\mathbf{0}_{3 \times 3}$ the zero matrix. In this formulation, we omit the control input term ($\mathbf{B}\mathbf{u}_m(t)$), as non-cooperative obstacles provide no such information; however, this can be extended for scenarios with partial observability of intent.

Sensor measurements yield noisy position observations $\mathbf{z}_m(t) = \mathbf{q}_m(t) + \mathbf{v}_m(t)$, where $\mathbf{v}_m(t) \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ and $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ is the measurement covariance which is tuned based on sensor specifications. The linear observation model is:

$$\mathbf{z}_m(t) = \mathbf{H}\mathbf{x}_m(t) + \mathbf{v}_m(t), \quad \mathbf{H} = [\mathbf{I}_3, \mathbf{0}_{3 \times 3}] \quad (3.9)$$

The KF recursively computes the minimum mean-squared-error estimate of $\mathbf{x}_m(t)$ and alternates between time prediction and measurement update steps. Starting from an initial state estimate $\hat{\mathbf{x}}_m(0|0)$ (e.g., from the first detection) and covariance $\mathbf{P}_m(0|0)$ (initialized with large diagonals to reflect high initial uncertainty), the prediction step propagates the prior:

$$\hat{\mathbf{x}}_m(t|t-1) = \mathbf{A}\hat{\mathbf{x}}_m(t-1|t-1), \quad (3.10)$$

$$\mathbf{P}_m(t|t-1) = \mathbf{A}\mathbf{P}_m(t-1|t-1)\mathbf{A}^\top + \mathbf{Q} \quad (3.11)$$

Upon receiving a measurement $\mathbf{z}_m(t)$, the update incorporates the innovation to yield the posterior:

$$\mathbf{y}_m(t) = \mathbf{z}_m(t) - \mathbf{H}\hat{\mathbf{x}}_m(t|t-1), \quad (3.12)$$

$$\mathbf{S}_m(t) = \mathbf{H}\mathbf{P}_m(t|t-1)\mathbf{H}^\top + \mathbf{R}, \quad (3.13)$$

$$\mathbf{K}_m(t) = \mathbf{P}_m(t|t-1)\mathbf{H}^\top\mathbf{S}_m(t)^{-1}, \quad (3.14)$$

$$\hat{\mathbf{x}}_m(t|t) = \hat{\mathbf{x}}_m(t|t-1) + \mathbf{K}_m(t)\mathbf{y}_m(t), \quad (3.15)$$

$$\mathbf{P}_m(t|t) = (\mathbf{I}_6 - \mathbf{K}_m(t)\mathbf{H})\mathbf{P}_m(t|t-1) \quad (3.16)$$

where $\mathbf{K}_m(t)$ is the Kalman gain, optimally blending prediction and measurement based on their relative uncertainties, and $\mathbf{I}_6$ is the 6×6 identity matrix. This formulation converges to accurate

estimates as observations accumulate, with the posterior covariance $\mathbf{P}_m(t|t)$ providing a measure of estimation confidence.

For trajectory forecasting, POF extends the filtered state forward over a prediction horizon L in the open-loop mode without assuming future measurements. The multi-step predictions are computed recursively:

$$\hat{\mathbf{x}}_m(t+k|t) = \mathbf{A}\hat{\mathbf{x}}_m(t+k-1|t), \quad k = 1, \dots, L \quad (3.17)$$

Initialized from the current posterior $\hat{\mathbf{x}}_m(t|t)$. The corresponding covariances propagate as:

$$\mathbf{P}_m(t+k|t) = \mathbf{A}\mathbf{P}_m(t+k-1|t)\mathbf{A}^\top + \mathbf{Q} \quad (3.18)$$

To optimize prediction relevance, we adaptively adjust L based on obstacle velocity magnitude $\|\dot{\mathbf{q}}_m(t)\|$:

$$L = \max \left(L_{\min}, \left\lceil \frac{\|\dot{\mathbf{q}}_m(t)\| \cdot T_{\text{plan}}}{d_{\text{step}}} \right\rceil \right) \quad (3.19)$$

where $L_{\min}$ is a minimum horizon, T_{plan} is the agent's planning timeframe, and d_{step} a spatial discretization step used for collision checks. This ensures longer predictions for faster obstacles and covers potential collision windows while minimizing computational overhead.

To manage multiple dynamic obstacles, each agent i dynamically maintains a set, $D_i(t)$, containing all obstacles currently within its perception sphere. An obstacle m is considered detected and added to this set if its distance to the agent is less than or equal to the perception radius r_{pi} . This condition can be expressed as a binary indicator:

$$y_{i,m}(t) = \begin{cases} 1, & \text{if } \|\mathbf{p}_i(t) - \mathbf{q}_m(t)\| \leq r_{pi} \\ 0, & \text{otherwise} \end{cases} \quad (3.20)$$

The set of detected obstacles for agent i at time t is therefore formally defined as:

$$D_i(t) = \{m \in M \mid y_{i,m}(t) = 1\} \quad (3.21)$$

An independent Kalman Filter is instantiated for each obstacle upon entering $D_i(t)$ and is terminated once it leaves. As long as an obstacle $m \in D_i(t)$, its corresponding KF is updated with new measurements at each timestep, and its trajectory is reforecasted according to (3.17); when $m \notin D_i(t)$ the filter is terminated and removed from memory.

The output of the probabilistic filter is the predicted trajectory, $\hat{\pi}_m(t)$, which is a time-ordered sequence of the obstacle's forecasted future positions over the adaptive horizon L . This sequence is formally defined as:

$$\hat{\pi}_m(t) = \{\hat{\mathbf{q}}_m(t|t), \hat{\mathbf{q}}_m(t+1|t), \dots, \hat{\mathbf{q}}_m(t+L|t)\}$$

Each position vector $\hat{\mathbf{q}}_m(t+k|t)$ in this sequence is extracted from the corresponding forecasted 6D state vector $\hat{\mathbf{x}}_m(t+k|t)$. This set of predicted positions is then passed to the optimization-based planner to be incorporated as dynamic, time-varying obstacle-avoidance constraints.

3.4.3 POF+MADER Integration

The integration of the Probabilistic Obstacle Filter (POF) with state-of-the-art optimization-based trajectory planners, such as MADER [2] or RMADER [3], forms the core of our proposed framework, denoted as POF+MADER (or POF+RMADER when extended to the robust variant). This architecture addresses the inherent limitation of assuming perfect a priori knowledge of dynamic obstacle trajectories by substituting real-time probabilistic predictions from POF into the planning pipeline. As illustrated in Fig. 3.4, the system comprises two interconnected modules: (1) decentralized detection and trajectory forecasting of obstacles entering an agent's perception sphere, executed via Algorithm 1; and (2) augmentation of the trajectory optimization process to incorporate these forecasts as dynamic constraints, detailed in Algorithm 2 and explained in this subsection. This coupling enables agents to generate feasible, collision-free polynomial trajectories that adapt proactively to uncertain obstacle motions, and eventually enhance safety without sacrificing computational tractability.

At the core of POF+MADER lies an extension of MADER's nonlinear optimization formulation, which generates smooth, dynamically feasible trajectories represented as piecewise polynomials in the MINVO basis [119]. For agent i , the original MADER problem minimizes a cost function that

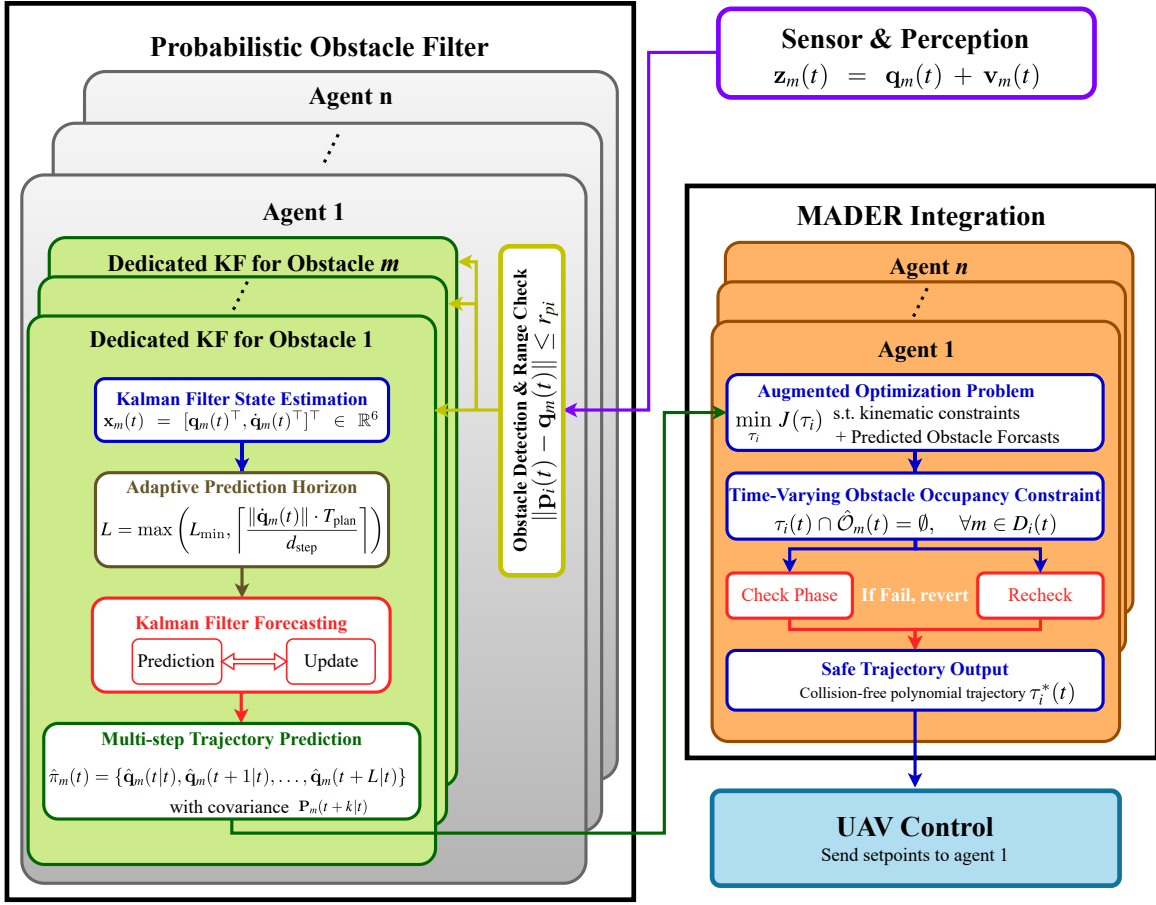


Figure 3.4

The POF+MADER system architecture; integrating decentralized probabilistic obstacle filter with optimization-based planners such as MADER [2].

includes a control effort term (proportional to squared jerk for cubic splines), weighted penalties on deviations from the goal, and adherence to kinematic limits. Specifically, for a clamped uniform B-Spline of degree $p = 3$ (cubic) with $n + 1$ control points $\{\mathbf{q}_0, \dots, \mathbf{q}_n\}^{\text{BS}}$ and knots defining intervals $j \in J$, the control effort is approximated as:

$$\int_{t_{\text{in}}}^{t_{\text{f}}} \|\mathbf{j}(t)\|^2 dt \propto \sum_{j \in J} \left\| \mathbf{Q}_j^{\text{BS}} \mathbf{A}_{\text{pos}}^{\text{BS}}(j) \begin{bmatrix} 6 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right\|_2^2$$

where $\mathbf{Q}_j^{\text{BS}}$ are the B-Spline control points for interval j , and $\mathbf{A}_{\text{pos}}^{\text{BS}}(j)$ is the interval-dependent

evaluation matrix. The full objective also includes a soft penalty on the final position: $\omega \|\mathbf{q}_{n-2} - \mathbf{g}\|_2^2$, with $\omega \geq 0$, as $\mathbf{q}_{n-2} = \mathbf{q}_{n-1} = \mathbf{q}_n$ under the final stop condition ($\mathbf{v}(t_f) = \mathbf{0}$, $\mathbf{a}(t_f) = \mathbf{0}$). Constraints encompass initial conditions ($\mathbf{x}(t_{\text{in}}) = \mathbf{x}_{\text{in}}$, fixing $\mathbf{q}_0, \mathbf{q}_1, \mathbf{q}_2$), bounds on velocity and acceleration using MINVO-transformed points ($|\mathbf{v}| \leq \mathbf{v}_{\text{max}}$ for $\mathbf{v} \in \mathcal{V}_j^{\text{MV}}$, $|\mathbf{a}_l| \leq \mathbf{a}_{\text{max}}$ for $l \in L \setminus \{n-1, n\}$), confinement to a planning sphere $\mathcal{S}$ of radius r centered at $\mathbf{d}$ ($\|\mathbf{q} - \mathbf{d}\|_2^2 \leq r^2$ for $\mathbf{q} \in \mathcal{Q}_j^{\text{MV}}$), and collision avoidance via separating planes π_{ij} (defined by normal $\mathbf{n}_{ij}$ and offset d_{ij}) between the agent's MINVO hulls $\mathcal{Q}_j^{\text{MV}}$ and other agents' committed polyhedra C_{ij} :

$$\mathbf{n}_{ij}^T \mathbf{c} + d_{ij} > 0 \quad \forall \mathbf{c} \in C_{ij}, \forall i \in I, j \in J \quad (3.22)$$

$$\mathbf{n}_{ij}^T \mathbf{q} + d_{ij} < 0 \quad \forall \mathbf{q} \in \mathcal{Q}_j^{\text{MV}}, \forall j \in J \quad (3.23)$$

The total planning time is heuristically set as $t_f - t_{\text{in}} = \|\mathbf{g} - \mathbf{d}\|_2 / v_{\text{max}}$.

In POF+MADER, we augment this formulation by introducing time-varying constraints derived from the predicted obstacle trajectories $\hat{\pi}_m(t)$ output by POF. For each detected obstacle $m \in D_i(t)$, the predicted positions $\hat{\mathbf{q}}_m(t + k|t)$ for $k = 1, \dots, L$ are inflated by a safety margin ϵ (accounting for UAV size, obstacle bounding box, and prediction uncertainty via covariance ellipsoids from $\mathbf{P}_m(t + k|t)$) to form restricted regions. These regions are approximated as convex polyhedra over discrete time intervals to ensure tractable inclusion in the optimization. The augmented problem becomes:

$$\min_{\tau_i} J(\tau_i)$$

s.t.

$$\begin{cases} \text{kinematic bounds and initial/final conditions,} \\ \tau_i(t) \cap \mathcal{O}_{\text{static}} = \emptyset, \quad \forall t \in [t_0, t_f], \\ \tau_i(t) \cap \tau_j(t) = \emptyset, \quad \forall j \neq i, \forall t \in [t_0, t_f], \\ \tau_i(t) \cap \hat{\mathcal{O}}_m(t) = \emptyset, \quad \forall m \in D_i(t), \forall t \in [t_0, t_f] \end{cases}$$

where $\mathcal{O}_{\text{static}}$ denotes static obstacle sets, $\pi_j(t)$ committed trajectories from other agents (broadcast in MADER's decentralized protocol), and $\hat{\mathcal{O}}_m(t)$ the inflated predicted occupancy of obstacle m , computed as the convex hull of $\hat{\mathbf{q}}_m(t) \oplus B(\epsilon + \sqrt{\lambda_{\text{max}}(\mathbf{P}_m(t))})$, with $B(r)$ a ball of radius r and

$\lambda_{\max}$ the maximum eigenvalue of the position submatrix of $\mathbf{P}_m(t)$ for uncertainty-aware buffering. This constraint enforces a minimum separation $d_{\text{safe}} = r_i + r_m + \epsilon$, where r_i and r_m are agent and obstacle radii. To ensure feasibility and robustness, POF+MADER employs a collision check-recheck mechanism, extending MADER’s two-phase validation. In the check phase, the optimized trajectory τ_i^* is discretized at fine temporal resolution (e.g., $\Delta t_c = 0.01$ s) and tested for intersections with $\hat{O}_m(t)$ using efficient separating hyperplane methods for polyhedral sets. Mathematically, for each time t_k and obstacle m , intersection is detected if no hyperplane separates the agent’s inflated position $\mathbf{p}_i(t_k) \oplus B(r_i)$ from $\hat{O}_m(t_k)$. If a potential collision is flagged, the recheck phase refines the test with higher fidelity (e.g., continuous-time overlap via root-finding on distance functions), confirming violations only if the minimum distance falls below d_{safe} :

$$\min_t d(\mathbf{p}_i(t), \hat{\mathbf{q}}_m(t)) < d_{\text{safe}}, \quad t \in [t_k - \delta, t_k + \delta]$$

where $d(\cdot, \cdot)$ is the Euclidean distance, and δ a local interval. This dual-phase approach mitigates false positives from discretization artifacts while maintaining real-time performance. For initial guesses, POF+MADER leverages an adapted version of MADER’s Octopus Search algorithm (a variant of A* tailored for B-Splines and dynamic constraints). This search generates candidate control points by sampling velocities satisfying $v_{\max}$ and $a_{\max}$, discarding nodes that violate linear separability with predicted obstacle hulls, sphere bounds, or voxel proximity checks. The resulting feasible path, closest to the goal if exact reachability fails, seeds the nonlinear solver, ensuring rapid convergence. Algorithm 2 encapsulates this process: predicted trajectories from POF serve as inputs, and the loop iteratively adjusts $\tau_i(t)$ (the committed segment of τ_i^*) upon collision detection. Unlike the original MADER, which reacts to known obstacle states, POF+MADER anticipates conflicts by leveraging forward-propagated estimates, enabling preemptive rerouting. For instance, if a fast-moving obstacle’s prediction indicates an impending intersection, the optimizer biases the solution toward evasive maneuvers, such as altitude changes or lateral deviations, while preserving trajectory continuity through soft penalties on deviations from prior commitments. In the RMADER variant, this integration further benefits from delay-aware broadcasting to ensure predictions remain synchronized across agents despite communication latencies.

Algorithm 2: POF+MADER Proactive Trajectory Planner

Data: Current agent state $\mathbf{x}_i(t_{curr})$, goal $\mathbf{g}_i$
Result: Collision-free trajectory segment τ_i^*

```
1 begin
  // Get latest forecasts from the filter
2  $\{\hat{O}_m(t)\}_{m \in D_i} \leftarrow \text{POF\_Module.getPredictions}();$ 
  // Find a feasible initial trajectory seed
3  $\tau_{seed} \leftarrow \text{A-star-Search}(\mathbf{x}_i(t_{curr}), \mathbf{g}_i, \{\hat{O}_m(t)\});$ 
  // Solve the augmented optimization problem
4  $\tau_i^* \leftarrow \text{NonlinearOptimize}(\tau_{seed}, \{\hat{O}_m(t)\});$ 
  // Validate the final trajectory for safety
5 if  $\text{isCollisionFree}(\tau_i^*, \{\hat{O}_m(t)\})$  then
6   return  $\tau_i^*$ ;
7 end
8 else
9   // Use previous trajectory
10  return  $\tau_i^{prev}$ ;
11 end
```

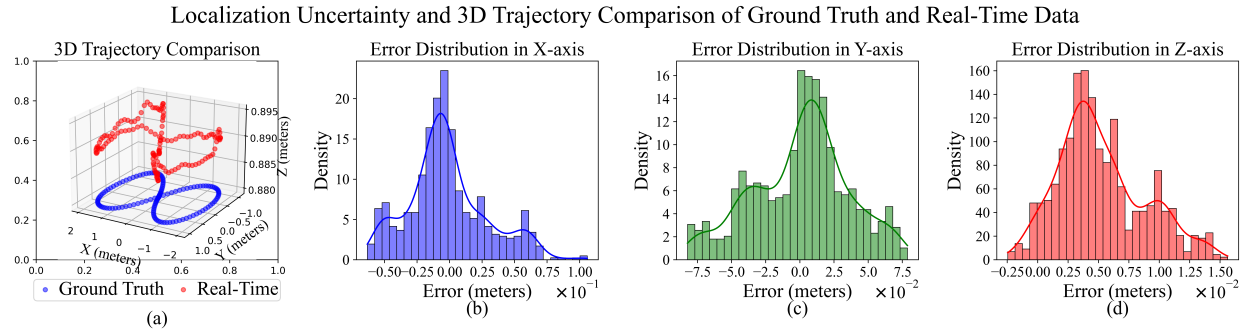


Figure 3.5
Visualization of localization uncertainty and 3D Crazyfly drone trajectory.

3.4.4 Localization Uncertainty in Obstacle Prediction

We conducted real-world experiments using a Crazyfly 2.1 drone within a motion capture system-equipped arena to characterize localization uncertainty in obstacle prediction. A predefined figure-eight trajectory was mathematically defined using a parametric function to represent the ground truth, and the drone was navigated along this path across five trials, as illustrated in Fig. 3.5(a). The deviation between the predefined positions and the real-time perceived positions,

recorded by the motion capture system, was computed to quantify localization errors. These empirical deviations, depicted in Fig. 3.5(b-d), provide a realistic estimate of localization uncertainty encountered during UAV navigation in dynamic environments.

To generalize this uncertainty, we model the observed position of a dynamic obstacle as corrupted by Gaussian noise:

$$\mathbf{q}_m^{\text{noisy}}(t) = \mathbf{q}_m(t) + \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad (3.24)$$

where:

- $\mathbf{q}_m^{\text{noisy}}(t) \in \mathbb{R}^3$ is the noisy observed position of obstacle m at time t ,
- $\mathbf{q}_m(t) \in \mathbb{R}^3$ is the true position of the obstacle,
- $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ represents a multivariate Gaussian noise distribution with mean vector $\boldsymbol{\mu}$ and a covariance matrix $\boldsymbol{\Sigma}$.

Empirical analysis of localization uncertainty across the X, Y, and Z dimensions yielded the following statistical parameters, derived from the motion capture data:

- **X-axis:** $\mu_x = 0.0232$ m, $\sigma_x^2 = 0.0197$ m²,
- **Y-axis:** $\mu_y = 0.0278$ m, $\sigma_y^2 = 0.0215$ m²,
- **Z-axis:** $\mu_z = 0.0053$ m, $\sigma_z^2 = 0.0034$ m².

These values reflect the mean and variance of position errors in each dimension. These empirically-derived parameters are then used to inform our Probabilistic Obstacle Filter (POF). Specifically, the covariance matrix $\boldsymbol{\Sigma}$ from our model is used to define the measurement noise covariance matrix, $\mathbf{R}$, within the Kalman Filter. By feeding the noisy position data $\mathbf{q}_m^{\text{noisy}}(t)$ as the measurement input $\mathbf{z}_m(t)$ and configuring $\mathbf{R}$ with our real-world data, the POF can accurately estimate the true state $\mathbf{x}_m(t)$ and produce robust future trajectory predictions $\hat{\pi}_m(t) = \{\hat{\mathbf{q}}_m(t|t), \hat{\mathbf{q}}_m(t+1|t), \dots, \hat{\mathbf{q}}_m(t+L|t)\}$.

3.5 Experimentation and Results

This section presents the experimental evaluation of the proposed approaches in both simulation and real-world environments. The evaluation includes five algorithms in simulation: the baseline MADER [2], POF+MADER, Robust MADER (RMADER) [3], POF+RMADER, and the Ego-Swarm [4] baseline. The real-world evaluation, due to hardware and resource constraints, focuses on MADER and POF+MADER only. All experiments were implemented in the Robot Operating System (ROS) framework, with physical trials conducted on Crazyflie 2.1 drones.

3.5.1 Experimental Scenarios

In both simulation and real-world tests, the agent was tasked with navigating from a predefined start location to a goal location while avoiding dynamic obstacles. Four representative dynamic obstacle motion models were considered as shown in Fig. 3.6, and described below:

- (a) **UAV encountering dynamic obstacles moving in the opposite direction along a straight line:** In this experiment, the obstacles move in a straight line. An agent was tasked to navigate in a way that it had to encounter all of the obstacles and avoid collisions.
- (b) **UAV encountering dynamic obstacles moving in the opposite direction along a sinusoidal wave motion:** The dynamic obstacles exhibit a sinusoidal wave motion, and an agent was tasked to navigate in a way that it has to face at least one dynamic obstacle.
- (c) **UAV encountering dynamic obstacles moving along a trefoil-knot trajectory:** In this experiment, an agent has to fly through multiple dynamic obstacles exhibiting trefoil-knot trajectory. The agent may or may not directly encounter obstacles.
- (d) **UAV encountering dynamic obstacles moving in a vertical wave motion:** In this experiment, an agent has to fly through obstacles oscillating vertically along the z-axis. The min-max of the z-axis for obstacle motion is set so that the probability of interaction with the agent is high.

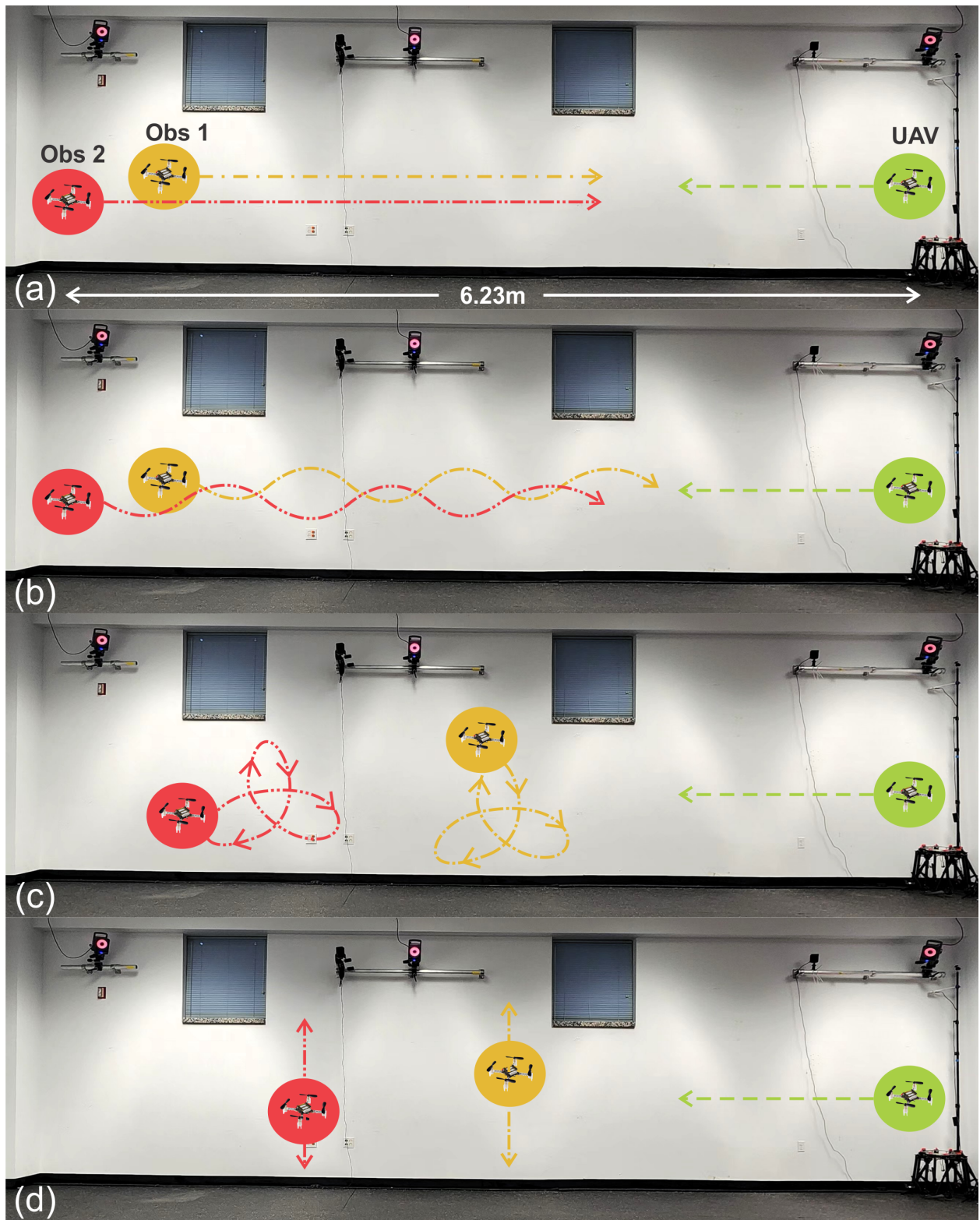


Figure 3.6
Real-world experimental arena equipped with VICON motion capture system.

In simulation, five dynamic obstacles were used; in hardware experiments, two obstacles were deployed due to space and safety constraints.

3.5.2 Evaluation Metrics

To ensure the statistical significance of the results, we employed the following evaluation metrics for all algorithms in both simulation and hardware:

- **Success rate (%age):** The ratio of runs in which the agents successfully navigate from their starting location to the target location without encountering any collisions.
- **Min obs separation (meter):** The mean of the shortest Euclidean distances between an agent to any obstacle, recorded in all the 10 runs.
- **Time within the collision (seconds):** The mean duration during which a collision between an agent and a dynamic obstacle continues to take place, recorded in all 10 runs.
- **Average navigation duration (seconds):** The mean duration for the agents to navigate from their starting location to their target location across all 10 runs.
- **Average velocity of the drone (meters/seconds):** The average velocity of the agent across all 10 runs in an experiment.
- **Number of collisions per 10 runs:** The total number of collisions in all 10 runs by the obstacles with an inflated agent.

3.5.3 Simulation Setup

The simulation experiments were conducted on a laptop with a 12th Gen Intel(R) Core(TM) i7-12700H CPU, running Ubuntu 20.04 and ROS Noetic. The agent was modeled as a sphere $S_i \in \mathbb{R}^3$ of radius 0.15 m, while obstacles were modeled as bounding boxes of size $0.3 \times 0.3 \times 0.3$ m. The agent parameters were set as: maximum velocity 10 m/s, maximum acceleration 20 m/s^2 , maximum jerk 30 m/s^3 , and perception range 10 m. The start–goal separation was 90 m to ensure that the agent encounters obstacles at its full cruising velocity.

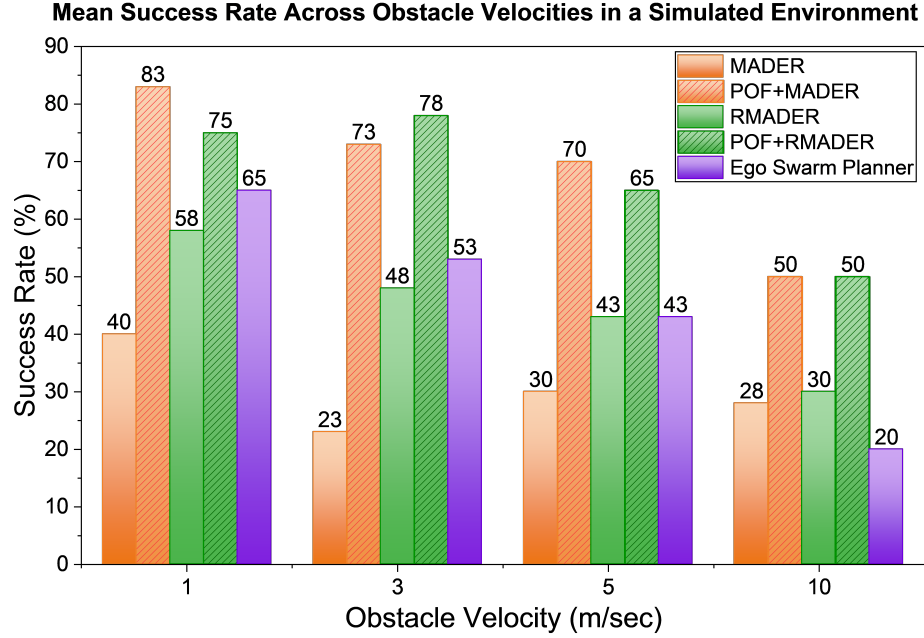


Figure 3.7

Success rate comparison of our approach (i.e., POF+MADER and POF+RMADER) and the baselines (i.e., MADER, RMADER, and Ego-Swarm) algorithms in a simulated environment.

Each algorithm was tested across four obstacle velocities (1, 3, 5, and 10 m/s) and four motion models. For each velocity–motion pair, 10 runs were executed. For instance, we executed 10 simulation runs where the obstacle followed a trefoil knot trajectory at a velocity of 1 m/s. We performed a total of 800 experimental runs in the simulation (i.e., 4 velocities $\times$ 4 motions $\times$ 10 runs = 160 runs per algorithm).

3.5.4 Simulation Results

The experimental setup, including dimensions, start and goal locations, and velocities of both obstacles and agents, was kept identical for all algorithms. The results, as presented in Table. 3.5 and Fig. 3.7, highlight the performance differentials between our approach (i.e., POF+MADER and POF+RMADER) and the baselines (i.e., MADER [2], RMADER [3], and Ego-Swarm [4]) algorithms.

Table 3.5

Performance differentials of our approach (i.e., POF+MADER and POF+RMADER) and the baselines (i.e., MADER, RMADER, and Ego-Swarm) algorithms, in a simulated environment.

Obstacle Motion Method	Success Rate (% age)				Min. Obstacle Separation (m)				Time Within Collision (s)				Avg Navigation Duration (s)				Avg Velocity of Drone (m/s)				Collisions per 10 Runs				
	1 m/s		3 m/s		5 m/s		10 m/s		1 m/s		3 m/s		5 m/s		10 m/s		1 m/s		3 m/s		5 m/s		10 m/s		
	30	0	20	20	0.65	0.55	0.47	0.39	0.07	0.07	0.11	0.10	13.32	13.24	13.32	13.21	6.19	6.82	6.77	6.81	9	11	10	9	
Straight line	MADER	30	0	20	20	0.65	0.55	0.47	0.39	0.07	0.07	0.11	0.10	13.32	13.24	13.32	13.21	6.19	6.82	6.77	6.81	9	11	10	9
	POF+MADER	90	80	80	50	0.73	0.73	0.73	0.62	0.02	0.04	0.06	0.05	13.21	13.03	13.02	13.19	6.83	6.88	6.81	6.79	1	2	3	6
	RMADER	50	30	30	0	0.55	0.41	0.50	0.30	0.49	0.27	0.49	0.74	17.72	17.05	17.61	17.43	5.20	5.40	5.22	5.27	11	16	19	34
	POF+RMADER	50	70	60	40	0.43	0.56	0.57	0.50	0.56	0.17	0.18	0.17	17.34	16.54	17.19	16.93	5.29	5.19	5.35	5.42	17	8	8	18
Sinusoid	ego planner	40	40	20	0	0.48	0.41	0.27	0.08	1.47	0.49	0.80	1.53	25.54	26.85	26.64	25.89	4.19	4.67	4.72	4.73	17	11	20	27
	MADER	10	20	0	0	0.60	0.56	0.32	0.29	0.10	0.12	0.19	0.07	13.03	13.02	13.44	13.07	6.88	6.90	6.74	6.87	14	11	15	25
	POF+MADER	70	70	50	40	0.70	0.72	0.71	0.67	0.05	0.03	0.06	0.18	12.97	12.98	13.21	13.42	6.35	6.33	6.87	6.74	4	3	6	9
	RMADER	40	30	30	20	0.50	0.45	0.36	0.45	0.41	0.23	0.55	0.62	17.36	16.85	17.41	17.43	5.31	5.46	5.29	5.29	17	16	24	27
Vertical wave	POF+RMADER	70	60	50	40	0.59	0.59	0.52	0.47	0.13	0.16	0.27	0.32	17.02	16.97	17.69	16.08	5.39	5.43	5.23	5.05	7	11	15	20
	ego planner	50	40	30	10	0.63	0.44	0.51	0.29	2.37	0.70	0.63	0.84	28.57	27.53	26.84	27.20	3.97	4.33	4.91	4.77	10	13	14	27
	MADER	50	30	40	30	0.69	0.46	0.63	0.46	0.17	0.25	0.17	0.15	13.26	12.96	12.96	12.96	6.80	6.92	6.93	6.83	8	15	7	10
	POF+MADER	80	70	80	40	0.87	1.27	1.17	0.76	0.09	0.12	0.11	0.11	13.22	13.50	14.12	13.30	6.79	6.70	6.47	6.35	2	6	2	8
Trefoil knot	RMADER	60	50	30	30	0.67	0.56	0.52	0.36	0.13	0.15	0.24	0.26	16.79	16.93	16.95	17.01	5.46	5.41	5.40	5.39	4	8	14	18
	POF+RMADER	80	80	60	50	0.72	0.85	0.77	0.59	0.05	0.04	0.13	0.08	17.01	17.13	16.95	17.23	5.36	5.32	5.40	5.30	2	2	6	7
	ego planner	70	50	30	20	0.84	0.59	0.44	0.43	0.94	0.23	0.31	0.19	27.20	27.68	28.75	25.14	2.37	2.46	2.41	2.40	3	9	13	16
	MADER	70	40	60	60	0.78	0.62	0.75	0.82	0.05	0.12	0.11	0.07	12.84	12.86	12.81	12.82	6.95	6.97	6.99	6.95	5	7	4	5
Trefoil knot	POF+MADER	90	70	70	70	0.77	0.76	0.78	0.82	0.11	0.08	0.07	0.04	12.81	12.98	12.86	12.98	6.96	6.69	6.94	6.91	1	3	3	3
	RMADER	80	80	80	70	0.86	0.72	0.81	0.66	0.09	0.06	0.05	0.03	16.73	17.00	16.68	16.60	5.47	5.39	5.48	5.51	3	4	3	3
	POF+RMADER	100	100	90	70	1.03	1.03	0.89	0.69	0.00	0.00	0.02	0.05	17.34	17.52	17.49	17.31	5.27	5.21	5.21	5.27	0	0	2	3
	ego planner	100	80	90	50	0.97	0.75	0.83	0.58	0.00	0.04	0.08	0.11	25.59	26.01	25.87	26.61	2.37	4.02	4.07	4.09	0	2	1	7

3.5.4.1 Comparison between MADER and POF+MADER

Across all motion models, POF+MADER substantially outperformed the baseline MADER [2]. The success rate improved from 30% to 68.75% (+38.75%), while the minimum agent–obstacle separation increased from 0.56 m to 0.8 m (+41.7%). Collision statistics show a reduction from 10.3 to 3.9 per 10 runs, and the mean collision duration decreased from 0.13 s to 0.07 s (−45.32%). These gains indicate that POF integration enables more conservative trajectory planning and ensures larger safety margins and drastically lowers collision risk.

The improvement, however, introduces a modest cost in efficiency. Average navigation time rose from 13.07 s to 13.18 s (+0.8%), with a corresponding decrease in average drone velocity. The effect is most apparent at higher obstacle velocities (≥ 3 m/s), where MADER tends to attempt direct paths toward the goal and suffers collisions, whereas POF+MADER actively adjusts to predicted obstacle motion.

3.5.4.2 Comparison between RMADER and POF+RMADER

The integration of POF also substantially improved the robustness of RMADER. Success rate improved from 44.38% in RMADER [3] to 66.88% (+22.5%), while minimum obstacle separation increased from 0.54 m to 0.68 m (+24.42%). Collisions, which averaged 13.8 per 10 runs in RMADER, were reduced to 7.9 under POF+RMADER, with collision duration reduced from 0.3 s to nearly 0.15 s. Importantly, these safety gains were achieved without increasing average navigation time or reducing mean velocity. This shows that POF integration strengthens RMADER’s robustness without compromising efficiency. This is particularly valuable in high-speed sinusoidal and trefoil-knot cases, where baseline RMADER experienced more failures.

3.5.4.3 Comparison between POF-based Methods and Ego-Swarm

The Ego-Swarm planner, which is optimized for aggressive, high-speed trajectories, struggled with reliability in our test scenarios mentioned in subsection 3.5.1, successfully completed only 45% of trials. Both POF+MADER and POF+RMADER outperformed in terms of safety and reliability, when compared to the Ego-Swarm [4] baseline. Ego-Swarm achieved an average navigation time

Table 3.6

Comparison of POF-based methods and the baselines (under localization uncertainty)

Obstacle Motion	Method	Success Rate (% age)				Min. Obstacle Separation (m)				Time Within Collision (s)				Avg Navigation Duration (s)				Avg Velocity of Drone (m/s)				Collisions per 10 Runs			
		1 m/s	3 m/s	5 m/s	10 m/s	1 m/s	3 m/s	5 m/s	10 m/s	1 m/s	3 m/s	5 m/s	10 m/s	1 m/s	3 m/s	5 m/s	10 m/s	1 m/s	3 m/s	5 m/s	10 m/s	1 m/s	3 m/s	5 m/s	10 m/s
Sinusoid	MADER	10	20	0	0	0.60	0.56	0.32	0.29	0.10	0.12	0.19	0.18	13.03	13.02	13.44	13.07	6.88	6.90	6.74	6.87	14	11	15	25
	POF+MADER	70	70	50	40	0.70	0.72	0.71	0.67	0.05	0.03	0.06	0.07	12.97	12.98	13.21	13.42	6.35	6.33	6.87	6.74	4	3	6	9
	RMADER	40	30	30	20	0.50	0.45	0.36	0.45	0.41	0.23	0.55	0.62	17.36	16.85	17.41	17.43	5.31	5.46	5.29	5.29	17	16	24	27
	POF+RMADER	70	60	50	40	0.59	0.59	0.52	0.47	0.13	0.16	0.27	0.32	17.02	16.97	17.69	16.08	5.39	5.43	5.23	5.05	7	11	15	20
	ego planner	50	40	30	10	0.63	0.44	0.51	0.29	2.37	0.70	0.63	0.84	28.57	27.53	26.84	27.20	3.97	4.33	4.91	4.77	10	13	14	27

(26.7 s) with mean velocity (3.8 m/s). Safety margins also improved markedly: from 0.53 m in Ego-Swarm to 0.8 m (+50.0%) in POF+MADER and 0.68 m (+26.5%) in POF+RMADER.

Ego-Swarm recorded the highest collision statistics, averaging 11.9 collisions per 10 runs with a mean collision duration of 0.67 s. In comparison, POF+MADER reduced collisions to 3.9 with an average duration of 0.09 s, while POF+RMADER reduced collisions to 7.9. These results indicate that Ego-Swarm’s aggressive trajectory generation favors speed but compromises safety. By contrast, POF-based methods achieve a more balanced outcome, trading slight increases in navigation time for substantial improvements in robustness. This makes them more appropriate for safety-critical UAV operations in dynamic environments.

3.5.4.4 Performance under Localization Uncertainty

To evaluate robustness against imperfect state estimation, we examined the impact of localization uncertainty on all algorithms under the sinusoidal motion model. This scenario was chosen for two reasons. First, both straight-line and vertical wave motions are predominantly confined to a single axis, which limits the complexity of their trajectories and makes them less susceptible to the effects of localization errors. In contrast, both trefoil-knot and sinusoid motions exhibit complex trajectories; however, the sinusoidal motion is characterized by continuous lateral oscillations and a less predictable periodicity, as opposed to the structured and repeating loops of the trefoil-knot. This intrinsic unpredictability in the lateral plane makes the sinusoidal motion particularly sensitive to localization uncertainty. Second, from the baseline results in Table 3.5, sinusoidal motion represented one of the most challenging cases, where baseline methods such as MADER and Ego-Swarm recorded their lowest success rates, whereas POF-based methods maintained significant improvements. This observation motivated our decision to further challenge POF-based methods

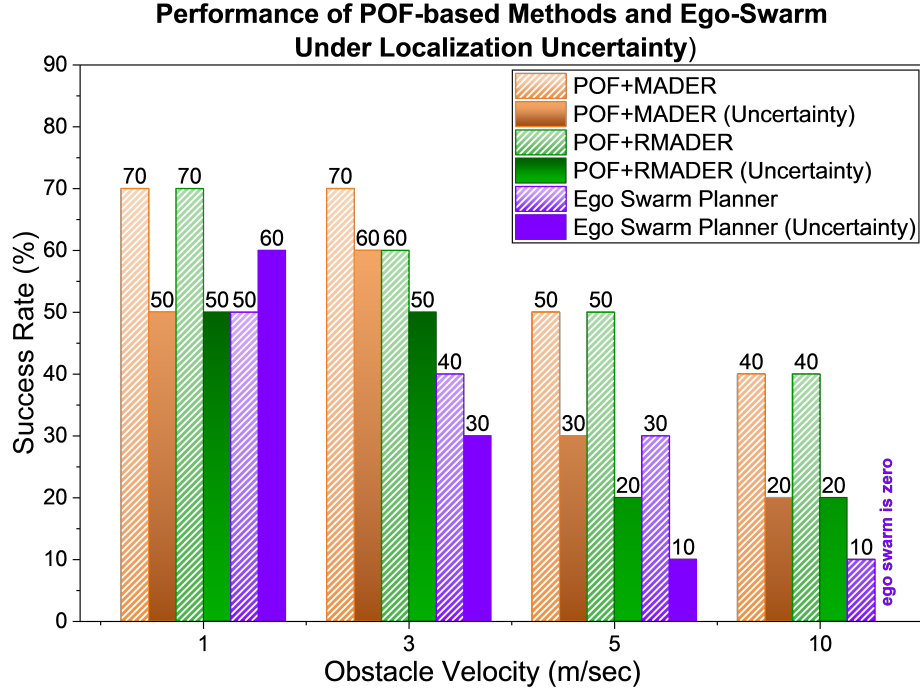


Figure 3.8
Success rate comparison of our approach (i.e., POF+MADER and POF+RMADER) and baselines (i.e., MADER [2], RMADER [3], and Ego-Swarm [4], under localization uncertainty in simulation.

by introducing localization uncertainty within the context of a sinusoidal obstacle trajectory.

The experimental setup remained identical to Section 3.5.3, with dynamic obstacles following sinusoidal paths at velocities of 1, 3, 5, and 10 m/s. At each velocity, 10 runs were conducted with localization uncertainty applied to obstacle positions. The comparative outcomes are summarized in Table 3.6.

As shown in Fig. 3.8 and Table 3.6, the results show that localization uncertainty degrades the performance of all algorithms, but to varying degrees. MADER and Ego-Swarm experienced the sharpest drops in success rate, confirming their limited resilience to state estimation errors. RMADER showed moderate robustness, but still suffered from non-negligible collision rates. In contrast, POF-integrated approaches remained consistently superior: POF+MADER retained a success rate 17.5% lower than in the nominal case yet still outperformed MADER by a wide margin, while POF+RMADER sustained the highest overall reliability with only minor reductions in performance. Importantly, both POF-based methods preserved significantly larger obstacle

separation distances than the baselines, which indicates that POF-integration substantially enhances robustness by maintaining conservative safety buffers and collision-free operation in most trials, under localization uncertainties.

3.5.5 Real-World Experiments

To validate implementation feasibility beyond simulation, we conducted hardware experiments using Crazyflie 2.1 UAVs in a $6.23 \times 3.6 \times 2.2$ m indoor arena equipped with a VICON motion capture system. In each experiment, one Crazyflie acted as the navigating agent while two additional Crazyflies emulated dynamic obstacles. Due to the limited onboard computation of the Crazyflie platform, trajectory planning was executed in real time on a host computer, with commands transmitted to the drones via Crazyradio PA. Reflective markers were mounted on all drones for motion capture, and the Crazyswarm library [73] was used to stream control inputs from the host to the UAVs. An illustration of the experimental setup is shown in Fig. 3.6.

The real-world evaluation was performed only on MADER and POF+MADER because of two main considerations: (i) MADER serves as the standard baseline in decentralized trajectory planning, which makes it the most appropriate reference for hardware validation; and (ii) POF+MADER is our proposed extension, enabling direct assessment of probabilistic filtering under realistic sensing, communication, and actuation conditions. With the hardware experiments, we want proof-of-concept feasibility and validation of POF integration in real-world on physical UAVs.

For the dynamic obstacles, we use the same four types of trajectories (i.e., straight line, sinusoid, vertical wave, and trefoil-knot trajectories), as shown in Fig. 3.6. For each trajectory, 10 runs were performed at obstacle velocities of 0.5, 1.0, and 1.5 m/s, resulting in 120 trials per algorithm. An experiment is considered successful if the agent navigates from their start location to the goal location without colliding with the obstacles. For safety purposes, the boundaries of the obstacles are inflated. Specifically, the actual diameter of the obstacle is 0.15 meters, and for collision avoidance, this was inflated to 0.3 meters (i.e., a 100% increase in size). A collision is considered to have occurred if an agent enters this inflated boundary, which ensures a safer buffer zone around

Table 3.7
Real-world Comparison of POF+MADER with MADER

Obstacle Motion	Method	Success Rate (% age)			Min. Obstacle Separation (m)			Time Within Collision (s)			Avg Navigation Duration (s)			Avg Velocity of Drone (m/s)			Collisions per 10 Runs		
		1 m/s	3 m/s	5 m/s	1 m/s	3 m/s	5 m/s	1 m/s	3 m/s	5 m/s	1 m/s	3 m/s	5 m/s	1 m/s	3 m/s	5 m/s	1 m/s	3 m/s	5 m/s
Straight line	MADER	50	40	0	0.18	0.15	0.14	0.96	1.29	1.81	13.60	13.53	14.39	0.61	0.62	0.63	5	6	11
	POF+MADER	90	80	70	0.30	0.27	0.24	0.19	0.41	0.49	13.38	13.67	13.42	0.62	0.61	0.62	1	2	4
Sinusoid	MADER	60	50	40	0.19	0.17	0.15	0.67	2.66	2.71	12.95	13.26	13.14	0.64	0.61	0.62	4	10	12
	POF+MADER	90	80	70	0.31	0.23	0.19	0.51	0.37	1.36	13.33	13.31	13.27	0.62	0.62	0.62	2	3	6
Vertical wave	MADER	20	10	10	0.16	0.12	0.13	2.14	3.06	3.07	13.08	13.42	13.70	0.59	0.61	0.61	9	10	12
	POF+MADER	60	50	40	0.19	0.22	0.16	1.02	1.66	1.90	13.07	13.68	13.50	0.58	0.61	0.61	4	5	8
Trefoil knot	MADER	80	60	70	0.26	0.22	0.30	0.41	1.43	0.92	13.60	13.57	13.32	0.60	0.61	0.62	2	5	3
	POF+MADER	80	80	80	0.43	0.37	0.46	0.51	0.48	0.88	13.11	12.60	13.05	0.61	0.63	0.63	2	2	2

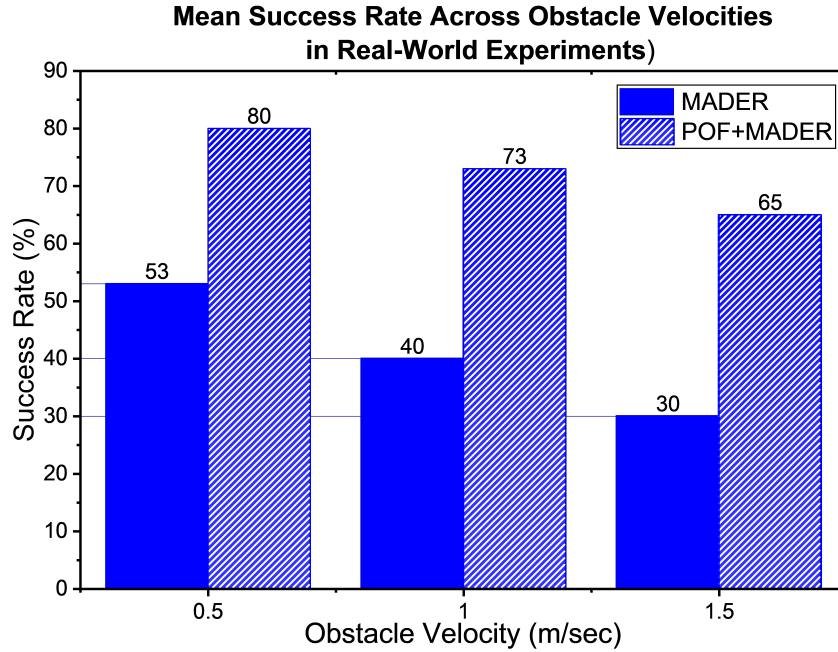


Figure 3.9
Performance evaluation of MADER and POF+MADER in a real-world environment.

the obstacle.

3.5.6 Real-World Results

Table 3.7 and Fig. 3.9 present the results. POF+MADER consistently achieved higher success rates, fewer collisions, and greater minimum separation than MADER, validating the safety improvements observed in simulation. For example, in sinusoidal motion at 1.5 m/s, POF+MADER achieved a 90% success rate compared to 65% for MADER.

3.6 Discussion

Our experimental evaluation, presented in Section 3.5, provided strong evidence that integrating a probabilistic obstacle filter (POF) into optimization-based planners like MADER and RMADER yields significant gains in robustness. This section moves beyond the quantitative results to interpret their deeper implications, discussing the fundamental advantages of our proactive planning architecture, the reasons for its resilience to uncertainty, and the practical considerations for real-world deployment.

3.6.1 The Advantage of Proactive Planning Under Uncertainty

A core insight from our comparative analysis is the fundamental advantage of proactive planning over purely reactive strategies. The baseline planners, particularly MADER and Ego-Swarm, perform adequately in simple scenarios but exhibit brittle performance as obstacle dynamics become more complex or perception becomes noisy. This is because their planning horizon is limited; they react to the current state of the world, leaving little time to generate smooth, safe maneuvers in response to fast-approaching or erratically moving obstacles.

In contrast, our POF-integrated methods succeed by effectively "looking into the future." POF provides the planner with a high-fidelity forecast of where obstacles are most likely to be, giving the optimizer a crucial "heads-up." This allows it to generate evasive trajectories that are not just reactive, but preemptive. The most compelling finding is the nature of the trade-off this provides: our results show nearly 39% improvement in success rate for a negligible 0.8% increase in navigation time. This demonstrates that the computational cost of filtering and prediction is far outweighed by the gains in safety and reliability, representing a highly favorable trade-off for any safety-critical application.

3.6.2 State Estimation as a Foundation for Robust Planning

The performance of our system under injected sensor noise reveals another key principle: robust planning is critically dependent on robust state estimation. The sharp decline in performance for the baseline planners under localization uncertainty highlights their sensitivity to noisy inputs. A

planner without a filtering layer is forced to react to every spurious measurement, which can lead to jerky, inefficient, or unsafe trajectories.

The resilience of POF+MADER and POF+RMADER stems directly from the POF’s ability to “see through the noise.” By maintaining a probabilistic belief over the obstacle’s state (including its velocity and covariance), the filter effectively smoothes the raw measurements and provides a stable, physically plausible estimate to the planner. The planner, therefore, operates on a clean, reliable stream of data, allowing it to make more consistent and accurate decisions. This confirms that for real-world robotics, where perfect perception is impossible, tightly coupling the planning and estimation processes is essential for achieving robust autonomy.

3.6.3 Limitations and Extensions

The combination of simulation and hardware results supports the feasibility of POF integration for real-world UAV navigation. Its lightweight nature makes this advanced predictive capability accessible to small aerial platforms without requiring expensive, high-power onboard computers. However, the current implementation operates under certain assumptions that define its scope and potential directions for extension.

3.6.3.1 Current Limitations

First, we employ a constant-velocity motion model that proved effective for non-cooperative obstacles in our experiments. This model supports computational efficiency and short-term prediction but complex maneuvers such as coordinated turns or evasive actions may require more advanced models.

Second, the Gaussian noise model effectively represents motion-capture and typical IMU-based localization uncertainties, but real-world sensors may exhibit non-Gaussian characteristics, including multi-modal distributions from vision-based detectors or heavy-tailed distributions from intermittent failures.

Third, hardware validation was conducted on Crazyflie 2.1 platforms in controlled indoor environments with limited dynamic obstacles. Although this provides compelling proof-of-concept feasibility, outdoor scenarios introduce GPS degradation, wind disturbances, and larger obstacle

populations that exceed this constrained operational envelope.

Based on these limitations, several extensions would broaden POF+MADER’s applicability to complex real-world deployments.

3.6.3.2 Multi-Agent Coordination Extensions

The current decentralized architecture treats each agent’s obstacle tracking independently. However, in scenarios where multiple agents have overlapping perception ranges, cooperative estimation strategies could substantially improve prediction accuracy. Distributed filtering approaches, such as consensus-based estimation in sensor networks [120], could be used to fuse obstacle measurements across agents to reduce uncertainty bounds. This collaborative sensing would be particularly valuable in GPS-denied environments where individual sensor reliability degrades due to occlusions.

Agents can further optimize performance by sharing learned obstacle behavior patterns via distributed databases. When one agent observes recurring motion characteristics (such as pedestrian crosswalk behavior or vehicle lane-following patterns), this information could propagate to other swarm members, enabling more accurate prediction than generic constant-velocity models provide.

3.6.3.3 Heterogeneous Obstacle Types Handling

Real-world environments contain diverse obstacle types with distinct motion characteristics. Integrating upstream obstacle classification would enable motion model selection tailored to obstacle type. For instance, pedestrian detection could invoke Social Force Models[121], which capture goal-directed behavior and social interactions, and vehicle detection would trigger car-following models such as the Intelligent Driver Model[122] that represent acceleration patterns and lane constraints.

To handle abrupt maneuver changes without explicit type information, Interacting Multiple Model (IMM) filters [123] could be used to maintain parallel motion hypotheses (constant velocity, coordinated turn, sudden stop) with dynamically updated probabilities based on measurement likelihood. This multi-hypothesis approach provides robustness to unpredictable behaviors such as birds’ evasive responses or vehicles’ emergency braking, common in outdoor scenarios. Additionally, extending obstacle representation from our spherical models to oriented bounding boxes

would enable tighter collision constraints for elongated obstacles and potentially reduce planning conservatism.

3.6.3.4 Outdoor Environment Adaptation

Outdoor deployment introduces systematic challenges addressable through targeted extensions. Wind disturbances violate constant-velocity assumptions over longer horizons; incorporating measured or estimated wind conditions into the process noise covariance (Q matrix) would maintain prediction fidelity and explicitly represent increased uncertainty in planning constraints. For navigation in GPS-denied areas, integrating POF with vision-based SLAM systems would require obstacle tracking and ego-motion estimation to be coupled.

These extensions build upon the modular POF+MADER foundation rather than requiring a fundamental redesign. Future work will prioritize heterogeneous classification and multi-model estimation to directly address current modeling limitations and enable outdoor validation.

3.7 Conclusions and Future Work

In this work, we presented a novel hybrid architecture, POF+MADER, that successfully bridges the critical gap between the need for proactive planning and the reality of perception uncertainty in dynamic environments. By integrating a lightweight, real-time probabilistic obstacle filter into the core of state-of-the-art multi-agent trajectory planners, we have eliminated their restrictive reliance on a priori knowledge of obstacle trajectories. The proposed approach transforms these planners into robust, predictive systems capable of generating safer paths by explicitly modeling and reacting to the forecasted motion of unknown dynamic obstacles.

Comprehensive validation through simulation and hardware experimentation shows the advantages of the proposed proactive POF-integrated methods. In simulation benchmarks, POF-based methods outperformed baseline methods in terms of safety and reliability, as demonstrated by collision rate reduction of up to 39% without compromising navigational efficiency. The practical feasibility of this lightweight architecture is then confirmed through its successful deployment on Crazyflie 2.1 UAVs. In these hardware trials, POF+MADER consistently outperformed MADER under realistic sensing and communication conditions, confirming the practical applicability of the

approach.

POF-based methods presents advantages, limitations, and areas for future direction, as outlined in subsection 3.6.3 such as integrating more advanced motion models and scaling the validation to larger and outdoor aerial platforms.

CHAPTER IV: SYNTRAG: A SYNTHETIC TRAJECTORY GENERATOR FOR NON-COOPERATIVE DYNAMIC OBSTACLES IN UAV NAVIGATION

Abstract

Unmanned Aerial Vehicles (UAVs) require robust navigation in dynamic environments to avoid collisions with non-cooperative obstacles like birds or other aerial objects, whose trajectories are non-linear, irregular, and not governed by road or air-traffic rules. Data-driven forecasting models such as Recurrent Neural Networks (RNNs) and Transformers, have shown strong performance in trajectory forecasting; however, progress in the UAV domain is constrained by the absence of publicly available data capturing such unstructured aerial motions. In this chapter, we present SynTraG, a parametric synthetic trajectory generator that produces a configurable and extensive corpus of 3D trajectories by mixing kinematic primitives that emulate common non-cooperative motions: linear flight, oscillatory weaving, complex looping maneuvers, and vertical undulations. The generator is formulated as a mixture of closed-form kinematic primitives with randomized parameters and explicit controls over path length, speed, acceleration, curvature, and spectral content. Heteroscedastic Gaussian noise is incorporated to model localization uncertainty (e.g., GPS/IMU errors) to enable probabilistic trajectory forecasting. We provide an open-source implementation of SynTraG and a sample dataset of 47,894 diverse trajectories, which enables robust training and benchmarking of forecasting models for UAV collision avoidance with non-cooperative dynamic obstacles. This work enables the development and standardized benchmarking of next-generation forecasting models for safer autonomous aerial navigation.

4.1 Introduction

The proliferation of Unmanned Aerial Vehicles (UAVs) in various applications such as surveillance, agriculture, package delivery, and disaster response, has increased the need for robust collision avoidance systems [12, 11, 111]. In unstructured environments, UAVs must handle dynamic obstacles with unpredictable, non-linear trajectories, such as birds or other aerial objects that do not cooperate with the ego-UAV, as shown in Fig. 4.10. Proactive avoidance requires an

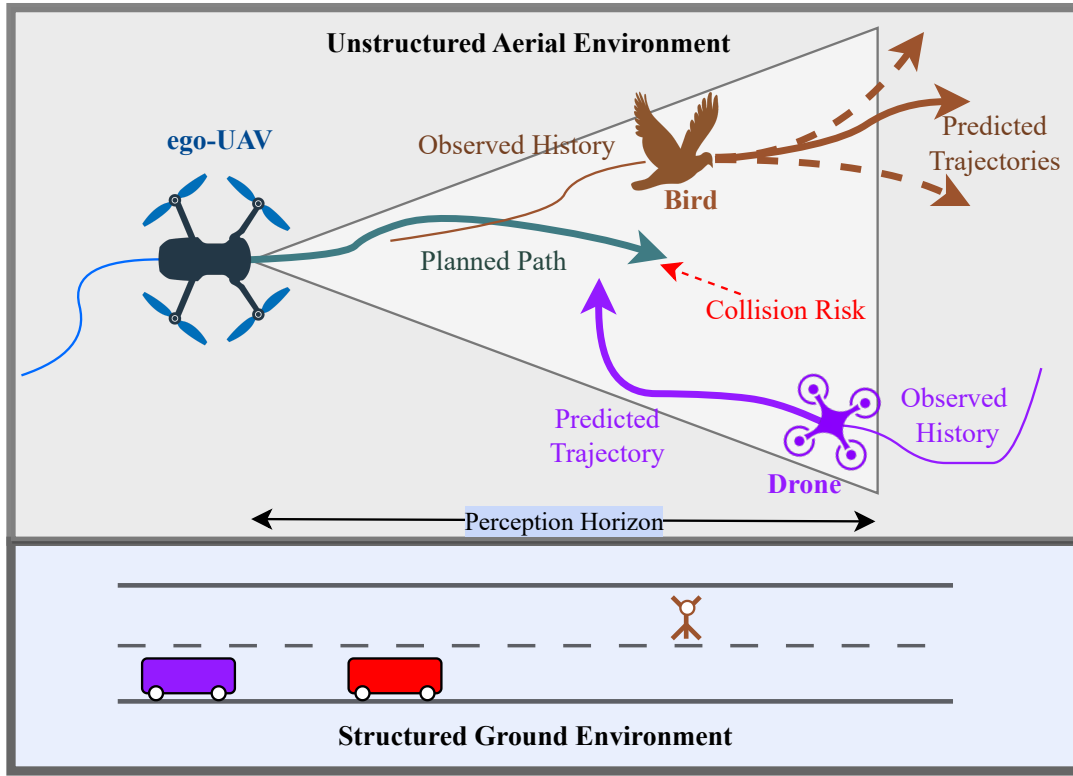


Figure 4.10

Illustration of the UAV navigation problem among non-cooperative dynamic obstacles.

accurate forecast of the future trajectory of these obstacles, which enables UAVs to adjust their trajectory preemptively.

Recent advances in deep learning, especially recurrent and Transformer-based sequence models, have shown strong performance in trajectory prediction for pedestrians [124, 125], road vehicles [61, 126], and social navigation [127]. However, these models require large and diverse training data to generalize across various motion patterns.

Existing datasets are focused on ground-based scenarios, such as vehicle or pedestrian tracking (e.g., nuScenes [61] or Waymo Open Motion [62]), which fall short for UAV contexts. Such datasets typically constrain motion to planar or interactive settings, leaving out the complex, erratic, and three-dimensional trajectories seen in bird movements or drone failures. Collecting real-world data for these obstacles presents significant challenges, including ethical issues in interacting with

wildlife, safety hazards during flight operations, and the rarity of such collision-prone events. To the best of our knowledge, there are no widely adopted datasets of *non-interactive, non-cooperative aerial* obstacle trajectories suitable for training forecasting models. As a result, synthetic dataset generation is a pragmatic and effective way to train such models.

We propose *SynTraG*, a parametric synthetic dataset generator to bridge this gap. It employs canonical kinematic equations with randomized parameters to generate diverse 3D trajectories that emulate non-interactive dynamic obstacles. These primitives were selected to form a basis set for complex aerial motion; straight-line motion represents predictable flight paths, sinusoidal and up-and-down patterns model periodic motions like flapping or gliding oscillations, and the trefoil knot serves as a challenging, high-curvature primitive representative of erratic or evasive maneuvers. For example, trefoil patterns capture the cyclic, knot-like maneuvers observed in bird flocks, which can be scaled and offset through parametric models to enhance realism. Furthermore, *SynTraG* also introduces localization uncertainty to enable risk-aware probabilistic trajectory forecasting for dynamic obstacles. To facilitate adoption, we have open-source the generator code and the data corpus, which comprises around 47,894 trajectory samples.

4.2 Related Work

4.2.1 Trajectory Forecasting

Trajectory forecasting for UAVs has gained significant advancement due to its critical role in autonomous navigation, collision avoidance, and air traffic management. Advances are largely driven by sequence modeling frameworks such as recurrent neural networks (RNN), transformers, and convolutional architectures designed for spatio-temporal data [128, 129]. For instance, QCNet-3D [130] introduces a UAV-specific predictor that models 3D motion through correlated axis-wise refinement and performs better than traditional 2D methods by addressing the unique complexities of aerial environments. Learning-based approaches increasingly integrate multimodal data from vision, LiDAR, and event sensors to improve long-horizon forecasting under adverse conditions [131]. Uncertainty quantification and probabilistic models are emerging to better capture the inherent stochasticity in UAV flight paths [132].

4.2.2 Existing Datasets for UAV Navigation

The development of UAV trajectory prediction methods depends strongly on access to high-quality datasets, which remain scarce in aerial domains. The MiTra dataset [133] leverages drone-collected traffic trajectories to characterize urban aerial movements across diverse traffic states. Similarly, the MMAUD dataset [134] incorporates multimodal sensor data (e.g., LiDAR, radar, and fisheye cameras) in urban UAV interception scenarios, enabling realistic evaluations in noisy and dynamic environments. Other datasets such as BuckTales [135] focus on multi-UAV tracking in wildlife habitats, capturing relatively predictable collective dynamics unsuitable for non-cooperative airborne obstacles. Datasets tailored to erratic or adversarial drones, or bio-inspired motions (e.g., birds), remain an important gap in the literature.

4.2.3 Synthetic Dataset for Motion Prediction

Synthetic trajectory generation has become a crucial tool for augmenting limited real-world UAV data, enabling more diverse and complex motion pattern training. Simulators such as AirSim [136] and SynDrone [137] enable configurable multi-agent UAV flights in varied environments but primarily focus on structured, cooperative scenarios. Transformer-based generative models, such as TrajGPT [138], advance controlled synthesis of realistic trajectories under learned spatio-temporal distributions. Probabilistic diffusion models such as DiffTraj [139] further support the creation of privacy-preserving GPS trajectories for improved forecasting pipelines. However, existing synthetic and real-world datasets remains limited in representing erratic, bio-inspired, or adversarial flight dynamics essential for collision avoidance in open airspace. Furthermore, existing generative models often learn from real-world data distributions that are unavailable for our target domain. SynTraG, therefore, adopts a first-principles, parametric approach of kinematically diverse and challenging aerial motions.

4.3 The SynTraG Framework

This section explains the mathematical modeling of the proposed *SynTraG* framework, explaining its key components for generating synthetic trajectories. This includes the definition of

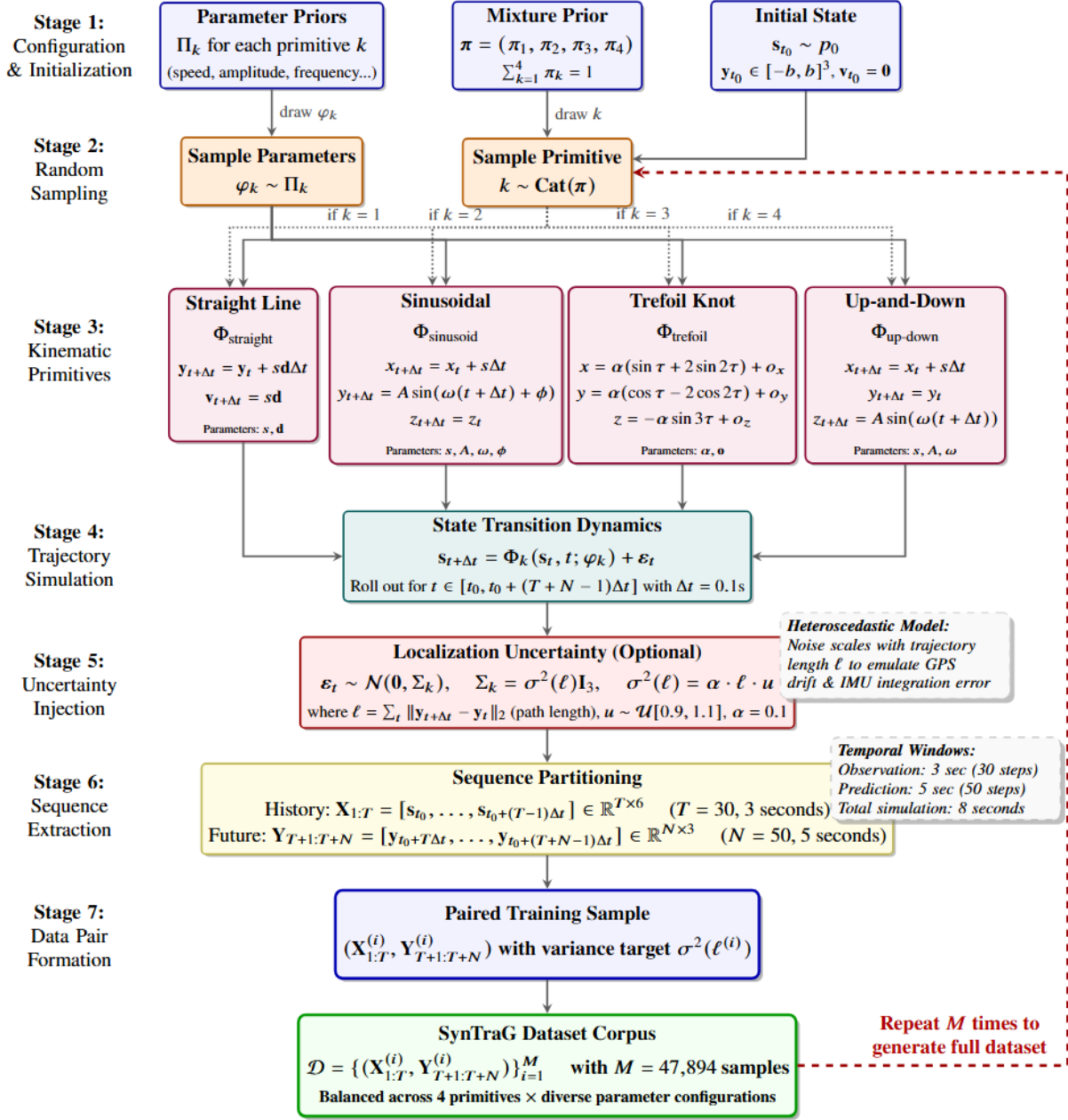


Figure 4.11
SynTraG generative pipeline architecture.

trajectory forecasting task, generator’s objective, mixture prior governing the selection of motion primitives, the conditional priors for primitive-specific parameters, the state-transition dynamics incorporating uncertainty modeling, and the kinematic equations for each motion primitive. Fig. 4.11 shows the overall system diagram of the framework.

4.3.1 Mathematical Formulation

4.3.1.1 Forecasting task

The trajectory forecasting problem is defined as given a sequence of historical states of a dynamic obstacle over a time horizon T , predict its future states over a subsequent time horizon N . The state at time t is defined as $\mathbf{s}_t = [x_t, y_t, z_t, v_{x,t}, v_{y,t}, v_{z,t}]^T \in \mathbb{R}^6$ representing the obstacle's state at time t , comprising its position $[x_t, y_t, z_t]^T \in \mathbb{R}^3$ and velocity $[v_{x,t}, v_{y,t}, v_{z,t}]^T \in \mathbb{R}^3$ in a 3D Cartesian coordinate system relative to a world frame. The input is a historical sequence of states $\mathbf{X}_{1:T} = [\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_T] \in \mathbb{R}^{T \times 6}$, where T is the length of the input sequence, corresponding to a time window of $(T \cdot \Delta t)$ seconds with a time step Δt seconds. The output is a forecast of future positions $\mathbf{Y}_{T+1:T+N} = [\mathbf{y}_{T+1}, \mathbf{y}_{T+2}, \dots, \mathbf{y}_{T+N}] \in \mathbb{R}^{N \times 3}$, where $\mathbf{y}_t = [x_t, y_t, z_t]^T \in \mathbb{R}^3$ denotes the 3D position coordinates of obstacle's trajectory at time $t = T + 1, \dots, T + N$, and N is the prediction horizon, spanning $(N \cdot \Delta t)$ seconds. A prediction model $f_\theta : \mathbb{R}^{T \times 6} \rightarrow \mathbb{R}^{N \times 3}$, with learnable parameters θ , maps $\mathbf{X}_{1:T}$ to $\hat{\mathbf{Y}}_{T+1:T+N}$.

4.3.1.2 Generator Objective

The goal of SynTraG is to synthesize a large set of paired sequences:

$$\mathcal{D} = \{(\mathbf{X}_{1:T}^{(i)}, \mathbf{Y}_{T+1:T+N}^{(i)})\}_{i=1}^M \quad (4.25)$$

consisting of M paired sequences, where each pair is derived from procedurally simulated trajectories. These trajectories are generated by sampling from a mixture of canonical motion primitives and rolling out closed-form kinematic equations under randomized parameters. The framework provides control over (i) the mixture weights across motion families (straight, sinusoid, up-and-down, and trefoil) and (ii) primitive-specific parameter distributions, enabling tunable balance and diversity to support robust model training and out-of-distribution (OOD) evaluation.

4.3.1.3 Mixture Prior over Motion Primitives

For each sample, a motion primitive index $k \in \{1, \dots, 4\}$ is drawn from a *categorical* distribution:

$$k \sim \text{Cat}(\boldsymbol{\pi}), \quad \boldsymbol{\pi} = (\pi_1, \pi_2, \pi_3, \pi_4), \quad \sum_{k=1}^4 \pi_k = 1 \quad (4.26)$$

Here, $\text{Cat}(\boldsymbol{\pi})$ governs the probabilities over the four primitives: `straight`, `sinusoid`, `trefoil`, `up_and_down`. A uniform prior ($\pi_k = 1/4$) ensures balanced generation, while skewed weights (e.g., setting $\pi_{\text{trefoil}} = 0$) can induce OOD scenarios, such as primitive hold-out during training.

4.3.1.4 Primitive-Specific Parameter Priors

Conditioned on k , a vector of motion parameters φ_k is drawn from a user-configurable prior Π_k :

$$\varphi_k \sim \Pi_k$$

Typically, Π_k factorizes into independent uniform distributions over primitive-specific parameters, controlling aspects such as amplitude, frequency, phase, offsets, and bounds on speed or acceleration. For instance, `sinusoid`: $A \sim \mathcal{U}[A_{\min}, A_{\max}]$, $\omega \sim \mathcal{U}[\omega_{\min}, \omega_{\max}]$, $\phi \sim \mathcal{U}[0, 2\pi]$; for `straight`: speed $s \sim \mathcal{U}[s_{\min}, s_{\max}]$ and direction $\mathbf{d} \in \mathbb{S}^2$). These priors shape trajectory length, speed/acceleration, curvature, and spectral content.

4.3.1.5 State Transition with Optional Uncertainty

Given (k, φ_k) , the state evolves according to discrete-time dynamics:

$$\mathbf{s}_{t+\Delta t} = \Phi_k(\mathbf{s}_t, t; \varphi_k) + \boldsymbol{\varepsilon}_t \quad (4.27)$$

where Φ_k is the deterministic kinematic mapping for primitive k , and $\boldsymbol{\varepsilon}_t$ is an optional noise term that models sensor inaccuracies or environmental perturbations. For uncertainty-aware generation, we employ zero-mean Gaussian noise:

$$\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \Sigma_k), \quad \Sigma_k = \sigma^2(\ell) I_3, \quad \sigma^2(\ell) = \alpha \ell \cdot u, \quad u \sim \mathcal{U}[u_{\min}, u_{\max}] \quad (4.28)$$

where ℓ is the sample’s path length (i.e., $\ell = \sum_t \|\mathbf{y}_{t+\Delta t} - \mathbf{y}_t\|_2$) and α is the scaling factor (e.g., $\alpha=0.1$) that controls the overall noise magnitude. u is a random variable drawn from a uniform distribution $\mathcal{U}[u_{\min}, u_{\max}]$ that introduces randomness to the variance for each trajectory to make them more diverse. This produces heteroscedastic variance targets compatible with probabilistic predictors.

4.3.1.6 Generative Program for One Sample

Let t_0 denote the start of the observation window. With an initial state $\mathbf{s}_{t_0}$ (sampled from a bounded prior over positions and zero initial velocity), the data pair $(\mathbf{X}, \mathbf{Y})$ is obtained by rolling out Eq. (4.27):

$$\begin{aligned} k &\sim \text{Cat}(\boldsymbol{\pi}), \quad \varphi_k \sim \Pi_k, \quad \mathbf{s}_{t_0} \sim p_0, \\ \mathbf{s}_{t+\Delta t} &= \Phi_k(\mathbf{s}_t, t; \varphi_k) + \varepsilon_t \text{ for } t \in [t_0, t_0 + (T+N-1)\Delta t], \\ \mathbf{X}_{1:T} &= [\mathbf{s}_{t_0}, \dots, \mathbf{s}_{t_0 + (T-1)\Delta t}], \quad \mathbf{Y}_{T+1:T+N} = [\mathbf{y}_{t_0 + T\Delta t}, \dots, \mathbf{y}_{t_0 + (T+N-1)\Delta t}] \end{aligned}$$

with $\mathbf{y}_t$ the position component of $\mathbf{s}_t$. Repeating this program M times yields the dataset $\mathcal{D}$, with ε_t ensuring probabilistic variance, if enabled.

4.3.1.7 Induced Mixture over Data Pairs

The generative process induces a probability distribution over data pairs $(\mathbf{X}_{1:T}, \mathbf{Y}_{T+1:T+N})$, modeled as a finite mixture of four components corresponding to the four kinematic primitives:

$$p(\mathbf{X}_{1:T}, \mathbf{Y}_{T+1:T+N}) = \sum_{k=1}^4 \pi_k \int p(\mathbf{X}_{1:T}, \mathbf{Y}_{T+1:T+N} \mid k, \varphi_k) d\Pi_k(\varphi_k) \quad (4.29)$$

Here, the mixture weights $\boldsymbol{\pi}$ control the balance between primitives, determining the relative frequency of each motion type in the dataset (e.g., more straight vs. trefoil trajectories). The parameter priors Π_k govern intra-primitive variability, allowing diverse trajectory characteristics (e.g., varying speeds or amplitudes within the sinusoidal primitive). This mixture structure ensures a rich and diverse dataset $\mathcal{D}$ which captures a wide range of non-cooperative obstacle behaviors. For evaluation, in-distribution (ID) protocols use the full mixture, while out-of-distribution (OOD) protocols can be configured by modifying $\boldsymbol{\pi}$ (e.g., setting $\pi_k = 0$ to exclude a primitive) or restricting Π_k 's support (e.g., limiting amplitude ranges to test generalization to unseen parameter).

4.3.2 Core Kinematic Motion Models

This subsection defines the kinematic equations for the four motion primitives in SynTraG: straight-line, sinusoidal, trefoil knot, and up-and-down. These primitives, implemented via the deterministic mapping $\Phi_k(\mathbf{s}_t, t; \varphi_k)$ in Eq. (4.27), generate diverse 3D trajectories that emulate non-cooperative aerial obstacles (e.g., birds or any other non-cooperative aerial object) within a bounded

workspace $[-\mathbf{b}, \mathbf{b}]^3 \subset \mathbb{R}^3$. The state $\mathbf{s}_t = [\mathbf{y}_t^\top, \mathbf{v}_t^\top]^\top \in \mathbb{R}^6$, comprising position $\mathbf{y}_t = [x_t, y_t, z_t]^\top$ and velocity $\mathbf{v}_t = [v_{x,t}, v_{y,t}, v_{z,t}]^\top$, evolves with a fixed timestep Δt . Each primitive is governed by parameters φ_k sampled from priors Π_k , as defined in Section 4.3.1.3 that ensures variability in trajectory characteristics such as length, speed, and curvature. These models are designed to capture realistic motion patterns, from linear flights to complex, bio-inspired maneuvers, supporting robust trajectory forecasting for UAV collision avoidance.

4.3.2.1 Straight-Line (Constant Velocity) Motion

This primitive models linear, non-maneuvering motion, such as a drone's steady flight or a bird's gliding path. The state evolves as:

$$\mathbf{y}_{t+\Delta t} = \mathbf{y}_t + \mathbf{s}\mathbf{d}\Delta t \quad (4.30a)$$

$$\mathbf{v}_{t+\Delta t} = \mathbf{s}\mathbf{d} \quad (4.30b)$$

where speed $s \sim \mathcal{U}[s_{\min}, s_{\max}]$ controls the magnitude of motion, and direction $\mathbf{d} \in \mathbb{S}^2$ (a unit vector on the 2-sphere) defines the 3D orientation. Parameters: $\varphi_{\text{straight}} = \{s, \mathbf{d}\}$. Acceleration is zero ($\mathbf{a}_t = \mathbf{0}$), reflecting constant velocity.

4.3.2.2 Sinusoidal Motion

This captures lateral oscillatory motion and approximate bird-like weaving in the y -direction. The state updates are:

$$x_{t+\Delta t} = x_t + s\Delta t \quad (4.31a)$$

$$y_{t+\Delta t} = A \sin(\omega(t + \Delta t) + \phi) \quad (4.31b)$$

$$z_{t+\Delta t} = z_t \quad (4.31c)$$

$$v_{x,t+\Delta t} = s \quad v_{y,t+\Delta t} = A\omega \cos(\omega(t + \Delta t) + \phi) \quad v_{z,t+\Delta t} = 0 \quad (4.31d)$$

with speed $s \sim \mathcal{U}[s_{\min}, s_{\max}]$, amplitude $A \sim \mathcal{U}[A_{\min}, A_{\max}]$, frequency $\omega \sim \mathcal{U}[\omega_{\min}, \omega_{\max}]$, and phase $\phi \sim \mathcal{U}[0, 2\pi]$. Parameters: $\varphi_{\text{sinusoid}} = \{s, A, \omega, \phi\}$. Acceleration: $a_{y,t} = -A\omega^2 \sin(\omega t + \phi)$.

4.3.2.3 Trefoil-Knot Motion

This models complex, looping 3D maneuvers, inspired by erratic bird flock patterns or evasive drone behaviors. Let $\tau = t + \Delta t$. The state evolves as:

$$x_{t+\Delta t} = \alpha(\sin \tau + 2 \sin 2\tau) + o_x \quad (4.32a)$$

$$y_{t+\Delta t} = \alpha(\cos \tau - 2 \cos 2\tau) + o_y \quad (4.32b)$$

$$z_{t+\Delta t} = -\alpha \sin 3\tau + o_z \quad (4.32c)$$

$$v_{x,t+\Delta t} = \alpha(\cos \tau + 4 \cos 2\tau) \quad (4.32d)$$

$$v_{y,t+\Delta t} = \alpha(-\sin \tau + 4 \sin 2\tau) \quad (4.32e)$$

$$v_{z,t+\Delta t} = -3\alpha \cos 3\tau \quad (4.32f)$$

with scale $\alpha \sim \mathcal{U}[\alpha_{\min}, \alpha_{\max}]$ and offset $\mathbf{o} = [o_x, o_y, o_z]^\top \sim \mathcal{U}[-o_{\max}, o_{\max}]^3$. Parameters: $\varphi_{\text{trefoil}} = \{\alpha, \mathbf{o}\}$. Accelerations are derived from second derivatives of the position equations.

4.3.2.4 Up-and-Down Motion

This represents vertical oscillatory motion, such as flapping wings or altitude-varying drone flight. The state updates are:

$$x_{t+\Delta t} = x_t + s\Delta t \quad (4.33a)$$

$$y_{t+\Delta t} = y_t \quad (4.33b)$$

$$z_{t+\Delta t} = A \sin(\omega(t + \Delta t)) \quad (4.33c)$$

$$v_{x,t+\Delta t} = s, \quad v_{y,t+\Delta t} = 0, \quad v_{z,t+\Delta t} = A\omega \cos(\omega(t + \Delta t)) \quad (4.33d)$$

with speed $s \sim \mathcal{U}[s_{\min}, s_{\max}]$, amplitude $A \sim \mathcal{U}[A_{\min}, A_{\max}]$, and frequency $\omega \sim \mathcal{U}[\omega_{\min}, \omega_{\max}]$. Parameters: $\varphi_{\text{up-and-down}} = \{s, A, \omega\}$. Acceleration is non-zero only in the z -direction: $a_{z,t} = -A\omega^2 \sin(\omega t)$.

4.4 Reference Instantiation and Dataset Corpus

This section details the instantiation of the SynTraG framework through the generation of a reference dataset corpus, designed to support robust trajectory forecasting for non-cooperative

dynamic obstacles in UAV navigation. The dataset, denoted $\mathcal{D}$, comprises 47,894 paired sequences, each consisting of historical state observations and future position predictions, synthesized using the kinematic primitives and generative process described in Sections 4.3.1 and 4.3.2. This corpus serves as a benchmark for evaluating forecasting models and is made publicly available to facilitate research in safety-critical aerial navigation.

4.4.1 Data Generation Process

The dataset is constructed by simulating trajectories over an 8-second temporal window, partitioned into a 3-second historical observation period ($T = 30$ timesteps with $\Delta t = 0.1$ seconds) and a 5-second future prediction horizon ($N = 50$ timesteps). Historical sequences $\mathbf{X}_{1:T} \in \mathbb{R}^{30 \times 6}$ capture the state $\mathbf{s}_t = [\mathbf{y}_t^\top, \mathbf{v}_t^\top]^\top \in \mathbb{R}^6$, where $\mathbf{y}_t = [x_t, y_t, z_t]^\top$ and $\mathbf{v}_t = [v_{x,t}, v_{y,t}, v_{z,t}]^\top$ represent position and velocity in a 3D Cartesian frame. Future sequences $\mathbf{Y}_{T+1:T+N} \in \mathbb{R}^{50 \times 3}$ provide the predicted positions $\mathbf{y}_t$ over the forecast horizon. Trajectories are sampled at intervals aligned with approximately one-quarter of the dominant motion period (e.g., $2\pi/\omega$ for sinusoidal or up-and-down primitives), ensuring diverse phase representations and comprehensive coverage of the dynamic behaviors induced by the mappings $\Phi_k(\mathbf{s}_t, t; \varphi_k)$. Initial states $\mathbf{s}_{t_0}$ are drawn from a bounded uniform prior over positions within $[-\mathbf{b}, \mathbf{b}]^3$ with zero initial velocity, and the rollout incorporates optional heteroscedastic Gaussian noise $\varepsilon_t \sim \mathcal{N}(\mathbf{0}, \Sigma_k)$ as per Eq. (4.27), where $\Sigma_k = \sigma^2(\ell)I_3$ and $\sigma^2(\ell) = 0.1 \cdot \ell \cdot u$ with $u \sim \text{Unif}(0.9, 1.1)$. This noise, proportional to path length $\ell = \sum_t \|\mathbf{y}_{t+\Delta t} - \mathbf{y}_t\|_2$, models localization uncertainty and supports probabilistic forecasting.

4.4.2 Dataset Composition and Visualization

The resulting dataset is structured as $\mathbf{X} \in \mathbb{R}^{47894 \times 30 \times 6}$ for observations and $\mathbf{Y} \in \mathbb{R}^{47894 \times 50 \times 3}$ for targets, reflecting a balanced distribution across the four motion primitives: straight-line, sinusoidal, trefoil knot, and up-and-down. A 3D visualization of representative trajectories is provided in Fig. 4.12, illustrating the diversity in initial positions and motion patterns, from linear paths to complex loops and vertical oscillations.

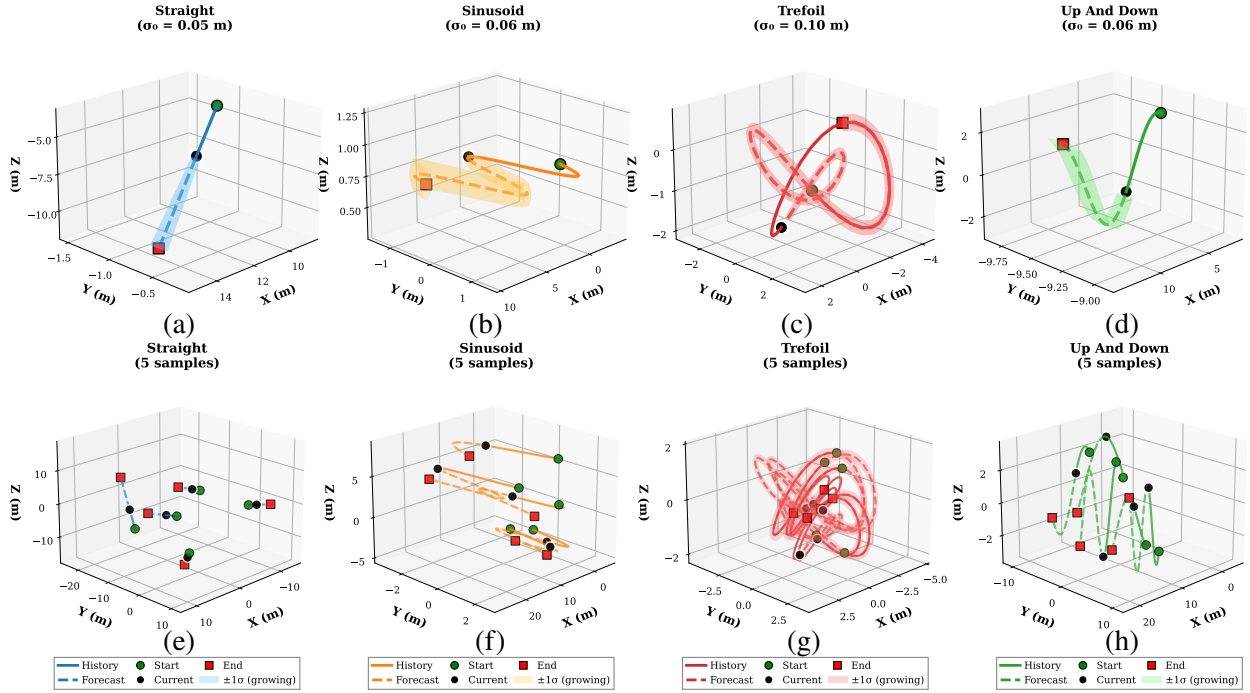


Figure 4.12

Representative 3D trajectories for each motion primitive with uncertainty quantification.

4.4.3 Statistical Analysis

The dataset’s statistical properties are summarized in Table 4.8, which reports the sample count, average trajectory length, minimum and maximum lengths, average speed, and variance metrics for each motion type. The average length is computed as the cumulative Euclidean distance over the 8-second simulation, while average speed is derived from the mean magnitude of the velocity vector $\|\mathbf{v}_t\| = \sqrt{v_{x,t}^2 + v_{y,t}^2 + v_{z,t}^2}$. Variance estimates, reflecting the heteroscedastic noise model, are calculated as 10% of the path length scaled by a uniform random factor $\sim \text{Unif}(0.9, 1.1)$, providing a realistic uncertainty profile for training mixture density networks (MDNs) or similar probabilistic models.

Table 4.8 also reveals the dataset’s diversity. For instance, sinusoidal and up-and-down trajectories exhibit greater length and speed variability, reflecting their oscillatory nature, while trefoil knots maintain consistent lengths due to their parametric structure.

Table 4.8

Dataset statistics per motion type.

Motion Type	Sample Count	Avg Length (m)	Min Length (m)	Max Length (m)	Avg Speed (m/s)	Min Speed (m/s)	Max Speed (m/s)	Avg. σ^2 (m ²)	Std. Dev. (m)
sinusoid	10280	44.27	8.22	116.53	4.07	1.00	12.38	1.99	1.24
straight	13650	27.24	6.65	66.16	1.26	0.50	2.00	0.62	0.22
trefoil	11568	38.53	18.21	59.95	4.60	4.43	4.79	2.23	0.65
up_and_down	12396	46.35	8.91	125.51	4.25	1.00	13.27	2.08	1.33

4.4.4 Accessibility and Usage

The complete dataset corpus in Python pickle (.pkl) format, along with the code and a detailed implementation for the SynTraG framework, is publicly available under the MIT License and accessible at: <https://github.com/syedi/SynTraG.git>. Users can leverage the dataset for training forecasting models, with pre-computed variance estimates facilitating probabilistic predictions. The framework’s configurability allows for regeneration of $\mathcal{D}$ with adjusted mixture weights $\boldsymbol{\pi}$ or parameter priors Π_k , enabling tailored datasets for specific research objectives.

4.5 Conclusion

This chapter introduces SynTraG, an open-source synthetic trajectory generator addressing the critical need for data to train UAV collision avoidance systems against non-cooperative dynamic obstacles. By synthesizing 47,894 diverse 3D trajectories via mixture of four kinematic primitives with optional uncertainty, SynTraG provides a flexible, open-source tool for robust forecasting model development. Reference instantiation demonstrate its utility, while the publicly available dataset and code facilitate further research. Future work will explore real-world validation, integration with multi-modal sensors, and extension to adversarial motion patterns.

CHAPTER V: AEROCast: PROBABILISTIC 3D TRAJECTORY PREDICTION FOR NON-COOPERATIVE DYNAMIC OBSTACLES

Abstract

Autonomous aerial vehicles operating in shared airspace must predict the future positions of non-cooperative obstacles to plan evasive maneuvers before a collision becomes unavoidable. Unlike cooperative systems that share intent, non-cooperative obstacles such as birds, uncontrolled drones, or debris exhibit multi-modal motion that deterministic predictors cannot adequately represent. Existing methods either rely on recurrent encoders that propagate temporal information sequentially, limiting their ability to capture long-range kinematic precursors of maneuver initiation, or produce point forecasts that provide no distributional information to downstream planners. This chapter presents AeroCast, a probabilistic trajectory prediction framework that combines a Transformer encoder with a Mixture Density Network output head to predict per-timestep Gaussian mixture distributions over future three-dimensional displacements. A translation-invariant consecutive displacement encoding and a calibration-oriented training objective address the input design and mode-degeneracy challenges specific to mixture-based aerial trajectory prediction. On a hybrid real-and-synthetic quadrotor corpus spanning nine motion categories, AeroCast reduces Average Displacement Error and Final Displacement Error by approximately 50% relative to the baselines over a five-second horizon, and achieves the lowest negative log-likelihood and Continuous Ranked Probability Score among all compared methods. Ablation analysis identifies velocity input and model capacity as the primary contributors to prediction quality, and positional encoding as essential for long-horizon trajectory coherence. AeroCast inference completes in 0.1 ms per sample, compatible with real-time onboard deployment at 100 Hz.

5.1 Introduction

The proliferation of unmanned aerial vehicles (UAVs) in shared airspace introduces a core safety requirement: an autonomous vehicle must anticipate the future positions of surrounding non-cooperative obstacles to plan evasive maneuvers before a collision becomes geometrically

unavoidable. Unlike cooperative multi-agent systems that share intent through communication protocols, non-cooperative obstacles such as birds, uncontrolled drones, or wind-borne debris provide no advance notice of their behavior. Reactive strategies that respond only to instantaneous obstacle positions offer insufficient safety margins at typical flight speeds, where the interval between detection and potential collision may be shorter than the vehicle’s minimum stopping distance [4]. Proactive avoidance requires a predictor that maps a finite observation history to a distribution over future positions. This task is more challenging in the aerial domain than in ground-domain forecasting [140]. Aerial obstacles move in unconstrained three-dimensional space without the road geometry or pedestrian social norms that regularize planar motion, and their behavior is inherently multi-modal: an approaching obstacle may continue straight, bank sharply, or climb, each with non-trivial probability.

Existing methods address these challenges only partially. Physics-based estimators such as Kalman filters and Interacting Multiple Model (IMM) trackers assume predefined motion models (constant velocity, coordinated turn) that poorly capture the erratic behavior of non-cooperative obstacles [141]. Learning-based recurrent architectures, notably the GRU-based VECTOR model for 3-D UAV prediction [142], achieve fast inference suitable for embedded deployment but produce deterministic point forecasts. A single-trajectory output gives a downstream planner no information about whether the prediction is confident or whether the obstacle is at a behavioral branch point, both situations are indistinguishable in the output. Transformer-based encoders offer a structural alternative: the self-attention mechanism provides every observation token with direct access to every other token in the window, shortening the information path between temporally distant timesteps to $O(1)$ and removing the sequential bottleneck inherent in recurrent processing [42]. To date, no prior work has applied a Transformer encoder specifically to non-cooperative aerial obstacle prediction with a multi-modal probabilistic output head targeting uncertainty.

This chapter presents AeroCast, a probabilistic trajectory prediction framework that pairs a Pre-LN Transformer encoder with a Mixture Density Network (MDN) output head to predict per-timestep Gaussian mixture distributions over future three-dimensional displacements. The input

representation encodes consecutive positional displacements normalized by a scale that enforces translation invariance and produces bounded inputs suited to Gaussian mixture modeling. The training objective combines negative log-likelihood with a mode-anchoring mean-squared-error term and a variance floor matched to the sensor noise level, which prevents the mode degeneracy that arises under pure NLL optimization with multiple mixture components. AeroCast is evaluated on a hybrid real-and-synthetic quadrotor trajectory corpus spanning nine kinematically diverse motion categories, with quantitative comparisons against four baselines at matched parameter budgets, per-category accuracy breakdowns, and a systematic ablation of all major design choices.

The main contributions are:

1. **A probabilistic Transformer-MDN predictor for non-cooperative aerial obstacles.** AeroCast combines a Pre-LN Transformer encoder with MDN output heads to produce per-timestep Gaussian mixtures over future 3-D displacements, reducing ADE and FDE by approximately 50% relative to the strongest recurrent baseline and achieving the lowest NLL and CRPS among all compared methods.
2. **A translation-invariant displacement encoding and calibration-oriented training protocol.** The consecutive displacement representation provides translation invariance, bounded input distributions, and Gaussian compatibility. The combined NLL and mode-anchoring objective with a physically grounded sigma floor prevents variance collapse and produces uncertainty estimates through Expected Calibration Error.
3. **A comprehensive experimental evaluation and benchmarking.** We conduct quantitative comparisons against four trained baselines at matched parameter budgets, a per-trajectory-type accuracy breakdown across nine kinematically diverse motion categories, a prediction horizon analysis tracking error growth from 1 to 5 seconds, and a systematic ablation of all major architectural and training choices.
4. **A hybrid real-and-synthetic benchmark for non-cooperative aerial obstacle prediction.** We contribute a dataset of 90,116 sliding-window sequences from 113 quadrotor flight

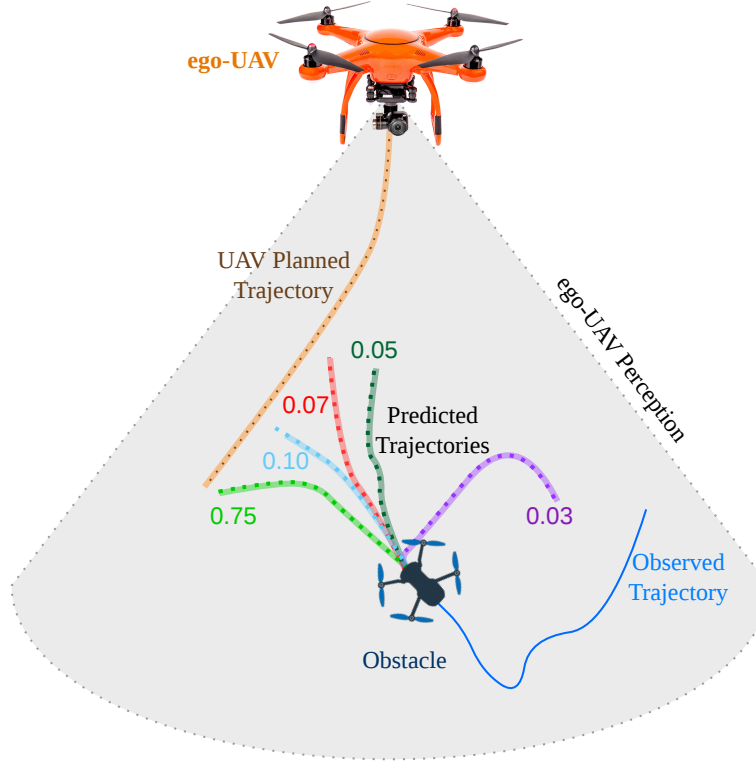


Figure 5.13

A typical scenario where an ego-UAV encounters a non-cooperative dynamic obstacle (an uncontrolled drone).

recordings across nine kinematically diverse motion categories that combines sub-millimeter Vicon ground truth with synthetic augmentation derived from a parametric trajectory generator. This dataset, together with a standardized evaluation protocol covering ADE, FDE, best-of-K, NLL, and ECE metrics, to our knowledge, constitutes the first structured benchmark for 3-D non-cooperative aerial obstacle trajectory prediction.

5.2 Related Work

5.2.1 Classical Estimation and Recurrent Forecasting for Aerial Trajectories

The classical approach to aerial obstacle tracking relies on Bayesian state estimators that maintain a finite bank of kinematic hypotheses updated recursively as observations arrive. Kalman filters and their multi-model extensions, including Interacting Multiple Model trackers, are computationally efficient and interpretable, but their predictive quality is bounded by the fidelity of the

assumed motion models to the underlying dynamics [141]. For non-cooperative aerial obstacles exhibiting abrupt heading reversals, irregular vertical excursions, or speed-coupled turn maneuvers, no compact set of constant-velocity or coordinated-turn hypotheses adequately spans the behavioral envelope, and the estimator systematically lags behind the true state evolution rather than anticipating it.

Learning-based recurrent architectures displaced hand-crafted motion models by extracting temporal structure directly from data. LSTM, GRU, and bidirectional GRU variants has been used as a standard components in trajectory forecasting pipelines. For aerial trajectory prediction specifically, VECTOR [142] introduced a velocity-enhanced GRU formulation for 3D UAV prediction, that achieves fast inference suitable for onboard deployment. However, the sequential update mechanism imposes an information path length proportional to observation length, so early kinematic signatures of an impending maneuver must propagate through every intermediate hidden transition before influencing the forecast. Moreover, all recurrent methods cited above produce deterministic point forecasts, providing no information to a downstream planner about whether the prediction reflects high confidence or an imminent behavioral bifurcation.

5.2.2 Transformer-Based Trajectory Forecasting

The Transformer formulation of Vaswani et al. [42] replaced recurrence with self-attention, allowing every observation token to interact directly with every other within the window. Global connectivity shortens the information path length to a constant number of steps between any two timesteps. Stability of deep encoder stacks is improved by pre-layer normalization, which places normalization inside the residual branch and reduces the optimization fragility of post-normalization Transformers [43]. These properties have been exploited in vehicle and pedestrian forecasting through hierarchical and vectorized attention models [143, 144]. To our knowledge, no prior work applies a Transformer encoder specifically to non-cooperative aerial obstacle prediction, and none couples such an encoder with a multi-modal probabilistic output head targeting calibrated uncertainty estimation. This work adopts the same architectural motivation but applies it to non-cooperative aerial obstacles, using a Transformer encoder over a 2 s history of 3-D position and

velocity so the model can attend globally to maneuver precursors over the full observation horizon.

5.2.3 Probabilistic Prediction and Calibration

Deterministic forecasts collapse distinct future behaviors into a single mean path, which is insufficient for risk-aware planning that treats uncertainty as a decision variable. Multi-modal output representations have become standard in ground-domain forecasting: Social-GAN [47] promotes diverse futures through adversarial training and a variety loss; Trajectron++ [58] combines a graph-structured recurrent backbone with conditional latent variables; and AgentFormer [143] extends multi-modal prediction to Transformer architectures with social context. These methods establish that single-hypothesis outputs are inadequate in dense or interactive scenes, but they were designed around ground-domain constraints (map topology, pedestrian norms) that have no counterpart in free-flight aerial prediction.

A separate question is whether the predicted uncertainty is *calibrated*, which means, whether a model’s stated confidence intervals match empirical coverage rates. Guo et al. [49] showed that modern deep networks are systematically miscalibrated and formalized Expected Calibration Error (ECE) as a summary statistic for the confidence–coverage mismatch. Lakshminarayanan et al. [145] demonstrated that deep ensembles produce well-calibrated uncertainty through multi-model averaging, at the cost of multiple forward passes that increase computational and memory requirements on embedded platforms. Kendall and Gal [146] distinguished aleatoric uncertainty (irreducible measurement noise and behavioral variability) from epistemic uncertainty (reducible with more data or richer models), a decomposition that clarifies which source of uncertainty a given architecture can represent. On the training side, Gneiting and Raftery [147] proved that strictly proper scoring rules, including negative log-likelihood, are the correct objective when the output is a probability distribution; mean squared error is not proper at the distribution level and can drive variance collapse. Ivanovic et al. [132] extended this perspective to trajectory forecasting by analyzing how predictive uncertainty propagates along the forecast horizon.

Most multi-modal trajectory predictors evaluate probabilistic quality only indirectly, through best-of- K error or diversity measures, and none of the cited methods enforces calibration during

Table 5.9

Positioning of AeroCast relative to representative trajectory prediction methods.

Method	3-D	Attn.	Multi-modal	Calib.	Aerial
Social-GAN [47]	—	—	✓	—	—
Trajectron++ [58]	—	—	✓	—	—
AgentFormer [143]	—	✓	✓	—	—
VECTOR [142]	✓	—	—	—	✓
AeroCast (ours)	✓	✓	✓	✓	✓

training. AeroCast encodes aleatoric ambiguity in a per-timestep Gaussian mixture trained with NLL as the primary objective, augmented by a mode-anchoring MSE term and a sigma floor matched to the sensor noise level. This design targets single-pass inference compatible with onboard deployment and produces uncertainty estimates that can be assessed through ECE.

5.2.4 Datasets for Non-Cooperative Aerial Prediction

Public trajectory datasets at the scale and structure available for ground-domain prediction do not exist for non-cooperative 3-D aerial obstacles. Pedestrian benchmarks such as ETH [148] and driving corpora such as nuScenes [61] and the Waymo Open Motion Dataset [62] contain planar, map-constrained, or cooperative traffic structure absent from free-flight aerial settings. Existing aerial corpora focus on cooperative multi-UAV operations or structured air-traffic recording, under-representing the abrupt and vertically coupled maneuvers relevant to collision avoidance in shared airspace. SynTraG [149] introduced a parametric generator for non-cooperative dynamic obstacles in UAV navigation. The present work combines a Vicon-collected real-flight corpus [150] with synthetic [149] augmentation derived from an extension of these primitives, coupling real 3-D obstacle motion with synthetic samples that share the same physical and noise assumptions used by the predictor.

Table 5.9 summarizes the positioning of AeroCast relative to representative prior work across the design dimensions identified above.

5.3 Problem Formulation

The trajectory prediction problem is formulated from the perspective of an ego aerial vehicle observing external dynamic obstacles in its vicinity. Let $\mathcal{X} = \{(\mathbf{x}_t, \mathbf{v}_t)\}_{t=1}^{T_{\text{obs}}}$ denote the observed

trajectory history of an obstacle, where $\mathbf{x}_t \in \mathbb{R}^3$ is the three-dimensional position relative to the ego vehicle at time t and $\mathbf{v}_t \in \mathbb{R}^3$ is the corresponding velocity vector. Positions $\mathbf{x}_t = [x_t, y_t, z_t]^\top$ are expressed in a world-fixed inertial frame with the z -axis opposing gravity; for the experimental validation this frame coincides with the Vicon system origin. Velocities are computed through centered finite differences with Gaussian smoothing:

$$\mathbf{v}_t = \frac{\mathbf{x}_{t+1} - \mathbf{x}_{t-1}}{2\Delta t} * \mathcal{G}(\sigma) \quad (5.34)$$

where Δt is the sampling interval and $\mathcal{G}(\sigma)$ is a Gaussian kernel with $\sigma = 0.5 \Delta t$, chosen to balance noise suppression and temporal resolution.

The objective is to predict the obstacle's future trajectory $\mathcal{Y} = \{\mathbf{y}_t\}_{t=T_{\text{obs}}+1}^{T_{\text{obs}}+T_{\text{pred}}}$, where $\mathbf{y}_t \in \mathbb{R}^3$ is the predicted position at time t . The prediction is model-free; no knowledge of the obstacle's dynamics, control law, or physical parameters is assumed. The obstacle may be a bird, an autonomous drone, a remotely piloted vehicle, or any other aerial agent. This model-agnostic requirement motivates a learning-based approach that infers motion patterns purely from observed kinematics.

5.3.1 Temporal Specifications

Observations span $T_{\text{obs}} = 2.0$ s sampled at 10 Hz, yielding input sequences of length 20. Predictions extend $T_{\text{pred}} = 5.0$ s into the future (50 timesteps at 10 Hz). The 10 Hz rate matches typical tracking-system update frequencies and provides sufficient temporal resolution to capture the motion dynamics of small agile platforms. The 5 s horizon provides a collision avoidance planner with enough lead time to identify conflicts and execute evasive maneuvers.

5.3.2 Consecutive Displacement Encoding

Raw position coordinates $\mathbf{x}_t$ are translation-variant where the same motion pattern yields different input values depending on where in the workspace the obstacle is located. To remove this redundancy the input representation encodes consecutive displacements:

$$\boldsymbol{\delta}_t = \mathbf{x}_{t+1} - \mathbf{x}_t, \quad t = 1, \dots, T_{\text{obs}} - 1 \quad (5.35)$$

The prediction target at horizon step τ is likewise a displacement: $\delta_{T_{\text{obs}}+\tau} = \mathbf{x}_{T_{\text{obs}}+\tau+1} - \mathbf{x}_{T_{\text{obs}}+\tau}$. The full input at timestep t concatenates the displacement with the contemporaneous velocity:

$$\mathbf{s}_t = \begin{bmatrix} \delta_t \\ \mathbf{v}_t \end{bmatrix} \in \mathbb{R}^6 \quad (5.36)$$

Displacements are normalized by a scale constant $\lambda = 2.5$ m, chosen so that the 99th percentile of per-step displacement magnitudes in the training set falls within $[-1, 1]^3$. This encoding provides three properties: (i) *translation invariance*, since $\mathbf{s}_t$ is unchanged by any rigid shift of the trajectory; (ii) *bounded input distribution*, since normalization by λ concentrates input mass within a compact region; and (iii) *Gaussian compatibility*, since step-to-step displacements of a smooth trajectory approximate a zero-mean distribution well matched to the GMM output parameterization.

Future positions are recovered from predicted displacements by cumulative summation anchored at the last observed position:

$$\hat{\mathbf{x}}_{T_{\text{obs}}+\tau} = \mathbf{x}_{T_{\text{obs}}} + \sum_{j=1}^{\tau} \hat{\delta}_j \cdot \lambda \quad (5.37)$$

5.3.3 Multi-Modal Distribution Representation

At each predicted timestep, the distribution over future displacements is modeled as a Gaussian Mixture Model (GMM):

$$p(\mathbf{y}_t | \mathcal{X}) = \sum_{k=1}^K \pi_k^{(t)} \mathcal{N}(\mathbf{y}_t | \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}) \quad (5.38)$$

where K is the number of mixture components, $\pi_k^{(t)}$ is the mixture weight for component k at time t with $\sum_k \pi_k^{(t)} = 1$, $\boldsymbol{\mu}_k^{(t)} \in \mathbb{R}^3$ is the component mean, and $\boldsymbol{\Sigma}_k^{(t)} = \text{diag}(\sigma_{k,x}^{(t)}, \sigma_{k,y}^{(t)}, \sigma_{k,z}^{(t)})$ is a diagonal covariance. The mixture components represent alternative behavioral modes; their weights encode mode likelihoods, and their covariances quantify positional uncertainty. This formulation provides a downstream planner with a richer signal than a scalar confidence score, where concentrated weights indicate high prediction confidence, and distributed weights indicate a behavioral decision point at which conservative planning is appropriate.

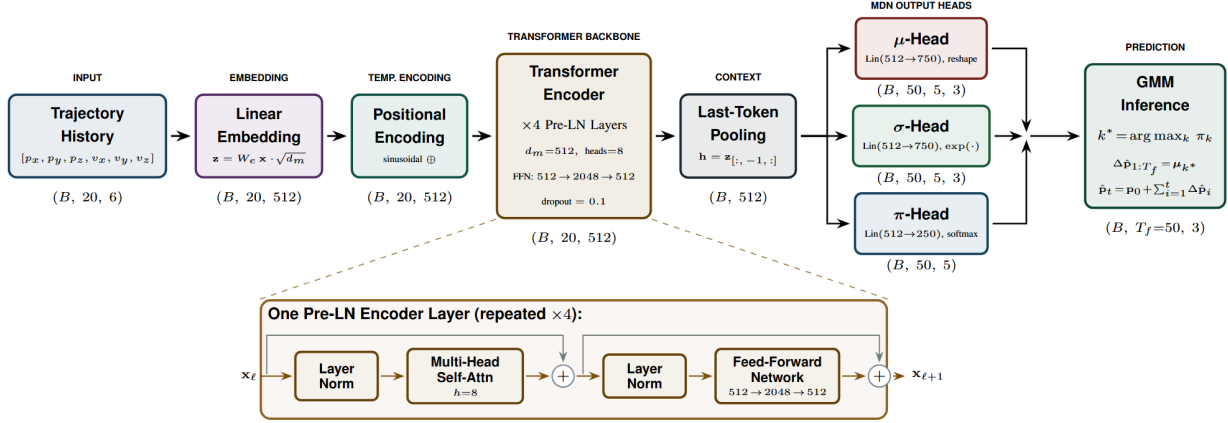


Figure 5.14
AeroCast overall system architecture.

5.4 Methodology

5.4.1 Architecture Overview

The AeroCast architecture comprises three functional blocks: an input embedding that projects the six-dimensional state $\mathbf{s}_t$ to a d_{model} -dimensional representation, a stack of L Transformer encoder layers performing temporal self-attention over the observation window, and three linear output heads that jointly produce the GMM parameters $\{\mu_k^{(\tau)}\}$, $\{\Sigma_k^{(\tau)}\}$, and $\{\pi_k^{(\tau)}\}$ for all $T_{\text{pred}} \times K$ mixture components in a single forward pass. Fig. 5.14 illustrates the complete pipeline. The model uses $d_{\text{model}} = 512$, $L = 4$ encoder layers, $H = 8$ attention heads ($d_k = 64$ per head), feedforward inner dimension $4d_{\text{model}} = 2048$, $K = 5$ GMM components, and dropout probability $p = 0.1$. The total parameter count is 13.15 M.

5.4.2 Input Embedding with Positional Encoding

Each state vector $\mathbf{s}_t \in \mathbb{R}^6$ is projected to a d_{model} -dimensional embedding:

$$\mathbf{h}_t^{(0)} = \sqrt{d_{\text{model}}} \mathbf{W}_{\text{emb}} \mathbf{s}_t + \mathbf{e}_t^{\text{pos}} \quad (5.39)$$

where $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{d_{\text{model}} \times 6}$ is a learned projection matrix and $\mathbf{e}_t^{\text{pos}}$ is the sinusoidal positional encoding [42]:

$$\mathbf{e}_t^{\text{pos}}[2i] = \sin\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right) \quad (5.40)$$

$$\mathbf{e}_t^{\text{pos}}[2i+1] = \cos\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right) \quad (5.41)$$

for $i = 0, \dots, d_{\text{model}}/2-1$. The $\sqrt{d_{\text{model}}}$ scaling prevents the fixed positional signal from dominating the learned content embedding at large model widths. Because self-attention is permutation-equivariant, the positional encoding is the sole mechanism by which the model distinguishes temporal order within the observation window [42]. The ablation in Section 5.7.4 quantifies the impact of removing it.

5.4.3 Transformer Encoder

The embedded sequence $\{\mathbf{h}_t^{(0)}\}_{t=1}^{T_{\text{obs}}}$ is processed through L identical encoder layers. Each layer applies pre-layer normalization (Pre-LN) [43]:

$$\tilde{\mathbf{h}}_t^{(\ell)} = \mathbf{h}_t^{(\ell-1)} + \text{MHA}(\text{LN}(\mathbf{H}^{(\ell-1)}))_t \quad (5.42)$$

$$\mathbf{h}_t^{(\ell)} = \tilde{\mathbf{h}}_t^{(\ell)} + \text{FFN}(\text{LN}(\tilde{\mathbf{h}}_t^{(\ell)})) \quad (5.43)$$

where $\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{T_{\text{obs}} \times d_{\text{model}}}$ is the sequence matrix and $\text{LN}(\cdot)$ denotes layer normalization. The multi-head attention operation uses H parallel heads of dimension $d_k = d_{\text{model}}/H$:

$$\text{MHA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O \quad (5.44)$$

$$\text{head}_h = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}}\right) \mathbf{V}_h \quad (5.45)$$

with $\mathbf{Q}_h = \mathbf{H} \mathbf{W}_h^Q$, $\mathbf{K}_h = \mathbf{H} \mathbf{W}_h^K$, $\mathbf{V}_h = \mathbf{H} \mathbf{W}_h^V$, where $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$.

The scaled dot-product produces a $T_{\text{obs}} \times T_{\text{obs}}$ affinity matrix, giving every timestep $O(1)$ access to every other timestep in the window, compared with $O(T_{\text{obs}})$ for recurrent encoders.

The position-wise feed-forward network applies a two-layer MLP with ReLU activation:

$$\text{FFN}(\mathbf{h}) = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2 \quad (5.46)$$

with $\mathbf{W}_1 \in \mathbb{R}^{4d_{\text{model}} \times d_{\text{model}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{model}} \times 4d_{\text{model}}}$. Dropout is applied after both the MHA and FFN outputs, before each residual addition.

After the L -th layer, a terminal layer normalization is applied to the output at the last temporal position to produce the context vector:

$$\mathbf{c} = \text{LN}(\mathbf{h}_{T_{\text{obs}}}^{(L)}) \in \mathbb{R}^{d_{\text{model}}} \quad (5.47)$$

This vector serves as the sole input to the three output heads; no recurrent or autoregressive decoding is employed.

5.4.4 GMM Output Heads

Three linear projections map $\mathbf{c}$ to the GMM parameters for all timesteps and components simultaneously:

$$\hat{\boldsymbol{\mu}} = \mathbf{W}_{\mu} \mathbf{c} \in \mathbb{R}^{T_{\text{pred}} \times K \times 3} \quad (5.48)$$

$$\log \hat{\boldsymbol{\sigma}} = \mathbf{W}_{\sigma} \mathbf{c} \in \mathbb{R}^{T_{\text{pred}} \times K \times 3} \quad (5.49)$$

$$\hat{\mathbf{z}}_{\pi} = \mathbf{W}_{\pi} \mathbf{c} \in \mathbb{R}^{T_{\text{pred}} \times K} \quad (5.50)$$

Standard deviations and mixture weights are obtained via:

$$\hat{\sigma}_{k,d}^{(\tau)} = \max(e^{\log \hat{\sigma}_{k,d}^{(\tau)}}, \sigma_{\min}) \quad (5.51)$$

$$\hat{\pi}_k^{(\tau)} = \frac{e^{\hat{z}_{\pi,k}^{(\tau)}}}{\sum_{j=1}^K e^{\hat{z}_{\pi,j}^{(\tau)}}} \quad (5.52)$$

The variance floor $\sigma_{\min} = 0.05$ m matches the Vicon measurement noise level. This bound prevents degenerate point-mass predictions and ensures that predicted confidence intervals remain physically interpretable. The full output dimensionality per forward pass is $T_{\text{pred}} \times K \times (3+3+1)$ scalars; with $T_{\text{pred}} = 50$ and $K = 5$ this yields $50 \times 5 \times 7 = 1,750$ outputs.

5.4.5 Training Objective

The model is trained to maximize the log-likelihood of observed displacements under the predicted GMM. The batch-averaged negative log-likelihood is:

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{NT_{\text{pred}}} \sum_{n=1}^N \sum_{\tau=1}^{T_{\text{pred}}} \log \sum_{k=1}^K \hat{\pi}_k^{(\tau)} \mathcal{N}(\boldsymbol{\delta}_{\tau}^{(n)} | \hat{\boldsymbol{\mu}}_k^{(\tau)}, \hat{\boldsymbol{\Sigma}}_k^{(\tau)}) \quad (5.53)$$

computed via the standard log-sum-exp identity for numerical stability.

Pure NLL training with $K > 1$ admits a degenerate solution in which all components spread to cover the data support without any single component being geometrically close to the ground truth. This degeneracy is consequential because ADE and FDE at evaluation time are computed from the dominant-weight component $k^* = \arg \max_k \hat{\pi}_k^{(\tau)}$. To break it without collapsing the predicted variance (as a large MSE weight would), a small auxiliary mean-squared error term is applied to k^* :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{NT_{\text{pred}}} \sum_{n=1}^N \sum_{\tau=1}^{T_{\text{pred}}} \|\hat{\mu}_{k^*}^{(\tau)} - \delta_{\tau}^{(n)}\|^2 \quad (5.54)$$

The combined objective is:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{NLL}} + \lambda_{\text{MSE}} \mathcal{L}_{\text{MSE}} \quad (5.55)$$

with $\lambda_{\text{MSE}} = 0.15$, chosen so that the MSE term contributes roughly 10–15% of the total loss magnitude. NLL remains the dominant signal, preserving the proper-scoring-rule property that encourages well-spread predictive distributions.

5.4.5.1 Curriculum Sequence-Length Scheduling

Sliding-window sequences drawn from short trajectory segments can exhibit irregular leading-edge behavior that destabilizes early training. A curriculum schedule addresses this by gradually increasing the effective observation length during the first $E_{\text{warm}} = 20$ epochs:

$$T_{\text{eff}}(e) = \min\left(T_{\text{obs}}, \max(5, \lfloor T_{\text{obs}} \cdot e / E_{\text{warm}} \rfloor)\right) \quad (5.56)$$

At epoch e , the model processes only the most recent $T_{\text{eff}}(e)$ timesteps of each window; the full 20-step window is used for all subsequent epochs. Additive Gaussian noise ($\sigma_{\text{aug}} = 0.02$ m) and random scale perturbation (uniform in $[0.95, 1.05]$) are applied to inputs during the warmup phase.

5.5 Dataset

We constructed the dataset combining real-world flights captured in a motion-capture arena with synthetic trajectories generated by an extension of our prior synthetic data framework Syn-TraG [149]. This section describes the physical collection methods, the synthetic augmentation procedure, the preprocessing pipeline, and the resulting dataset statistics.

5.5.1 Physical Data Collection

5.5.1.1 Hardware Platform

Real trajectories were collected with a Crazyflie 2.1 nano-quadrotor (Bitcraze AB, $92 \times 92 \times 29$ mm, 27 g with a 250 mAh single-cell LiPo battery installed) flown as the obstacle agent in a Vicon-equipped indoor arena of dimensions $6 \times 4 \times 2$ m. The arena is instrumented with twelve infrared cameras providing rigid-body 6-DoF pose at 100 Hz with sub-millimeter spatial accuracy. The platform was chosen for its small inertia and high agility, which enable the abrupt heading reversals and multi-axis maneuvers required to exercise the multi-modal regime of the prediction problem.

5.5.1.2 Trajectory Categories

Trajectories were collected across nine motion categories chosen to span the kinematic envelope an autonomous vehicle is likely to encounter when sharing airspace with non-cooperative agents. The categories fall into three groups: *periodic* (Circle, Oval, Figure-8), which approximate loitering and patrolling behaviors with strong temporal regularity; *non-trivially 3-D* (Helix, Trefoil, Lissajous, Staircase), which couple horizontal and vertical motion in ways that challenge planar prediction; and *aperiodic* (Star, Random), which contain abrupt heading reversals and unstructured motion that resist physics-based filter modeling. Reference paths for each category were generated as parametric curves and tracked under closed-loop position control; parameter values (radius, amplitude, frequency, altitude offset) were randomized across flights to span the dataset’s diversity in speed and extent.

5.5.2 Synthetic Data Augmentation

Approximately 43% of the data corpus is synthetic, generated by an extension of the SynTraG framework [149] that expands the four kinematic primitives described in the original work to the nine categories used here. Synthetic trajectories follow the same parametric structure as the physical reference paths and are generated with randomized speed profiles, radii, altitudes, and noise levels to fill regions of the kinematic state space that are under-represented in the

physical collection, particularly in the high-speed and high-altitude regimes that the limited arena volume cannot accommodate. Heteroscedastic Gaussian noise consistent with the Vicon noise floor ($\sigma_{\text{ms}} = 0.05$ m, the same value used as the sigma floor in the prediction head) is added to synthetic samples to match the distribution of measurement noise in the real data. Synthetic and real samples are pooled before splitting and treated identically downstream; no synthetic-versus-real label is provided to the model.

5.5.3 Preprocessing and Sliding-Window Sampling

Each recorded trajectory is downsampled from 100 Hz to 10 Hz to match the prediction-system update rate. Velocities are reconstructed from the position stream by the centered finite difference with Gaussian smoothing defined in Eq. (5.34). Sliding windows are extracted with stride one: each window comprises a 20-step observation followed by a 50-step prediction target, corresponding to 2 s of history and 5 s of horizon at 10 Hz. Stride one is chosen because temporally adjacent windows differ in their prediction targets by a single timestep at the leading edge, which exposes the model to maximum target diversity per trajectory; the implied correlation between adjacent samples is bounded by the trajectory-level split described next. The state vector at each input timestep is the displacement-velocity pair $\mathbf{s}_t = [\boldsymbol{\delta}_t^\top, \mathbf{v}_t^\top]^\top \in \mathbb{R}^6$ defined in Section 5.3.2.

The train/validation partition is performed at the trajectory level: each recording is assigned in its entirety to one split, ensuring that no observation from a validation trajectory appears in training and that each window’s 5-second prediction target is statistically independent of all training samples. The split ratio is 85/15 with a fixed random seed. Stratification by trajectory category ensures that all nine motion types appear in both splits in proportion to their frequency in the full corpus.

5.5.4 Dataset Statistics

Table 5.10 summarizes the full corpus by category, reporting the sliding-window sample count, mean per-window RMS speed, peak speed, and the percentage contributed by physical and synthetic flights. The total sample count is 90 116 from 113 unique trajectory recordings. Sample counts are roughly comparable across the nine categories; the balanced distribution ensures that all motion types receive adequate representation during training and that aggregate metrics are not dominated

Table 5.10

Dataset summary by trajectory category.

Category	Sequences	Mean speed (m/s)	Max speed (m/s)	Real (%)	Synth. (%)
Circle	10,248	0.78	1.43	64	36
Oval	10,062	0.79	1.37	62	38
Figure-8	9,874	0.74	1.77	60	40
Helix	10,136	0.79	1.57	57	43
Trefoil	10,410	0.71	1.90	58	42
Lissajous	9,748	0.73	1.34	50	50
Staircase	9,512	0.50	0.88	46	54
Star	10,286	0.65	1.36	62	38
Random	9,840	0.55	1.04	53	47
Total	90,116	0.69	1.41	57	43

by any single category. Mean per-window speeds range from 0.5 m/s on Staircase trajectories (slow vertical maneuvers) to 0.79 m/s on the more agile periodic categories.

5.6 Experimental Setup

5.6.1 Baseline Methods

Four baseline architectures are trained under conditions identical to AeroCast. All baselines share the same MDN output head ($K=5$ mixture components), the same training objective as described in Section 5.4.5, and the same data splits.

GRU-MDN [142] uses a three-layer unidirectional GRU encoder with hidden dimension 512. This baseline isolates the effect of replacing sequential temporal encoding with global self-attention.

LSTM-MDN [151] uses a three-layer LSTM encoder with hidden dimension 512. The explicit cell state provides a distinct inductive bias relative to GRUs for retaining information over the full 20-step observation window.

BiGRU-MDN [152] uses a three-layer bidirectional GRU with hidden dimension 512. The backward pass gives the encoder access to future-looking context within the observation window, at the cost of roughly doubling the recurrent parameter count relative to GRU-MDN.

MLP-MDN [153] uses a three-layer MLP with hidden dimensions (2048, 2048, 1024) that receives the flattened 20-step observation window (120 input features) and maps it directly to

GMM parameters. This baseline tests whether a feed-forward architecture with sufficient capacity can implicitly capture temporal structure.

Parameter counts are 4.75 M (GRU-MDN), 6.07 M (LSTM-MDN), 13.17 M (BiGRU-MDN), and 7.63 M (MLP-MDN). AeroCast uses 13.15 M parameters, closely matching BiGRU-MDN.

5.6.2 Evaluation Metrics

Six metrics are used to evaluate trajectory accuracy, distributional quality, and calibration.

Average Displacement Error (ADE) is the mean Euclidean distance between predicted and ground-truth positions over all future timesteps:

$$\text{ADE} = \frac{1}{NT_{\text{pred}}} \sum_{n=1}^N \sum_{\tau=1}^{T_{\text{pred}}} \left\| \hat{\mathbf{x}}_{T_{\text{obs}}+\tau}^{(n)} - \mathbf{x}_{T_{\text{obs}}+\tau}^{(n)} \right\|_2 \quad (5.57)$$

Final Displacement Error (FDE) is the Euclidean distance at the last predicted timestep:

$$\text{FDE} = \frac{1}{N} \sum_{n=1}^N \left\| \hat{\mathbf{x}}_{T_{\text{obs}}+T_{\text{pred}}}^{(n)} - \mathbf{x}_{T_{\text{obs}}+T_{\text{pred}}}^{(n)} \right\|_2 \quad (5.58)$$

minADE@K and *minFDE@K* are best-of- K variants obtained by drawing $K=5$ trajectory samples from the predicted GMM and reporting the minimum ADE or FDE across samples. These metrics reward coverage of the ground truth by any mixture component.

Negative Log-Likelihood (NLL) is the mean per-timestep negative log-likelihood under the predicted GMM, as defined in Eq. (5.53). Lower values indicate higher probability assigned to observed future displacements.

Continuous Ranked Probability Score (CRPS) is an energy-score-based proper scoring rule that jointly rewards distributional accuracy and calibration [147]:

$$\text{CRPS} = \frac{1}{NKT_{\text{pred}}} \sum_{n,\tau} \left(\mathbb{E}_{\hat{Y}} \left\| \hat{Y} - \mathbf{y}_{\tau}^{(n)} \right\| - \frac{1}{2} \mathbb{E}_{\hat{Y}, \hat{Y}'} \left\| \hat{Y} - \hat{Y}' \right\| \right) \quad (5.59)$$

estimated via Monte Carlo samples from the predicted GMM.

Expected Calibration Error (ECE) measures agreement between predicted uncertainty and empirical coverage. For z -score thresholds z_{ℓ} spanning $[0.1, 3.0]$, the nominal coverage is $p_{\ell} = 2\Phi(z_{\ell}) - 1$ (where Φ is the standard normal CDF) and the empirical coverage $\hat{p}_{\ell}$ is the fraction of

ground-truth displacements falling within z_ℓ predicted standard deviations of the predicted mean:

$$\text{ECE} = \frac{1}{|\mathcal{Z}|} \sum_{\ell} |p_\ell - \hat{p}_\ell| \quad (5.60)$$

A perfectly calibrated model achieves $\text{ECE} = 0$. All six metrics are computed on the held-out validation split; lower is better for each.

5.6.3 Training Protocol

All five models are trained with the combined NLL and mode-anchoring objective defined in Eq. (5.55) ($\lambda_{\text{MSE}} = 0.15$). Optimization uses AdamW [154] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and weight decay 10^{-5} . The batch size is 128 for all five models. The initial learning rate is $\eta_0 = 10^{-4}$ with a minimum of $\eta_{\min} = 10^{-6}$. Gradients are clipped at ℓ_2 norm 0.5. Training runs for up to 300 epochs with early stopping at patience 120 epochs based on validation NLL.

The learning rate schedule is architecture-dependent. AeroCast uses CosineAnnealingLR [155] with $T_{\max} = 300$. The four baselines use CosineAnnealingWarmRestarts with $T_0 = 20$ and $T_{\text{mult}} = 2$; the periodic warm restarts empirically improve convergence for the shallower recurrent architectures.

5.6.4 Implementation Details

All projection matrices (embedding, attention, feed-forward, and output heads) are initialized with Xavier uniform initialization [156]. The sinusoidal positional encoding is fixed (non-learnable); layer normalization modules use standard learnable affine parameters. The PyTorch implementation uses `TransformerEncoderLayer` with Pre-LN and a terminal `LayerNorm` after all L layers, as described in Eq. (5.47). All experiments are conducted on a single NVIDIA GPU. The total parameter count for AeroCast is 13.15 M, of which 12.61 M reside in the four Transformer encoder layers and the remainder is split across the input embedding and three output heads.

5.7 Results and Analysis

5.7.1 Quantitative Comparison

Table 5.11 reports all evaluation metrics on the held-out validation split across nine trajectory types, evaluated over a 5-second (50-step) prediction horizon. All five models use $K=5$ mixture

Table 5.11

Quantitative comparison on 13,500 held-out validation sequences (5-second, 50-step horizon).

Method	ADE [m]	FDE [m]	mADE @5 [m]	mFDE @5 [m]	NLL	CRPS [m]	ECE
GRU-MDN	0.127	0.211	0.122	0.185	-10.31	0.169	0.674
LSTM-MDN	0.149	0.238	0.146	0.220	-9.37	0.197	0.676
BiGRU-MDN	0.111	0.190	0.108	0.175	-10.06	0.145	0.669
MLP-MDN	0.125	0.219	0.115	0.183	-11.40	0.157	0.645
AeroCast (ours)	0.055	0.099	0.052	0.088	-13.28	0.049	0.647

components and are trained under the protocol described in Section 5.6.

AeroCast achieves an ADE of 0.055 m and an FDE of 0.099 m, reducing both metrics by approximately 50% relative to BiGRU-MDN (ADE = 0.111 m, FDE = 0.190 m), which is the strongest baseline among the baselines. The best-of-5 metrics follow the same ordering: mADE@5 of 0.052 m and mFDE@5 of 0.088 m for AeroCast, compared with 0.108 m and 0.175 m for BiGRU-MDN. The small gap between single-mode and best-of- K results (0.055 vs. 0.052 m ADE) indicates that the dominant mixture component already tracks the ground truth closely; the remaining components contribute supplementary coverage rather than compensating for a misaligned primary mode.

AeroCast also records the lowest NLL (-13.28) and CRPS (0.049 m). The NLL margin over the next-best method, MLP-MDN (-11.40), is 1.88 nats per timestep, and the remaining baselines fall further behind. On CRPS, AeroCast attains roughly one-third the value of BiGRU-MDN (0.049 vs. 0.145 m), which is consistent with predicted distributions that are both accurate in location and appropriately spread.

Calibration, measured by ECE, is more tightly clustered across methods. MLP-MDN records 0.645, marginally below AeroCast at 0.647 while other baselines fall within a 0.03 band. This narrow spread suggests that the shared training elements, particularly the sigma floor at $\sigma_{\min}=0.05$ m and the NLL-dominated loss, govern calibration more than the choice of encoder. The ECE values across all methods (0.64–0.68) leave room for improvement through post-hoc recalibration or temperature scaling.

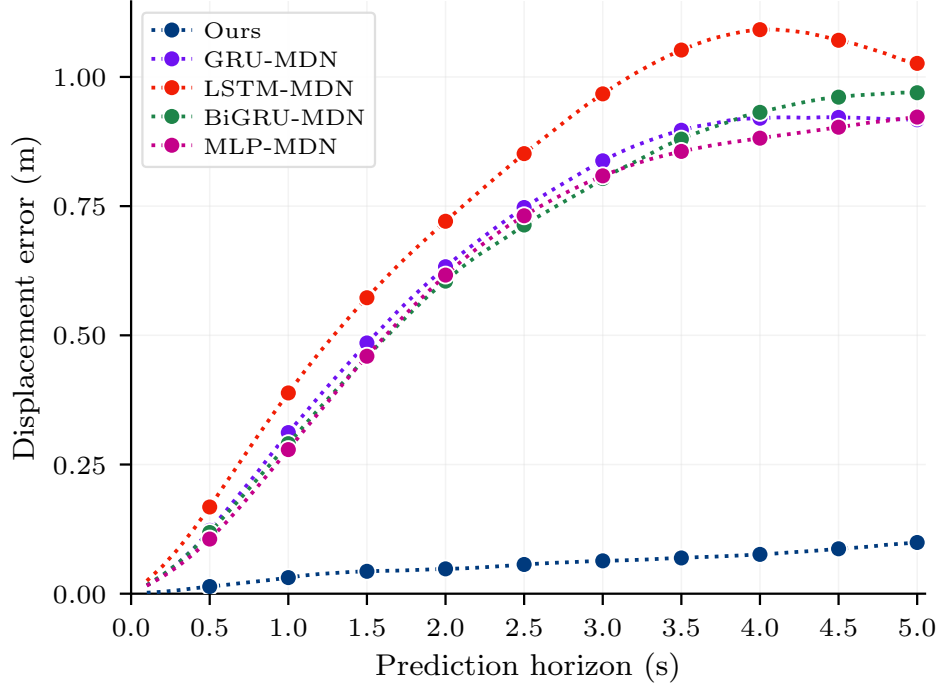


Figure 5.15
AeroCast position error at increasing prediction horizons over the full validation split.

Among the baselines themselves, BiGRU-MDN achieves the lowest trajectory error ($ADE = 0.111$ m), which is consistent with its bidirectional access to the observation window. LSTM-MDN places last on both ADE and FDE despite a larger parameter budget than GRU-MDN (6.07M vs. 4.75M), which suggests that the explicit cell-state mechanism does not confer an advantage at the 20-step observation length used here. MLP-MDN, which flattens the full window into a single vector, has achieved competitive NLL (-11.40) and the best ECE (0.645), though its trajectory-error performance ($ADE = 0.125$ m) lags behind the recurrent models with temporal inductive biases.

5.7.2 Prediction Horizon Analysis

Fig. 5.15 plots mean displacement error as a function of prediction horizon for all five methods. Error increases monotonically with horizon length for every model, as expected from displacement-based cumulative prediction. AeroCast maintains the lowest error at each evaluated horizon, and the gap relative to the baselines widens beyond the 2 s mark.

At short horizons (<1 s), all methods produce low errors that are within a narrow range of each other: near-future positions are tightly constrained by the current kinematic state regardless

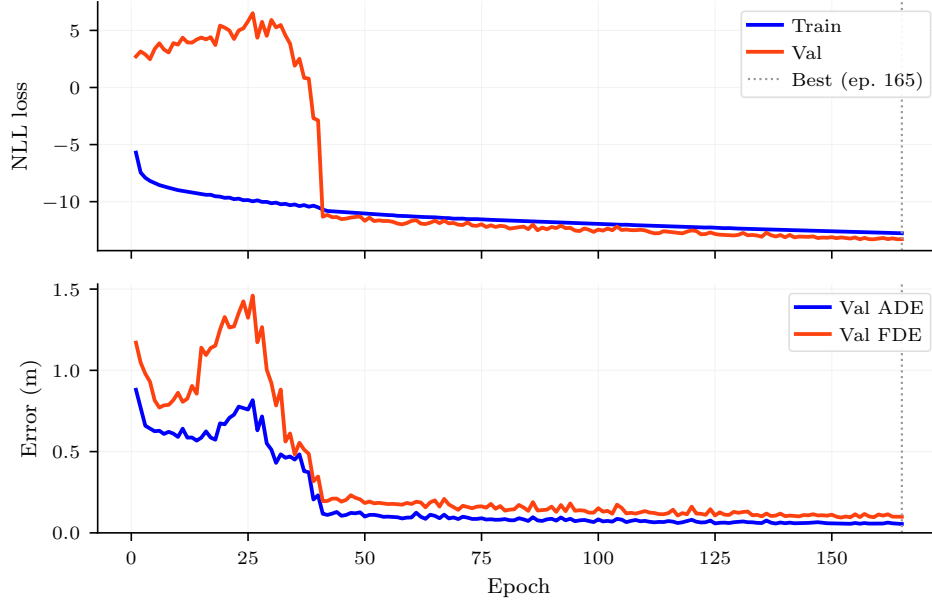


Figure 5.16
Validation and Training Loss Training Loss convergence over the epochs.

of encoder architecture. Beyond 2 s, the recurrent baselines diverge from the ground truth at a higher rate than AeroCast. This pattern is consistent with the hypothesis that global self-attention propagates early maneuver signatures across the full observation window more directly than the sequential hidden-state chain in recurrent encoders, though confirming this interpretation would require attention-map analysis beyond the scope of this evaluation.

Fig. 5.16 shows the training and validation loss curves. The validation NLL decreases steadily and stabilizes without the oscillatory divergence sometimes observed in pure-NLL training of mixture models [45]. The combined objective and sigma floor appear to regularize the optimization effectively.

5.7.3 Per-Trajectory-Type Analysis

Table 5.12 disaggregates ADE and FDE by trajectory category for all five methods. AeroCast achieves the lowest ADE and FDE on every category. The margin over the strongest baseline varies with the kinematic characteristics of each type.

On periodic categories (Circle, Oval, Figure-8, Trefoil), AeroCast records ADE values between 0.040 m and 0.061 m. BiGRU-MDN, the strongest baseline on these types, ranges from 0.081 m

Table 5.12

Per-trajectory-type ADE / FDE (m) on the held-out validation split (5-second, 50-step horizon).

Type	AeroCast		GRU-MDN		LSTM-MDN		BiGRU-MDN		MLP-MDN	
	ADE	FDE	ADE	FDE	ADE	FDE	ADE	FDE	ADE	FDE
Trefoil	0.060	0.121	0.141	0.261	0.165	0.292	0.122	0.233	0.138	0.270
Helix	0.044	0.085	0.103	0.183	0.120	0.206	0.089	0.165	0.092	0.204
Star	0.069	0.198	0.161	0.423	0.189	0.478	0.140	0.381	0.157	0.441
Oval	0.048	0.087	0.113	0.189	0.132	0.234	0.097	0.170	0.110	0.196
Figure-8	0.061	0.113	0.142	0.243	0.167	0.273	0.124	0.219	0.140	0.252
Circle	0.040	0.092	0.095	0.197	0.109	0.222	0.081	0.177	0.093	0.206
Random	0.109	0.386	0.254	0.824	0.297	0.931	0.221	0.540	0.249	0.857
Lissajous	0.069	0.116	0.161	0.249	0.189	0.281	0.140	0.223	0.158	0.259
Staircase	0.059	0.133	0.138	0.285	0.162	0.321	0.120	0.256	0.136	0.296

to 0.124 m. The difference shows that the Transformer encoder captures periodic phase structure at least as effectively as the bidirectional recurrent model, without relying on an explicit recurrent inductive bias.

Categories with non-trivial 3-D coupling (Helix, Lissajous, Staircase) show a similar pattern. AeroCast records 0.044 m on Helix, 0.069 m on Lissajous, and 0.059 m on Staircase; the corresponding BiGRU-MDN values are 0.089 m, 0.140 m, and 0.120 m. Staircase trajectories, characterized by slow vertical maneuvers with discrete altitude transitions, present a qualitatively different displacement distribution from the smooth periodic types, yet the relative improvement remains comparable.

The largest absolute errors for all methods occur on Random trajectories. AeroCast obtains ADE of 0.109 m and FDE of 0.386 m; BiGRU-MDN reaches 0.221 m and 0.540 m. The FDE reduction on Random (28.5%) is smaller than the reductions exceeding 45% observed on periodic types, which reflects the greater intrinsic difficulty of long-horizon prediction on aperiodic motion with genuine behavioral multi-modality. Star trajectories, which contain abrupt heading reversals at each arm tip, produce the second-highest AeroCast FDE (0.198 m), though this still represents a 48% reduction relative to BiGRU-MDN (0.381 m).

Fig. 5.17 corroborates these trends visually across representative samples from multiple categories. On structured trajectories such as Trefoil and Oval, AeroCast predictions closely follow the

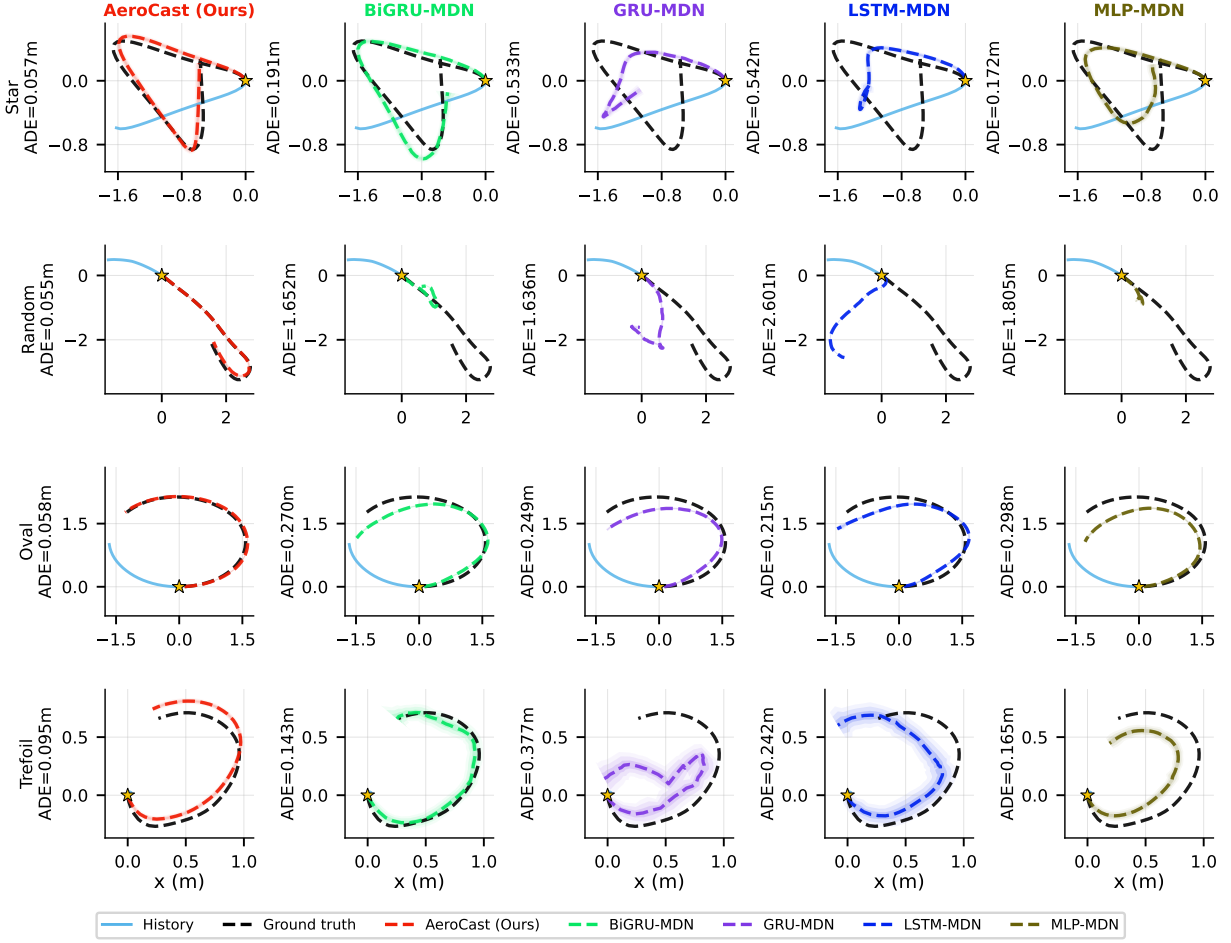


Figure 5.17

Qualitative comparison of AeroCast with the baseline models on various trajectory categories.

ground-truth geometry, and the baselines exhibit visible drift at curvature maxima. On Star trajectories, all methods incur larger errors at the arm tips, but AeroCast recovers the correct heading more rapidly after each reversal. Fig. 5.18 presents a detailed view of a single Trefoil sequence with 3-D overlay, planar projections, and per-step displacement error. The per-step error curve shows that AeroCast maintains lower error throughout the prediction window.

5.7.4 Ablation Study

Table 5.13 reports results from a systematic ablation of four design dimensions: mixture cardinality, model capacity, positional encoding, and input feature set. Each variant modifies exactly one dimension relative to the full model ($K=5$, $d=512$, $L=4$, sinusoidal positional encoding,

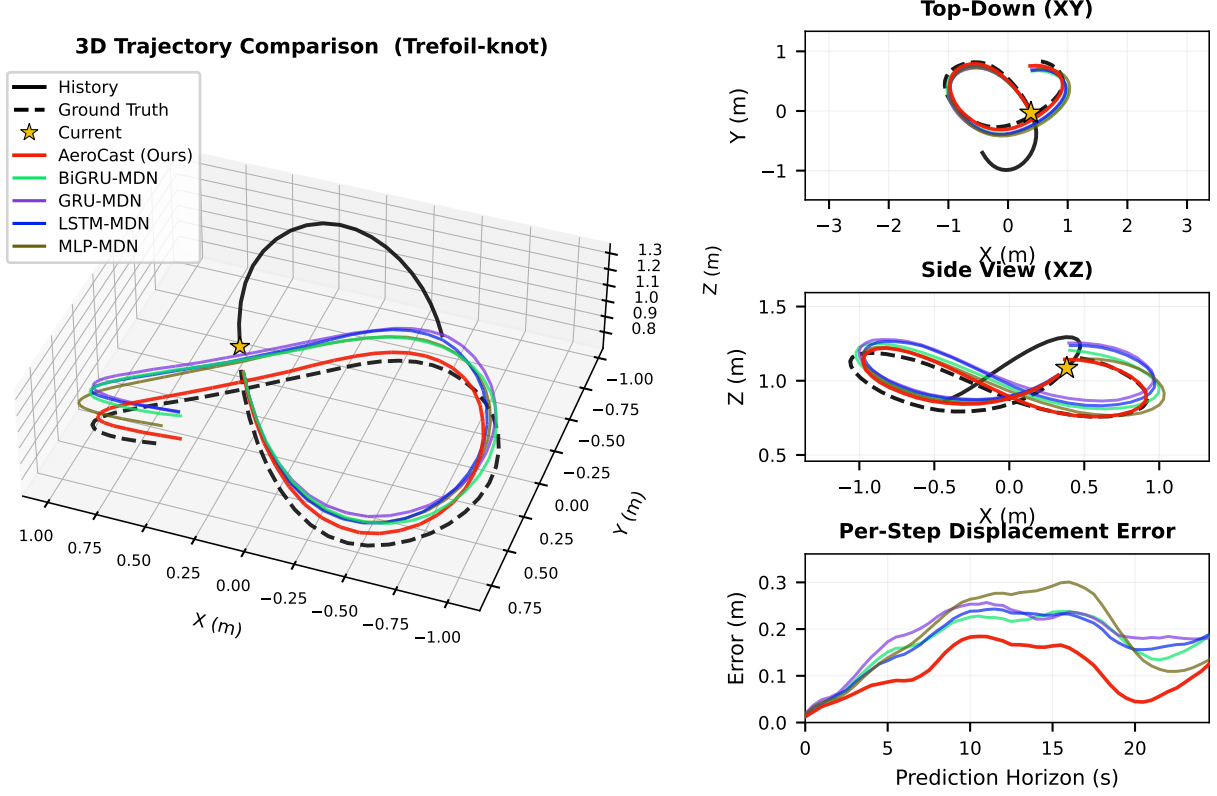


Figure 5.18
Qualitative comparison of AeroCast with the baseline models on a Trefoil trajectory.

6-D displacement-velocity input).

5.7.4.1 Mixture Cardinality

Reducing K from 5 to 1 (unimodal Gaussian) raises ADE from 0.055 m to 0.133 m and worsens NLL from -13.285 to -12.377 which shows that the multi-modal output head provides more capacity for covering the kinematic diversity present in the dataset. At $K=3$, ADE increases to 0.076 m, but ECE improves to 0.6405, the lowest among all ablation configurations. A fewer-component mixture thus yields slightly better-calibrated uncertainty at the expense of reduced trajectory accuracy. At $K=10$, NLL improves to -14.021 , but ADE rises to 0.101 m and ECE degrades to 0.6665. The additional components improve the global density fit but do not individually anchor to the ground-truth path, so the argmax-weight mode used for ADE/FDE evaluation is less precise. $K=5$ offers the best trade-off between distributional fit and dominant-mode accuracy.

Table 5.13
Ablation study on the held-out validation split.

Variant	Architecture					ADE↓	FDE↓	NLL↓	ECE↓
	K	d	L	PE	Inp	[m]	[m]		
Full model (ours)	5	512	4	✓	6	0.055	0.099	-13.285	0.647
<i>Number of mixture components K</i>									
$K = 3$	3	512	4	✓	6	0.076	0.130	-12.806	0.6405
$K = 10$	10	512	4	✓	6	0.101	0.163	-14.021	0.6665
$K = 1$ (uni.)	1	512	4	✓	6	0.133	0.213	-12.377	0.6536
<i>Model depth & width</i>									
Tiny ($d=128, L=2$)	5	128	2	✓	6	0.247	0.513	-12.354	0.6662
Medium ($d=256, L=4$)	5	256	4	✓	6	0.079	0.179	-12.898	0.6569
Small ($d=256, L=2$)	5	256	2	✓	6	0.112	0.223	-12.634	0.6628
<i>Positional encoding</i>									
No pos. enc.	5	512	4	×	6	0.118	0.410	-13.146	0.6600
<i>Input features</i>									
Pos.-only (3-D)	5	512	4	✓	3	0.098	0.187	-11.684	0.7883

5.7.4.2 Model Capacity

The Tiny variant ($d=128, L=2$, 1.85M parameters) incurs ADE of 0.247 m and FDE of 0.513 m, a fourfold degradation relative to the full model. Increasing to $d=256, L=2$ (Small) reduces ADE to 0.112 m; adding depth ($d=256, L=4$, Medium) brings it to 0.079 m. The full model ($d=512, L=4$) achieves 0.055 m, a further 30% reduction over the Medium variant. NLL improvements are more modest across this progression (−12.354 for Tiny to −13.285 for Full) that suggest that additional capacity improves the geometric precision of the dominant mixture component more than the overall distributional fit.

5.7.4.3 Positional Encoding

Removing the sinusoidal positional encoding increases ADE from 0.055 m to 0.118 m and FDE from 0.099 m to 0.410 m. The asymmetry is notable as ADE roughly doubles, but FDE increases by a factor of four. Without explicit temporal ordering, the model retains some ability to produce

reasonable displacement predictions on average by relying on token content alone, but it cannot maintain trajectory coherence through the full 50-step horizon, as reflected in the FDE degradation. NLL changes only modestly (-13.285 to -13.146), so the contribution of positional encoding is primarily to geometric accuracy rather than distributional fit.

5.7.4.4 Input Features

Removing velocity and retaining only 3-D displacements raises ADE from 0.055 m to 0.098 m (78% increase) and FDE from 0.099 m to 0.187 m. NLL drops from -13.285 to -11.684 , and ECE rises from 0.647 to 0.7883. The velocity signal conveys instantaneous heading and speed information that cannot be reliably recovered from a 20-step displacement history, particularly on trajectories where speed changes precede directional changes. The ECE degradation is the most severe observed in any single-factor ablation, which implies that velocity context is critical for both positional accuracy and for producing uncertainty estimates whose spread reflects actual kinematic variability.

5.7.5 Inference Latency

Table 5.14 reports model size and per-sample inference latency measured at batch size 1 on a single NVIDIA GPU. AeroCast completes a forward pass in 0.100 ms, within the 10 ms budget for a 100 Hz prediction loop and comparable to BiGRU-MDN (0.125 ms). GRU-MDN and LSTM-MDN are faster (0.042 and 0.056 ms) due to smaller parameter counts. MLP-MDN is the fastest at 0.004 ms. All five methods fall well within a single control cycle at 100 Hz; the computational bottleneck in an onboard deployment would reside in the upstream perception pipeline rather than in the prediction step.

5.7.6 Discussion

Across all seven evaluation metrics and all nine trajectory categories, AeroCast either matches or improves upon every baseline. The trajectory-error and distributional advantages are consistent regardless of whether the motion is periodic, 3-D-coupled, or aperiodic.

On periodic trajectories, where a recurrent model could in principle memorize the repeating

Table 5.14

Comparison of model complexity and inference latency (with batch size 1 over 50 runs after warm-up)

Method	Parameters	Size (MB)	Latency (ms)
AeroCast (ours)	13.15M	54.0	0.100
GRU-MDN	4.75M	19.0	0.042
LSTM-MDN	6.07M	24.3	0.056
BiGRU-MDN	13.17M	52.7	0.125
MLP-MDN	7.63M	30.5	0.004

displacement pattern, AeroCast still attains lower ADE. It is due to the fact that self-attention over the full 20-step window allows the encoder to identify the instantaneous phase of the motion more directly than a recurrent encoder that must propagate phase information through intermediate hidden states. On aperiodic types, the same global connectivity may help the model detect early kinematic signatures of heading changes that sequential processing would attenuate. Both of these analyses are our own hypotheses that would require attention-weight analysis to confirm.

The ablation results rank velocity input and model capacity as the two largest contributors to prediction quality. Positional encoding plays a significant role and its removal has a limited effect on ADE but quadruples FDE, that means temporal ordering is essential specifically for long-horizon trajectory coherence. The finding that velocity removal degrades ECE from 0.647 to 0.7883, more severely than any other single-factor ablation, points to a concrete mechanism: without instantaneous speed information, the model cannot differentiate slow-moving obstacles (whose future positions occupy a compact region) from fast-moving ones (whose future positions are more dispersed), resulting in miscalibrated predictive spreads.

Several limitations qualify the conclusions drawn above. All experiments use a single dataset comprising indoor quadrotor flights and parametrically generated synthetic trajectories; generalization to outdoor environments, larger vehicles, or biological aerial agents remains untested. The evaluation considers single-obstacle scenarios; multi-obstacle settings, where inter-agent interactions introduce additional behavioral modes, would require architectural extensions.

5.8 Conclusion

This chapter presented AeroCast, a probabilistic trajectory prediction framework for non-cooperative aerial obstacles that pairs a Pre-LN Transformer encoder with a Mixture Density Network output head. Three design choices underpin the approach: a consecutive displacement encoding that enforces translation invariance and bounded input statistics, a combined NLL and mode-anchoring training objective with a physically motivated sigma floor, and curriculum sequence-length scheduling for early training stability.

AeroCast is evaluated against four baselines on a hybrid real-and-synthetic quadrotor dataset spanning nine motion categories. AeroCast reduces ADE and FDE by approximately 50% relative to the strongest baseline (BiGRU-MDN) and achieves the lowest NLL and CRPS among all compared methods. These improvements hold across all nine trajectory types, with the largest absolute gains on aperiodic and structurally complex categories. Ablation analysis identifies velocity input as the single most important feature for both trajectory accuracy and calibration quality, and positional encoding as essential for maintaining prediction coherence over the full 5-second horizon. AeroCast inference latency (0.100 ms per sample) remains within the real-time budget for onboard deployment at 100 Hz.

Future work includes extension to multi-obstacle settings with inter-agent interaction modeling, online adaptation to specific obstacle agents observed over time, and integration with a receding-horizon planner that consumes the predicted GMM distributions as probabilistic obstacle constraints.

CHAPTER VI: NANOBENCH: A MULTI-TASK BENCHMARK DATASET FOR NANO-QUADROTOR SYSTEM IDENTIFICATION, CONTROL, AND STATE ESTIMATION

Abstract

Existing aerial-robotics benchmarks target vehicles from hundreds of grams to several kilograms and typically expose only high-level state data. They omit the actuator-level signals required to study nano-scale quadrotors, where low-Reynolds-number aerodynamics, coreless DC motor nonlinearities, and severe computational constraints invalidate models and controllers developed for larger vehicles. We introduce *NanoBench*, an open-source multi-task benchmark collected on the commercially available Crazyflie 2.1 nano-quadrotor (takeoff weight 27 g) in a Vicon motion capture arena. The dataset contains over 170 flight trajectories spanning hover, multi-frequency excitation, standard tracking, and aggressive maneuvers across multiple speed regimes. Each trajectory provides synchronized Vicon ground truth, raw IMU data, onboard extended Kalman filter estimates, PID controller internals, and motor PWM commands at 100 Hz, alongside battery telemetry at 10 Hz, aligned with sub-0.5 ms consistency. NanoBench defines standardized evaluation protocols, train/test splits, and open-source baselines for three tasks: nonlinear system identification, closed-loop controller benchmarking, and onboard state estimation assessment. To our knowledge, it is the first public dataset to jointly provide actuator commands, controller internals, and estimator outputs with millimeter-accurate ground truth on a commercially available nano-scale aerial platform. All data, code, and evaluation scripts are publicly available.¹

6.1 Introduction

Learning-based methods have substantially advanced quadrotor autonomy. Deep reinforcement learning policies match professional pilots in drone racing [157], adaptive controllers trained on minutes of flight data enable agile flight in strong wind [158], and data-driven model predictive controllers that incorporate learned residual dynamics outperform rigid-body baselines [159]. These techniques require instrumented datasets for training and reproducible evaluation. Most publicly available datasets that support such evaluation were collected on platforms weighing hundreds of grams to several kilograms. For nano-scale quadrotors, platforms below 50 g, which

¹<https://github.com/syediu/nanobench-iro2026.git>

are widely used in embedded autonomy, safe learning, swarm robotics, autonomous navigation and mapping, and collision-aware trajectory planning [160, 161, 162, 73, 111, 141], there is currently no open benchmark that exposes the full closed-loop stack.

Nano-scale quadrotors such as the Crazyflie 2.1 (27 g takeoff mass) operate in aerodynamic regimes that differ qualitatively from those of larger vehicles. Propellers with diameters below 50 mm generate airflows at Reynolds numbers on the order of 10^4 , where laminar separation bubbles and viscous losses significantly affect thrust and torque characteristics [163]. Coreless DC motors introduce deadbands and response lags absent in brushless propulsion [70]. Proximity to surfaces induces severe ground effect disturbances at low altitudes [164]. Computationally, a 168 MHz Cortex-M4 microcontroller strictly limits algorithms to lightweight extended Kalman filters [165], and therefore demands highly optimized software architectures.

These physical and computational constraints invalidate assumptions derived from larger vehicles, yet no standardized benchmark exists to study nano-scale dynamics under these constraints. Thrust models transfer poorly across Reynolds number regimes [166]. Policies trained in simulation degrade on physical hardware due to inaccurate thrust modeling [71, 167]. Lightweight estimators require precisely synchronized multi-modal recordings for proper validation [168]. Despite this demand, no existing dataset provides actuator commands, controller internals, and estimator outputs with external ground truth on a commercially available nano-scale platform. Researchers currently evaluate system identification, control, and state estimation algorithms on private or synthetic data, precluding systematic comparison [149].

This paper presents *NanoBench*, an open-source multi-task benchmark that addresses this gap. The dataset is collected on the standard Crazyflie 2.1 quadrotor inside a Vicon motion capture arena using a data collection framework based on direct radio communication via the cflib Python library. Our contributions are as follows:

1. **Nano-quadrotor flight dataset.** We release over 170 trajectories on the Crazyflie 2.1 covering hover, multi-frequency excitation, standard tracking, and aggressive flight across multiple speed regimes. Every trajectory includes time-synchronized Vicon ground truth,

raw IMU measurements, onboard EKF state estimates, PID controller internals, and motor PWM commands at 100 Hz, with battery telemetry at 10 Hz.

2. **Cross-correlation time alignment.** We develop and validate a synchronization procedure that cross-correlates onboard gyroscope angular rates with Vicon-derived angular velocity to estimate and correct the firmware-to-host clock offset. Across all flights, the residual misalignment is below 0.5 ms.
3. **Multi-task evaluation suite.** We define train/test splits, metrics, and reporting conventions for three tasks — system identification, controller benchmarking, and state estimation — and release open-source baseline implementations that establish reference performance levels on each.
4. **A standardized multi-task evaluation protocol.** Defined train/test splits, metrics, and reporting conventions for system identification, controller benchmarking, and state estimation, with open-source baseline implementations establishing reference performance levels for each task.

The remainder of this paper is organized as follows. Section 6.2 positions NanoBench against prior literature. Section 6.3 describes the hardware platform, acquisition pipeline, synchronization procedure, trajectory design, and benchmark task formulations. Section 6.4 details the experimental setup, dataset composition, and benchmark evaluations. Section 6.5 discusses limitations and conclusions.

6.2 Related Work

6.2.1 Aerial Robot Datasets and Benchmarks

Aerial robotics datasets can be partitioned by the downstream tasks they support and the platform scales they cover. The EuRoC MAV dataset [169], UZH-FPV [170], NTU VIRAL [171], and INSANE [172] provide high-fidelity exteroceptive sensor data for visual-inertial odometry and multi-sensor fusion. However, these benchmarks treat the vehicle as a passive sensor carrier and

omit actuator and control signals. As a result, they cannot support closed-loop dynamics modeling, controller evaluation, or estimator benchmarking.

Actuation-aware datasets capture the inputs necessary for system identification. The Blackbird dataset [173] and Agilicious framework [174] log motor commands for agile flight research. However, they feature quadrotors weighing near 1 kg equipped with brushless motors. These heavy platforms exhibit rigid-body dynamics that differ fundamentally from the coreless DC actuation and severe aerodynamic nonlinearities of nano-quadrotors, precluding direct transfer of models or control policies.

At the nano scale, Busetto et al. [76] recently released a system identification dataset for a brushless Crazyflie variant that logs motor RPM. The dataset covers only four trajectories, omits onboard EKF and PID logs entirely, and records no battery voltage. Table 6.15 details these gaps. These omissions restrict the benchmark to single-task system identification on non-standard hardware that leave multi-task evaluation on the commercially standard Crazyflie 2.1 unaddressed.

6.2.2 System Identification for Aerial Vehicles

Quadrotor dynamics modeling ranges from calibrated physics to fully learned representations. Forster [70] established baseline quadratic thrust parameters via least-squares fitting. Torrente et al. [159] and NeuroBEM [166] augmented nominal rigid-body equations with data-driven residuals to capture unmodeled aerodynamic effects. Neural ordinary differential equations offer purely data-driven alternatives. Zhou et al. [175] trained Neural ODEs for multi-step state prediction, while KNODE-MPC [176] and O’Connell et al. [158] embedded adapted models within predictive controllers to improve tracking accuracy.

Crucially, existing simulation ecosystems configure these learned and analytical models with static coefficients [177]. This constant-parameter assumption ignores the severe thrust degradation caused by single-cell LiPo battery discharge. No existing benchmark provides the synchronized voltage and kinematics data required to model this non-stationary battery effect.

Table 6.15
Comparison of aerial robotics datasets.

Feature	EuRoC [169]	Blackbird [173]	UZH-FPV [170]	Busetto et al. [76]	NanoBench (Ours)
Platform mass	~2 kg	~1 kg	~0.8 kg	<50 g	~27 g
Platform availability	Discontinued	Custom	Custom	Commercial	Commercial
Number of trajectories	11	176	27	4	170+
Ground truth system	Vicon/Leica	OptiTrack	Leica	MoCap	Vicon
Ground truth rate	200 Hz	360 Hz	200 Hz	Not specified	100 Hz
Raw IMU data	✓	✓	✓	–	✓
Motor commands	–	✓(RPM)	–	✓(RPM)	✓(PWM)
Onboard estimator output	–	–	–	–	✓
Controller internals	–	–	–	–	✓
Battery telemetry	–	–	–	–	✓
Camera data	Stereo	Synthetic	Stereo + event	–	–
Multi-task evaluation	–	–	–	SysID only	SysID + Control + Estimation
Calibration tools	–	–	–	–	✓
Open-source	✓	✓	Partial	✓	✓

6.2.3 Control of Nano-Quadrotors

The Crazyflie serves as a primary testbed for validating control architectures. Researchers routinely implement cascaded PID loops, geometric controllers on $SE(3)$, and minimum-snap trajectory trackers [33]. Safe reinforcement learning frameworks mostly utilize the Crazyflie platform [161], with recent studies achieving zero-shot sim-to-real transfer [167]. Song et al. [71] demonstrated that thrust calibration errors represent the dominant source of tracking degradation during this transfer process. Open-source software ecosystems simplify swarm coordination and simulated execution [73, 162, 177, 12], and lower the barriers to entry for multi-agent research.

Despite this active development, each research group runs its own private flight experiments and reports numbers against its own reference trajectories. Without a shared dataset, differences in results between papers reflect hardware setup as much as algorithm design.

6.2.4 State Estimation for Micro Aerial Vehicles

State estimation on sub-50 g platforms faces extreme hardware limitations. Visual-inertial odometry pipelines are simply too expensive for a 168 MHz Cortex-M4 [160], so nano-quadrotors run lightweight EKFs with carefully characterized IMU noise models [168, 165].

Evaluating these filters properly requires the estimator’s internal state to be recorded alongside an external position reference at matched timestamps. Without that pairing, it is not possible to

Table 6.16

Recorded signals and nominal sampling rates.

Signal	Source	Rate (Hz)
Position ($\mathbf{p} \in \mathbb{R}^3$), orientation ($\mathbf{q} \in S^3$)	Vicon	100
Linear velocity ($\dot{\mathbf{p}} \in \mathbb{R}^3$)	Vicon (derived)	100
Accelerometer ($\mathbf{a}_{\text{IMU}} \in \mathbb{R}^3$) [G]	BMI088	100
Gyroscope ($\boldsymbol{\omega}_{\text{IMU}} \in \mathbb{R}^3$) [rad/s]	BMI088	100
EKF state ($\hat{\mathbf{p}}, \hat{\mathbf{p}}, \hat{\boldsymbol{\eta}}$)	Firmware	100
Controller setpoints ($\mathbf{p}_{\text{ref}}, \psi_{\text{ref}}$)	Firmware	100
PID outputs (roll, pitch, yaw, thrust)	Firmware	100
Motor PWM ($u_1, u_2, u_3, u_4 \in [0, 65535]$)	Firmware	100
Battery voltage V_{bat} [V]	Firmware	10

separate sensor noise from filter divergence or to compare estimators across different hardware runs.

6.3 Methodology

6.3.1 Hardware Platform

We used the Crazyflie 2.1 nano-drone (Bitcraze AB), as the data collection platform. The vehicle measures $92 \times 92 \times 29$ mm and weighs 27 g with a 250 mAh single-cell LiPo installed. Four 7×16 mm coreless DC motors drive 46 mm four-blade propellers; at full charge (4.2 V), combined peak thrust is approximately 0.6 N, giving a thrust-to-weight ratio of roughly 2.2. The onboard STM32F405 (168 MHz, Cortex-M4F, 192 kB SRAM) runs a cascaded PID controller and extended Kalman filter; a secondary nRF51822 handles radio communication. Inertial data is provided by a Bosch BMI088 six-axis IMU. Motor commands are 16-bit PWM values in $[0, 65535]$, and the unregulated single-cell battery directly couples state of charge to actuator performance.

Ground truth 6-DoF pose is provided by a 12-camera Vicon system covering a $6 \times 4 \times 2$ m volume. Retroreflective markers (6.4 mm diameter) in an asymmetric configuration enable unique rigid-body identification at 100 Hz with sub-millimeter accuracy.

6.3.2 Data Acquisition Pipeline

Data collection uses a custom ROS 1 framework that communicates with the Crazyflie via cflib over a Crazyradio PA USB dongle. Three concurrent data paths operate during each flight:

Vicon path. A `vrpn_client_ros` node publishes `geometry_msgs/PoseStamped` messages on a per-rigid-body topic at 100 Hz. The `ViconRecorder` module subscribes to this topic, computes linear velocity via first-order backward difference, converts quaternion orientation to Euler angles, and writes each sample to a CSV file. On every callback, the pose is also forwarded to the Crazyflie EKF via `cflib's send_extpose()` to provide the onboard estimator with an external position reference.

Firmware telemetry path. The `CflibLogger` module configures firmware log blocks as specified in a YAML configuration file. Each block declares a set of firmware variables, their data types, and a polling frequency. The `cflib` library fetches these variables over the CRTP radio protocol. Upon reception, each sample is timestamped with both the host wall-clock time and the Crazyflie firmware tick (milliseconds since boot) and written to a per-block CSV file. Table 6.16 enumerates the recorded signals.

Command path. The `ExperimentRunner` orchestrates the flight sequence. High-level commands (`takeoff`, `land`) use the firmware's onboard trajectory planner, which generates smooth polynomial setpoints internally. During trajectory execution, position setpoints are streamed at 100 Hz via `send_position_setpoint()`, which bypasses the high-level planner and feeds the PID controller directly.

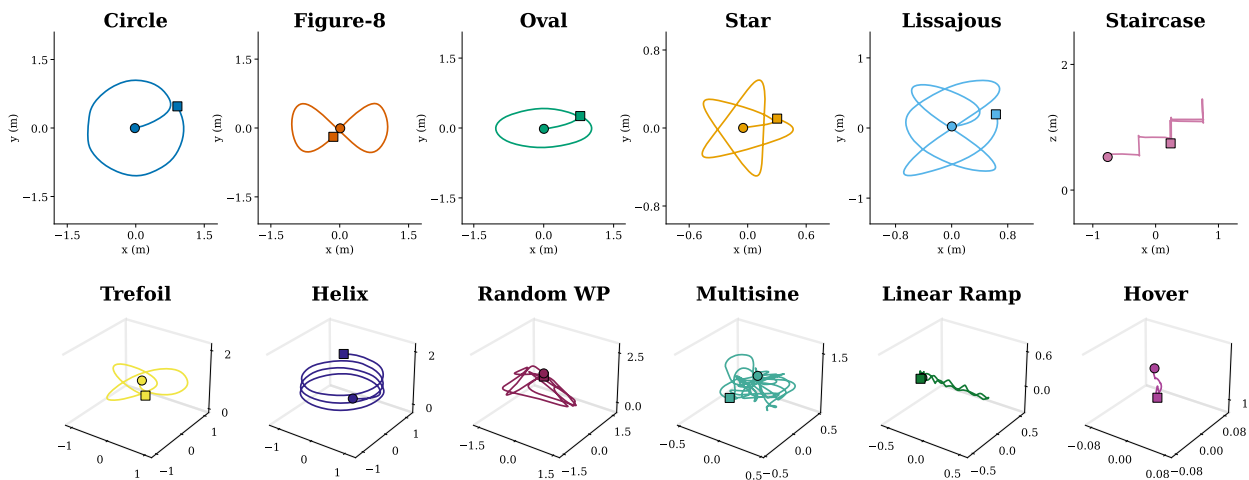


Figure 6.19
NanoBench trajectory types visualized from recorded Vicon ground-truth data.

Table 6.17

Category B: Tracking Trajectory Parameters.

Trajectory Type	Complexity Mode	Scale (m)	Speed (m/s)
Circle	Continuous yaw	$r = 1.0$	0.5, 0.75, 1.0
Figure-8	Direction reversal	$A \in \{1.0, 1.3\}$	0.5, 0.75, 1.0
Oval	Asymmetric axes	1.0×0.4	0.5, 0.75, 1.0
Star	Sharp transients	$r \in \{0.6, 1.0\}$	0.5, 0.75, 1.0
Lissajous (3:2)	Dense planar	$A = 0.6$	0.5, 0.75, 1.0
Trefoil Knot	3D oscillation	$\Delta z = 0.15$	0.5, 0.75, 1.0
Helix	Sweeping altitude	$\Delta z = 1.0$	0.5, 0.75, 1.0
Linear Ramp	1D acceleration	$L = 2.0$	0.5, 0.75, 1.0
Random Waypoint	Unstructured 3D	Volume bound	0.75
Staircase	Vertical steps	$\Delta z = 0.5$	0.5

6.3.3 Time Synchronization

Three clocks run independently during each flight: the Vicon Tracker, the Crazyflie firmware tick counter, and the ROS host. The host timestamps Vicon poses and firmware packets on arrival, which places both streams in a shared reference frame. Residual alignment offset arise from radio transmission latency (typically 2 ms to 5 ms) and operating system scheduling jitter.

We estimate this offset by cross-correlating the onboard gyroscope $\omega_{\text{gyro}}(t)$ with the angular velocity $\omega_{\text{vicon}}(t)$ obtained by differentiating Vicon quaternions through a Savitzky–Golay filter. The time offset Δt^* is estimated as

$$\Delta t^* = \arg \max_{\Delta t} \sum_k \omega_{\text{gyro}}(t_k) \cdot \omega_{\text{vicon}}(t_k + \Delta t) \quad (6.61)$$

where the search range is $\Delta t \in [-2, 2]$ s at 1 ms resolution. This formulation exploits the sharp rotational features present during flight maneuvers to localize the offset with sub-millisecond precision.

Once Δt^* is found, firmware signals are shifted to the Vicon time base and resampled onto a uniform grid by linear interpolation. Each trajectory is then written to a single csv file with all modalities on one time axis.

6.3.4 Trajectory Design

The trajectory taxonomy is organized into three active categories designed to collectively excite all dynamic modes of the platform. The dataset encompasses 12 distinct trajectory types spanning system identification excitation, geometric tracking at multiple speeds, and long-duration battery-drain hovers. Fig. 6.19 visualizes all 12 types from recorded flight data.

Category A: System identification excitation. Multi-sine trajectories superimpose sinusoidal position commands at logarithmically spaced frequencies from 0.1 Hz to 5 Hz:

$$x(t) = \sum_{n=1}^{N_f} \frac{A}{n} \sin(2\pi f_n t), \quad y(t) = \sum_{n=1}^{N_f} \frac{A}{n} \cos(2\pi f_n t + \frac{\pi}{4}) \quad (6.62)$$

where A is the amplitude, f_n are logarithmically spaced, and the phase offset between axes ensures simultaneous two-axis excitation. Category A consists of a single trajectory (A1b_multisine_sysid), which executes a 60 s 3D multi-sine at altitude ≈ 1.1 m with $N_f = 15$ components spanning $[0.1, 5]$ Hz, amplitudes of 0.8 m (horizontal) and 0.35 m (vertical), and smooth 2 s fade-in/out windows.

Category B: Standard trajectory tracking. Ten geometric trajectories exercise complementary dynamic modes at multiple speeds. Table 6.17 defines the parameterization for this suite. Most shapes are parameterized symmetrically, such as the circular paths defined by $x(t) = r \cos(\omega t)$ and $y(t) = r \sin(\omega t)$ where $\omega = v/r$. Execution speeds range from 0.5 m s^{-1} to 1.0 m s^{-1} , representing the practical operating envelope for nano-quadrotors in confined environments.

Category C: Battery-drain. Two long-duration hover flights (C4_battery_drain) provide continuous recordings from ≈ 4.2 V down to ≈ 3.1 V while the vehicle holds position at ≈ 1.0 m, explicitly capturing voltage-related degradation across the full discharge curve. These flights last 101–134 s and serve both the voltage-conditioned thrust model and as battery-state benchmarks for all three tasks.

6.3.5 Benchmark Task Formulation

NanoBench establishes standardized protocols for three distinct evaluation tasks.

Task 1: System Identification. This task requires algorithms to predict the 6-DoF state trajectory $\hat{\mathbf{x}}(t)$ over horizons $h \in \{0.1, 0.5, 1.0\}$ s given initial conditions and motor commands $\{u_i^{(k)}\}$. Formulations must model rigid-body dynamics in the world frame $\mathcal{W}$:

$$m\ddot{\mathbf{p}} = \mathbf{R}(\mathbf{q}) \begin{bmatrix} 0 \\ 0 \\ \sum_i T_i \end{bmatrix} - mg \mathbf{e}_3 - \mathbf{D}_t \dot{\mathbf{p}} \quad (6.63)$$

$$\mathbf{J} \dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega} - \mathbf{D}_r \boldsymbol{\omega} \quad (6.64)$$

where $\mathbf{R}(\mathbf{q}) \in SO(3)$ is the rotation from body to world frame, $\mathbf{D}_t$ and $\mathbf{D}_r$ are diagonal drag matrices, and $\boldsymbol{\tau}$ is the torque vector. Training utilizes Category A data at nominal voltage ($V > 3.8$ V); testing utilizes Categories B and C across the full discharge curve.

For each rollout initialized at $t \in \mathcal{T}$, the model generates open-loop predictions $\hat{\mathbf{y}}_{t+h|t}$ for $h = 1, \dots, H$. The component-wise prediction error at horizon h is

$$e_{t,h}^{(c)} = \begin{cases} \left\| \mathbf{x}_{t+h}^{(c)} - \hat{\mathbf{x}}_{t+h|t}^{(c)} \right\|_2, & c \in \{p, v, \omega\}, \\ 2 \operatorname{atan2}(\|q_v\|_2, q_w), & c = R, \end{cases} \quad (6.65)$$

where $x^{(c)}$ denotes the corresponding Euclidean state component and $[q_v \ q_w]^\top = q_{t+h}^{-1} \otimes \hat{q}_{t+h|t}$ is the relative quaternion. The mean absolute error at horizon h and its cumulative H -step variant are

$$\operatorname{MAE}^{(c)}(h) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} e_{t,h}^{(c)}, \quad \operatorname{MAE}_{1:H}^{(c)} = \sum_{h=1}^H \operatorname{MAE}^{(c)}(h). \quad (6.66)$$

Task 2: Controller Benchmarking. This task evaluates closed-loop geometric tracking performance. Controllers execute paired Category B reference trajectories $\mathbf{p}_{\text{ref}}(t)$, and all evaluation pairs are subsetting to matched initial battery voltages ($\Delta V \leq 0.05$ V) to isolate controller performance from voltage-induced thrust variation. The position tracking error at time step k is

$$e_p(k) = \left\| \mathbf{p}(t_k) - \mathbf{p}_{\text{ref}}(t_k) \right\|_2 \quad (6.67)$$

Aggregate performance is reported as tracking RMSE and 95th-percentile deviation across all k . Control effort is reported as

$$E_u = \frac{1}{N} \sum_{k=1}^N \sum_{i=1}^4 u_i(k)^2 \quad (6.68)$$

For real-flight controllers, each condition is repeated five times, offline controllers use deterministic rollouts and are reported without repetition.

Task 3: State Estimation. This task isolates the accuracy of lightweight onboard estimators against Vicon ground truth. The estimator is configured at takeoff and runs exclusively for the duration of that flight. Estimator output $\hat{\mathbf{p}}(t_k)$ is evaluated against ground truth $\mathbf{p}_{\text{gt}}(t_k)$ using Absolute Trajectory Error (ATE), computed after aligning the estimated trajectory to ground truth via Horn’s $SE(3)$ method:

$$\text{ATE} = \left(\frac{1}{N} \sum_{k=1}^N \left\| \mathbf{p}_{\text{gt}}(t_k) - \mathbf{T} \hat{\mathbf{p}}(t_k) \right\|_2^2 \right)^{1/2} \quad (6.69)$$

where $\mathbf{T}$ is the rigid transformation returned by the alignment. Additional metrics include Relative Trajectory Error (RTE) over localized sliding windows, per-axis velocity residuals, and attitude RMSE in Euler angles. Each condition is repeated five times per estimator per trajectory type; results are reported as mean $\pm$ standard deviation across runs.

6.4 Experimental Validation and Results

6.4.1 Dataset Statistics

NanoBench contains 12 fully post-processed trajectories spanning Categories A through C, with a total aligned flight time of approximately 97.5 min and 603,942 synchronized samples at 100 Hz. Table 6.18 reports per-category statistics.

Battery voltage across the dataset spans 3.08 V to 4.2 V, which covers the full operational discharge curve of a single-cell LiPo. The Category B tracking trajectories operate in the mid-to-nominal voltage regime (3.8 V to 4.2 V). Category C battery-drain hovers capture the complete discharge from ≈ 4.2 V to ≈ 3.1 V.

The synchronization pipeline (Section 6.3.3) was validated across all 170 post-processed trajectories. The estimated clock offset between the firmware and Vicon time bases has a median of

Table 6.18

Trajectory categories in the NanoBench dataset.

Category	Trajectory types	Flight time	Samples
A: Excitation	Multi-sine (3D)	8.5 min	61,404
B: Tracking	Circle, figure-8, oval, star, trefoil, lissajous, helix, ramp, random waypoints, staircase	82.0 min	491,970
C: Battery-drain	Battery-drain hover	7.0 min	50,568
Total		97.5 min	603,942

282 ms with a standard deviation of 17.3 ms across flights, which is within the latency of the radio link. After correction, residual alignment precision is limited by the cross-correlation resolution (1 ms search grid at 100 Hz) and the peak correlation coefficient, which ranges from 0.51 (staircase climb, low angular excitation) to 0.94 (multisine, high angular excitation).

6.4.2 Task 1: System Identification Baselines

6.4.2.1 Baselines

We evaluated five baseline models of increasing complexity, to benchmark Task 1. All models are trained on the `multisine` excitation flight (80/20 train/validation split) and tested on held-out Category B tracking trajectories, listed in Table 6.17. Below, we briefly explain the baselines for Task 1, which are adapted from Busetto et al. [76].

1. **Naive baseline.** A constant extrapolation model that predicts $\hat{\mathbf{x}}(t_k + h) = \mathbf{x}(t_k)$ for all horizons h . This serves as a simple lower bound.
2. **Physics model.** A rigid-body dynamics model implementing (6.63)–(6.64) with a quadratic motor-thrust map and fourth-order integration at 100 Hz. The baseline airframe weighs 27 g, with three retroreflective markers and the wireless charging deck attached, the total flying mass is 40.85 g. All dynamics models use the measured flying mass, and inertia and drag parameters are taken from [76]. This model is not learned; it uses calibrated but fixed parameters.

3. **Residual MLP.** A feedforward network operating in the 12D state space (position, velocity, $SO(3)$ rotation vector, angular velocity) predicts a residual $\Delta \mathbf{x}$ on top of the current state. The network consists of five fully connected layers with 64 hidden units and ReLU activations. It is trained to minimize a horizon-weighted mean-squared error over 50-step prediction windows using Adam and cosine-annealed learning rate.
4. **Physics + residual.** A hybrid model that composes the physics baseline with the residual MLP. The physics model is run in closed loop to obtain a normalized prediction $\hat{\mathbf{x}}_{\text{phys}}$, and the residual network predicts a correction in the same normalized space.
5. **LSTM.** A recurrent model that conditions on the initial state $\mathbf{x}_0$ and the sequence of motor commands. A single-layer LSTM with 64 hidden units processes the control sequence and outputs per-step state increments, which are accumulated to obtain the predicted trajectory. As in the MLP case, training minimizes a horizon-weighted MSE over 50-step windows.

For all learned models we use batch size 256, an exponential temporal weighting factor $\lambda = 0.1$ in the loss, and train for 500 epochs.

6.4.2.2 Results and Analysis

Table 6.19 reports MAE per state component at horizons $h \in \{1, 10, 50\}$ steps and the cumulative score over $h = 1 \dots 50$. Based on Table 6.19 and Fig. 6.20, we find the following three observations:

First-principles accuracy degrades rapidly beyond short horizons. At $h=1$ (10 ms), the physics model achieves sub-millimeter position MAE (0.3 mm), significantly lower than naive extrapolation (5.7 mm), which shows that rigid-body dynamics accurately describe the immediate state evolution of the platform. However, the advantage reverses sharply at longer horizons, by $h=50$, physics-model position MAE reaches 636 mm, 2.2 times worse than naive. The velocity channel reveals the mechanism. Physics-model velocity MAE grows from 51 mm s^{-1} at $h=1$ to 2.57 m s^{-1} at $h=50$, which indicates that small thrust-modeling errors compound through numerical integration and corrupt the translational state.

Table 6.19
Numerical performance of cumulative simulation error at $h = 1, 10, 50$.

Model	MAE _{p,h} [m]			MAE _{v,h} [m/s]			MAE _{R,h} [rad]			MAE _{ω,h} [rad/s]		
	$h=1$	$h=10$	$h=50$	$h=1:50$	$h=1$	$h=10$	$h=50$	$h=1:50$	$h=1$	$h=10$	$h=50$	$h=1:50$
Naive	0.0057	0.0585	0.2866	7.3866	0.0085	0.0786	0.3542	9.4178	0.0033	0.0285	0.0915	2.7563
Physics	0.0003	0.0265	0.6364	10.9641	0.0511	0.5085	2.5673	65.0322	0.0019	0.0198	0.1425	3.2693
Residual	0.0045	0.0445	0.2282	5.6810	0.0121	0.1096	0.6708	16.9002	0.0084	0.1051	0.1971	8.8737
Phys+Res	0.0036	0.0121	0.2621	4.4071	0.0201	0.1370	1.0771	26.7469	0.0065	0.1359	0.1825	9.8645
LSTM	0.0058	0.0332	0.2562	5.9194	0.0199	0.1077	0.4782	11.9639	0.0162	0.1339	0.3059	9.8302

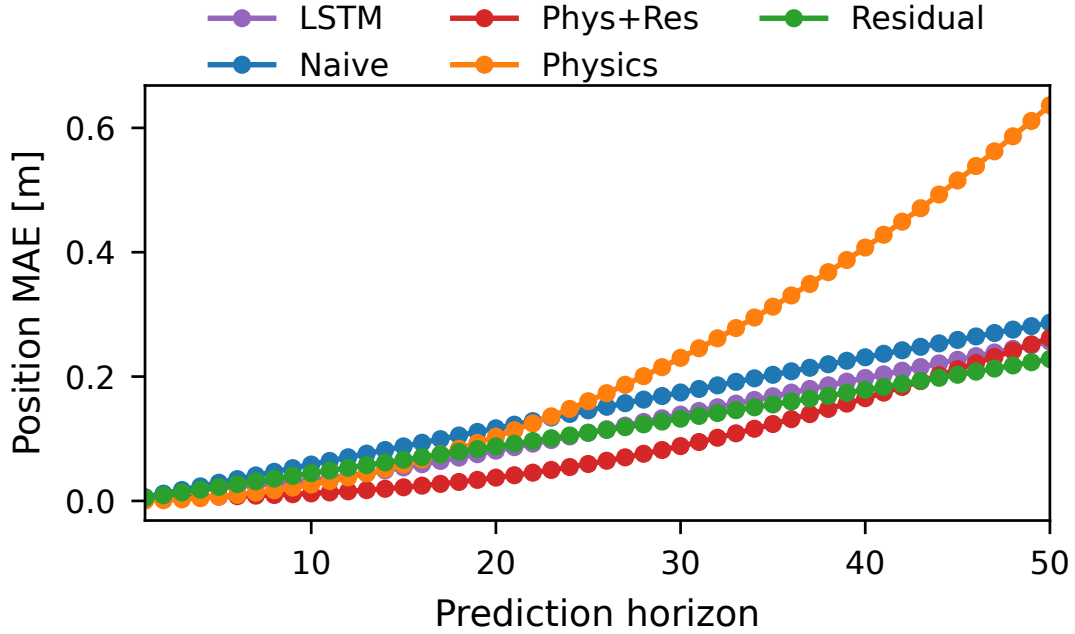


Figure 6.20
Task 1: Position MAE vs. prediction horizon.

The hybrid architecture achieves the best cumulative accuracy. The physics + residual model achieves the lowest cumulative position simulation error and outperforms the pure residual MLP (5.68 m) and the LSTM (5.92 m). The advantage is most distinct at $h=10$, where it achieves 12.1 mm position MAE, 3.7 times lower than the residual MLP and 2.7 times lower than the LSTM. At $h=50$, however, the pure residual MLP achieves the lowest position MAE (228 mm), which suggests that the physics component eventually introduces compounding errors that the residual correction only partially absorbs. The crossover between these two models near $h = 30$ defines a practical horizon boundary for each architecture.

Translational and rotational objectives trade off under a joint loss. The LSTM achieves competitive velocity MAE (0.48 ms^{-1} at $h=50$, second only to naive), but its angular velocity MAE reaches 5.15 rads^{-1} , an order of magnitude above the naive baseline (0.45 rads^{-1}). The physics model produces identical angular velocity predictions to naive at all horizons because the implementation holds $\dot{\omega} \equiv 0$, which reflects the difficulty of modeling torque transients from coreless DC motors without dedicated angular acceleration data.

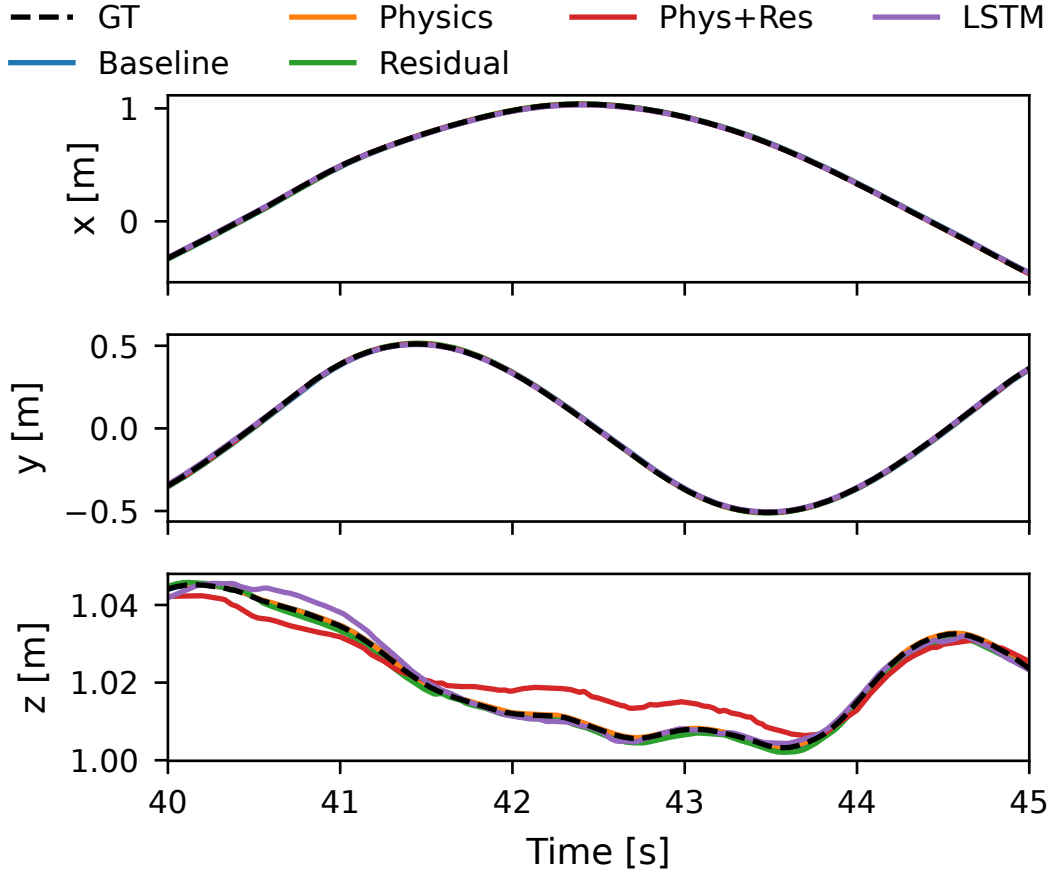


Figure 6.21

Task 1: Predicted (colored) vs. Vicon ground-truth (black) trajectories for each baseline on a held-out Category B (Helix trajectory) sequence, along each axis.

Figure 6.21 visualizes the baselines predicted versus ground-truth trajectories on Helix trajectory as a test sequence.

6.4.3 Task 2: Controller Benchmarking

6.4.3.1 Baselines

We benchmark five controllers across two distinct evaluation paradigms to separate physical hardware performance from idealized, data-driven rollout. **Real-flight** controllers execute directly onboard the platform, utilizing Vicon ground truth for state estimation: (1) **PID**, the default firmware’s cascaded PID formulation; and (2) **Mellinger**, a geometric tracking controller operating on $SE(3)$ [33]. Conversely, **Offline-learned** controllers are trained from logged flight data and

Table 6.20

Task 2: Controller Benchmarking Results

Type	Method	RMSE _p ↓ (m)	ADE _p ↓ (m)	ADE _v ↓ (m/s)	$\bar{e}_\psi$ ↓ (deg)	Div. (%)↓
Real flight	PID	0.29±0.03	0.27±0.02	0.72±0.14	5.82±0.58	4.20±2.46
	Mellinger	0.28±0.00	0.27±0.00	0.89±0.18	5.27±1.61	0.01±0.01
Offline learned	BC-MLP	0.15±0.00	0.13±0.00	0.19±0.00	1.07±0.00	0.00±0.00
	BC-LSTM	0.18±0.00	0.16±0.00	0.22±0.00	0.97±0.00	0.00±0.00
	MPPI	1.32±0.00	1.13±0.00	0.72±0.00	4.50±0.00	75.22±0.00

evaluated via closed-loop rollouts against a learned dynamics model: (3) **BC-MLP** and (4) **BC-LSTM**, which are behavior cloning policies trained via supervised imitation of the onboard PID utilizing the imitation framework [178]; and (5) **MPPI** [179], a Model Predictive Path Integral controller using a neural network forward model trained on the same dataset. Given the fundamental disparity between the physical environment and the learned simulator, hardware and offline metrics are reported independently in Table 6.20 to prevent invalid direct comparisons.

6.4.3.2 Results and Analysis

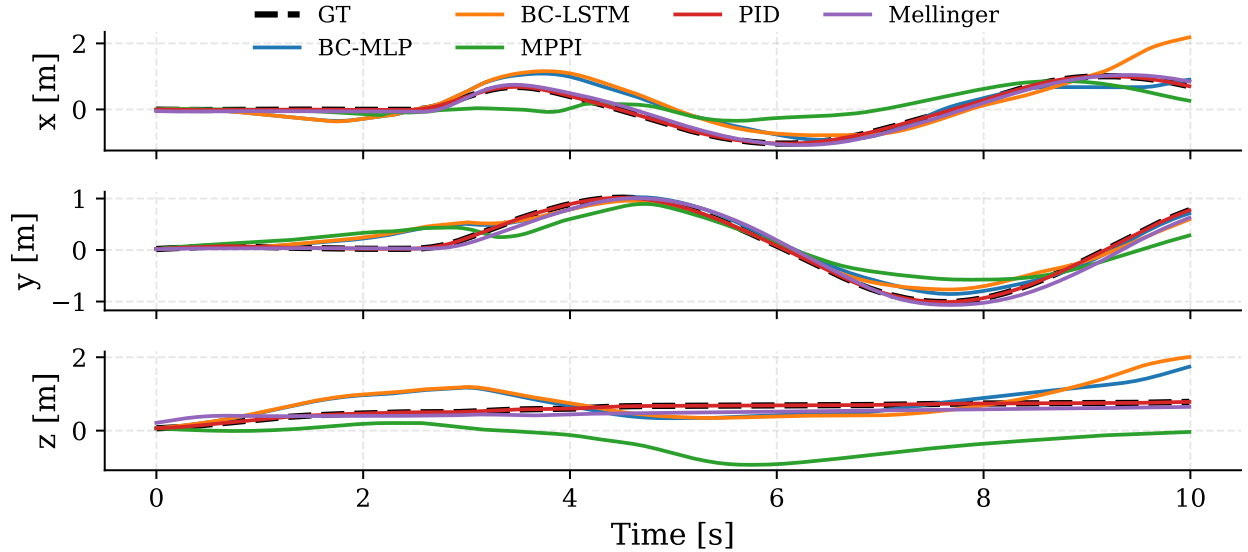


Figure 6.22

Task 2: Closed-loop position (xyz, m) for each baseline vs. Ground Truth. Real-flight (PID, Mellinger) and offline (BC-MLP, BC-LSTM, MPPI) trajectories.

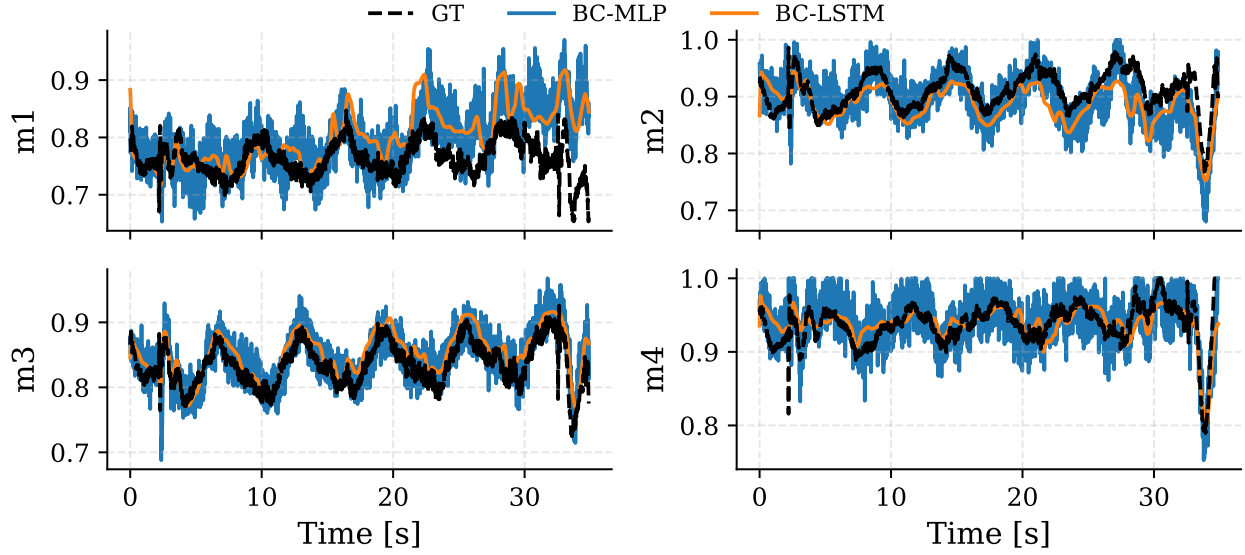


Figure 6.23

Task 2: Motor PWM outputs over time for each controller baseline (normalized).

Table 6.20 details the position and velocity Root Mean Square Error (RMSE), position Average Displacement Error (ADE), heading error, and trajectory divergence rate for all baselines.

On hardware, the geometric controller reduces divergence at the cost of higher velocity transients. In physical flight, PID and Mellinger demonstrate nearly identical position tracking, yet they diverge significantly in robustness and velocity regulation. The cascaded PID exhibits a 4.20 % (± 2.46 %) trajectory divergence rate, whereas the Mellinger controller reduces this to 0.01 %, representing a significant improvement. This enhanced reliability stems from the Mellinger controller’s $SE(3)$ formulation, which provides broader stability guarantees that prevent the aggressive attitude divergence occasionally observed with the PID. However, the Mellinger controller yields a velocity RMSE (0.89 ms^{-1}) that is $1.24\times$ higher than that of the PID (0.72 ms^{-1}). This trade-off is consistent with the geometric controller executing sharper attitude corrections, which inherently induce velocity transients on a nano-scale platform strictly bottlenecked by its 0.6 N peak thrust authority. Heading errors remain comparable across both PID and Mellinger. Qualitative tracking performance and corresponding motor commands are visualized in Fig. 6.22 and Fig. 6.23, respectively.

Offline-learned controllers are evaluated in simulation and cannot be directly compared

with real-flight results. BC-MLP and BC-LSTM achieve position RMSEs of 0.15 m and 0.18 m respectively, with zero divergence, but these metrics reflect the idealized learned rollout plant rather than physical flight readiness—the zero-variance confidence intervals across all offline metrics confirm purely deterministic evaluation. MPPI diverges on 75.22 % of trajectories (position RMSE 1.32 m), indicating that its sampling-based optimization is poorly matched to the narrow actuation limits of sub-50 g platforms without domain-specific tuning.

MPPI exhibits severe performance degradation within the nano-scale operational envelope. MPPI yields a position RMSE of 1.32 m alongside a severe 75.22 % divergence rate. This breakdown suggests that the receding-horizon optimizer is either exploiting inaccuracies in the learned forward dynamics model or that standard sampling parameters are ill-suited for the exceptionally narrow actuation limits of sub-50 g vehicles. The MPPI result confirms that NanoBench discriminates between controllers suited to highly constrained platforms and those requiring domain-specific tuning, an open challenge for model-based predictive control on sub-50 g vehicles.

6.4.4 Task 3: State Estimation

6.4.4.1 Baselines

The **onboard EKF** runs at 100 Hz on the STM32F405 and fuses IMU data with the Vicon pose relayed via `send_extpose()` to produce the full 9-DoF state estimate used during flight. It is configured at takeoff and runs for the duration of that flight. Metrics are ATE after Horn’s $SE(3)$ alignment as in (6.69), RTE over 1 m path-length windows, per-axis velocity residuals, and attitude RMSE in Euler angles.

6.4.4.2 Results and Analysis

Table 6.21 reports EKF estimation accuracy on trefoil knot trajectories over three speed regimes, with all runs pooled (mean $\pm$ std across 28 trajectories). At slow and medium speeds, the onboard EKF maintains position tracking within 22 mm ATE RMSE (11 and 10 runs respectively), with attitude RMSE below 3° and velocity RMSE below 0.07 m s^{-1} . At the fast regime (1.0 m s^{-1}), the EKF exhibits divergence (ATE $> 3 \text{ m}$ across 7 runs), indicating that the lightweight EKF on the

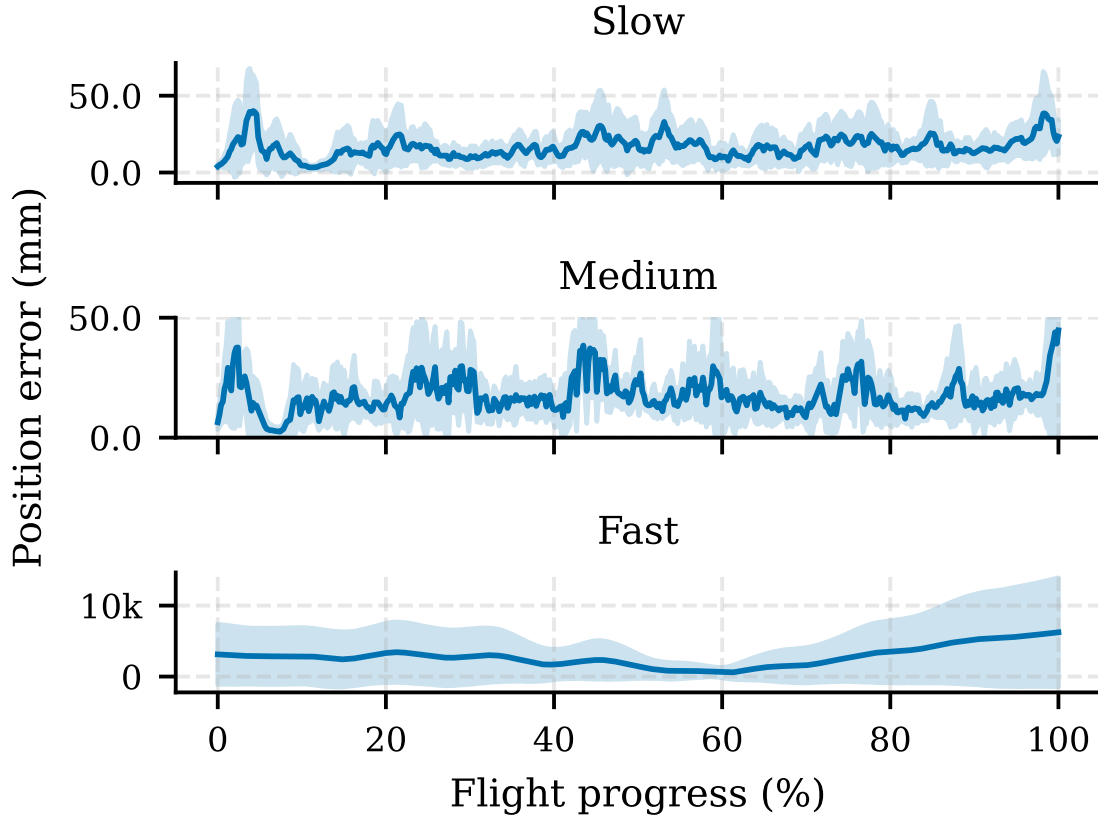


Figure 6.24

Task 3: EKF position error (mean $\pm$ std, all runs pooled by speed) on trefoil trajectories.

STM32F405 cannot maintain state consistency at the platform’s dynamic limits.

The RTE over 1 m path-length windows remains below 31 mm at slow and medium speeds, confirming local consistency. Figure 6.24 visualizes the position error with standard deviation bands (all runs pooled by speed).

The three tasks are linked by the same platform and synchronized data: Task 1 open-loop dynamics error, Task 2 closed-loop tracking, and Task 3 estimator accuracy all reflect the same underlying hardware and trajectory set. NanoBench enables systematic comparison of baselines across the autonomy stack on a platform where no prior public benchmark existed.

6.5 Conclusion

NanoBench introduces the first open-source, multi-task benchmark for nano-scale quadrotors, collecting over 170 flight trajectories on the commercially standard Crazyflie 2.1 in a Vicon motion

Table 6.21

Task 3: EKF state estimation accuracy on trefoil trajectories. Mean $\pm$ std across all runs (pooled).

Speed	N	ATE RMSE (mm)	ATE Mean (mm)	RTE 1m (mm)	Vel RMSE (m/s)	Att RMSE (deg)
Slow	11	21.1 $\pm$ 3.3	16.5 $\pm$ 3.2	28.2 $\pm$ 6.8	0.069 $\pm$ 0.015	2.12 $\pm$ 0.05
Medium	10	21.8 $\pm$ 3.0	17.2 $\pm$ 1.7	30.3 $\pm$ 2.9	0.067 $\pm$ 0.013	2.71 $\pm$ 1.43
Fast	7	3142.7 $\pm$ 3936.6	2774.4 $\pm$ 3487.9	384.2 $\pm$ 400.8	7.629 $\pm$ 8.992	29.67 $\pm$ 28.52

capture arena. It is distinguished from prior aerial datasets by jointly providing synchronized motor PWM commands, onboard EKF and PID controller internals, and millimeter-accurate ground truth, signals that no existing benchmark exposes at the nano scale.

Baseline evaluations across three tasks explained platform-specific behaviors that larger-vehicle benchmarks cannot replicate. In Task 1, rigid-body dynamics achieve sub-millimeter one-step accuracy but diverge beyond a 100 ms horizon due to compounding thrust-model error. In Task 2, the Mellinger controller reduces trajectory divergence by two orders of magnitude relative to the cascaded PID, while MPPI’s 75 % divergence rate confirms that model-based predictive methods require careful platform-specific tuning within the 0.6 N thrust envelope. In Task 3, the onboard EKF maintains sub-22 mm ATE at slow and medium speeds but loses consistency at fast speed, marking the estimation boundary of the 168 MHz Cortex-M4.

All data, code, and evaluation scripts are publicly available.²

²<https://github.com/syediu/nanobench-iro2026.git>

CHAPTER VII: CONCLUSION

7.1 Summary of Contributions

This dissertation addressed the problem of safe UAV trajectory planning in dynamic environments populated by non-cooperative obstacles whose future motion is uncertain and whose trajectories are not shared with the ego-vehicle. The work identified three interconnected limitations in existing approaches and proposed four contributions that collectively form a modular framework for uncertainty-aware aerial navigation. This section summarizes the findings of each contribution.

POF+MADER (Chapter 3). The first contribution introduced a Probabilistic Obstacle Filter based on the Kalman Filter and integrated it with the MADER [2] and Robust-MADER [3] optimization-based trajectory planners. This integration eliminated the assumption that dynamic obstacle trajectories are perfectly known a priori, replacing it with real-time uncertainty-encoded forecasts formulated as time-varying constraints in the trajectory optimization. The adaptive prediction horizon mechanism extended the forecast for faster obstacles and shortened it for slower ones, ensuring computational resources were allocated proportionally to the threat posed by each obstacle. Evaluation across 800 simulation runs spanning four motion models and four obstacle velocities demonstrated a 38.75% improvement in success rate for POF+MADER over the baselines, with only a 0.8% increase in navigation time. The minimum agent-obstacle separation increased by 41.7%, and collision duration decreased by 45.3%. Under injected localization noise, POF-based methods maintained substantially higher reliability than all baselines, which shows that the filtering layer provides robustness to perception uncertainty. Real-world experiments on Crazyflie 2.1 UAVs validated these findings, with POF+MADER achieving a 25% collision rate reduction under realistic sensing and communication conditions.

SynTraG (Chapter 4). The second contribution of this dissertation is SynTraG, a parametric synthetic trajectory generator designed to create configurable three-dimensional datasets of non-cooperative aerial obstacle motion. Rather than relying on collected flight data alone, SynTraG generates trajectories from a mixture of four closed-form motion primitives: straight-line motion,

sinusoidal motion, trefoil-knot motion, and vertical undulation. The parameters of these primitives are randomized to vary speed, amplitude, frequency, curvature, and spectral characteristics, allowing the generated corpus to span a broad range of motion patterns. To better approximate realistic sensing conditions, the generator also includes a heteroscedastic Gaussian noise model whose variance scales with path length. This provides noisy trajectories and variance targets that are directly compatible with downstream probabilistic forecasting models. The resulting corpus of 47,894 trajectories helps address a longstanding lack of public data for training and evaluating aerial trajectory predictors. In addition, SynTraG supports out-of-distribution testing by modifying primitive mixture weights or restricting parameter ranges, which makes it possible to evaluate how well learned models generalize beyond the nominal training distribution.

AeroCast (Chapter 5). The third contribution is AeroCast, a probabilistic trajectory prediction model that combines a Pre-LN Transformer encoder with a Mixture Density Network output head to predict Gaussian mixture distributions over future three-dimensional displacements at each timestep. The design is motivated by three considerations. First, representing the input as consecutive displacements improves translation invariance and keeps the input statistics bounded. Second, the training objective combines negative log-likelihood with a mode-anchoring term and a physically motivated variance floor, which helps prevent the mixture components from becoming excessively diffuse or degenerate. Third, the Transformer architecture enables each observation in the input sequence to attend directly to every other observation, shortening the effective information path compared with recurrent alternatives. On a hybrid dataset of 90,116 real and synthetic sequences across nine motion categories, AeroCast reduced ADE and FDE by about 50% relative to the strongest recurrent baseline, BiGRU-MDN. It also achieved the best NLL and CRPS among the evaluated methods while maintaining an inference time of 0.1 ms per sample. Ablation studies further showed that velocity is the most influential input feature for both forecasting accuracy and calibration, and that positional encoding is necessary to preserve coherent predictions across the full 5-second horizon.

NanoBench (Chapter 6). The fourth contribution is NanoBench, a multi-task benchmark built

around the Crazyflie 2.1 nano-quadrotor. The benchmark includes more than 170 flight trajectories with synchronized Vicon ground truth, raw IMU measurements, onboard EKF estimates, PID controller internals, and motor PWM commands recorded at 100 Hz. What distinguishes NanoBench from earlier aerial datasets is that it exposes actuator-level and estimator-level signals together with millimeter-accurate external ground truth on a commercially available nano-scale platform. Baseline experiments across three tasks demonstrate behaviors that are difficult to observe in larger-vehicle benchmarks. For system identification, a rigid-body physics model achieved sub-millimeter one-step prediction accuracy but accumulated error beyond a 100 ms horizon, whereas a hybrid physics-plus-residual model delivered the best overall long-horizon performance. For controller benchmarking, the Mellinger geometric controller reduced trajectory divergence by two orders of magnitude relative to cascaded PID control, while MPPI showed a 75% divergence rate, highlighting the need for platform-specific tuning within the limited 0.6 N thrust envelope. For state estimation, the onboard EKF maintained sub-22 mm absolute trajectory error at slow and medium speeds but became inconsistent at higher speeds, indicating a practical estimation limit for the 168 MHz Cortex-M4 platform.

7.2 Limitations

Each contribution in this dissertation is developed under a specific set of assumptions, and those assumptions define where the results should and should not be expected to hold. *POF+MADER* relies on a constant-velocity motion model, which is suitable for short-horizon prediction but cannot capture more complex behaviors such as coordinated turns, evasive maneuvers, or abrupt heading changes. Its Gaussian noise model is also matched more closely to motion-capture and conventional IMU-based localization noise than to the non-Gaussian error characteristics often encountered in vision-based detection, such as partial observations, multi-modal uncertainty, or intermittent failures. In addition, the hardware validation was performed in a controlled indoor arena with only two dynamic obstacles, so the framework has not yet been tested in outdoor conditions or in denser multi-obstacle environments.

SynTraG is limited by the structure of its generator. Although the four motion primitives

cover a useful range of aerial behaviors, they do not capture interaction-driven dynamics such as flocking, pursuit-evasion, or obstacle-aware goal-directed motion. As a result, the dataset is valuable for controlled benchmarking, but it does not yet represent the full behavioral complexity of real multi-agent aerial scenes.

AeroCast was trained and evaluated on a single hybrid dataset consisting of indoor quadrotor trajectories and parametrically generated synthetic data. Its behavior in outdoor environments, on larger aerial vehicles, or on biological agents such as birds has not yet been established. The current evaluation also focuses on single-obstacle forecasting; extending the method to settings with multiple interacting agents would likely require additional mechanisms, such as interaction-aware attention. Finally, the observed ECE range of 0.64 to 0.68 suggests that there is still room to improve probabilistic calibration through techniques such as temperature scaling or other post-hoc recalibration methods.

NanoBench is also bounded in scope. It is built on a single platform, the Crazyflie 2.1, and all data were collected in one indoor motion-capture environment. The benchmark therefore does not yet cover outdoor flight, GPS-based estimation, or visual-inertial odometry. In addition, although the battery-drain experiments span the full discharge curve, those trajectories were recorded only during hover, leaving the effect of voltage-dependent thrust degradation during aggressive maneuvers largely unexplored.

7.3 Future Work

Several natural directions follow from the contributions and limitations of this dissertation. The most direct next step is to integrate *AeroCast* with POF+MADER in an end-to-end pipeline, replacing the Kalman Filter-based obstacle predictor with Transformer-MDN forecasts and incorporating the predicted Gaussian mixture outputs directly into the planner as probabilistic collision constraints. Such an integration would connect the planning and prediction objectives of the dissertation more tightly and would allow direct evaluation of whether improved probabilistic forecasting leads to measurable planning-level safety gains.

A second direction is to extend *AeroCast* beyond single-obstacle prediction to scenarios involv-

ing multiple interacting agents. Graph-based attention mechanisms or other factored representations could model dependencies among correlated obstacles, such as birds flying in a flock or drones moving in formation, without requiring a separate predictor for every agent. A related extension would be online adaptation, where the predictor gradually specializes to agents observed over longer durations in persistent monitoring or surveillance settings.

A third direction concerns deployment outside controlled indoor environments. One practical improvement would be to incorporate measured or estimated wind information into the process-noise model of the probabilistic filter so that prediction remains reliable when the constant-velocity assumption is violated by environmental disturbances. Another would be to integrate POF+MADER with vision-based SLAM, enabling operation in GPS-denied outdoor environments while also introducing new estimation challenges that remain open in the current framework.

Finally, NanoBench can be extended in several ways. Adding battery-drain data for aggressive maneuvers, outdoor experiments with GPS-based estimation, and trajectories from additional nano-scale platforms would make the benchmark more broadly useful. These extensions would also support cross-platform comparisons of modeling, control, and state-estimation methods at the nano scale.

REFERENCES

- [1] Federal Aviation Administration, “UTM Concept of Operations Version 2.0 (UTM ConOps v2.0),” FAA, U.S. Department of Transportation, Washington, DC, USA, Tech. Rep., Aug. 2022. [Online]. Available: https://www.faa.gov/sites/faa.gov/files/2022-08/UTM_ConOps_v2.pdf
- [2] J. Tordesillas and J. P. How, “Mader: Trajectory planner in multiagent and dynamic environments,” *IEEE Transactions on Robotics*, 2022.
- [3] K. Kondo, J. Tordesillas, R. Figueroa, J. Rached, J. Merkel, P. C. Lusk, and J. P. How, “Robust mader: Decentralized and asynchronous multiagent trajectory planner robust to communication delay,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [4] X. Zhou, J. Zhu, H. Zhou, C. Xu, and F. Gao, “Ego-swarm: A fully autonomous and decentralized quadrotor swarm system in cluttered environments,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [5] A. P. Cohen, S. A. Shaheen, and E. M. Farrar, “Urban air mobility: History, ecosystem, market potential, and challenges,” *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [6] S. Liu, X. Li, H. Lu, and Y. He, “Multi-object tracking meets moving uav,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [7] D. C. Schedl, I. Kurmi, and O. Bimber, “An autonomous drone for search and rescue in forests using airborne optical sectioning,” *Science Robotics*, 2021.

- [8] B. Duo, A. Kong, Q. Wu, X. Yuan, and Y. Li, “Joint path and pick-up design for connectivity-aware uav-enabled multi-package delivery,” *Trans. Intell. Transport. Sys.*, Dec. 2024.
- [9] Z. Xu, H. Jin, X. Han, H. Shen, and K. Shimada, “Intent prediction-driven model predictive control for uav planning and navigation in dynamic environments,” *IEEE Robotics and Automation Letters*, 2025.
- [10] B. Şenbaşlar and G. S. Sukhatme, “Dream: Decentralized real-time asynchronous probabilistic trajectory planning for collision-free multirobot navigation in cluttered environments,” *IEEE Transactions on Robotics*, 2025.
- [11] M. Hassanalian and A. Abdelkefi, “Classifications, applications, and design challenges of drones: A review,” *Progress in Aerospace Sciences*, 2017.
- [12] J. Baca, S. I. Ullah, and P. Rangel, “Coaxial modular aerial system and the reconfiguration applications,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [13] J. Horyna, T. Baca, V. Walter, D. Albani, D. Hert, E. Ferrante, and M. Saska, “Decentralized swarms of unmanned aerial vehicles for search and rescue operations without explicit communication,” *Autonomous Robots*, 2023.
- [14] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, “Ego-planner: An esdf-free gradient-based local planner for quadrotors,” *IEEE Robotics and Automation Letters*, 2021.
- [15] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, “Robust and efficient quadrotor trajectory generation for fast autonomous flight,” *IEEE Robotics and Automation Letters*, 2019.
- [16] M. Pantic, I. Meijer, R. Bähneemann, N. Alatur, O. Andersson, C. Cadena, R. Siegwart, and L. Ott, “Obstacle avoidance using raycasting and riemannian motion policies at khz rates

- for mavs,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [17] A. J. Sathyamoorthy, U. Patel, T. Guan, and D. Manocha, “Frozone: Freezing-free, pedestrian-friendly navigation in human crowds,” *IEEE Robotics and Automation Letters*, 2020.
- [18] R. Takemura, N. Aoki, and G. Ishigami, “Energy-and-perception-aware planning and navigation framework for unmanned aerial vehicles,” *Advances in Mechanical Engineering*, 2023.
- [19] Z. C. Lee, Young-Hee, “Study on the uncertainty propagation of feature-based visual navigation for critical flight phases of urban air mobility,” *Proceedings of the 37th International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2024)*, 2024.
- [20] Z. Han, L. Xu, and F. Gao, “Learning to plan maneuverable and agile flight trajectory with optimization embedded networks,” *ArXiv*, 2024.
- [21] D. Fridovich-Keil, A. Bajcsy, J. F. Fisac, S. L. Herbert, S. Wang, A. D. Dragan, and C. J. Tomlin, “Confidence-aware motion prediction for real-time collision avoidance¹,” *Int. J. Rob. Res.*, Mar. 2020.
- [22] K. D. Katyal, G. D. Hager, and C.-M. Huang, “Intent-aware pedestrian prediction for adaptive crowd navigation,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [23] F. Vanegas and F. Gonzalez, “Enabling uav navigation with sensor and environmental uncertainty in cluttered and gps-denied environments,” *Sensors*, 2016.

- [24] The White House, “Executive order 14307: Unleashing american drone dominance,” 2025, june 6, 2025.
- [25] Federal Aviation Administration, “Normalizing unmanned aircraft systems beyond visual line of sight operations,” Notice of Proposed Rulemaking, 90 Fed. Reg. 38238, Docket No. FAA-2025-1908, 2025, aug. 7, 2025.
- [26] P. Kopardekar, J. Rios, T. Prevot, M. Johnson, J. Jung, and J. E. Robinson, “Unmanned aircraft system traffic management (utm) concept of operations,” in *AIAA Aviation Forum and Exposition*, no. ARC-E-DAA-TN32838, 2016.
- [27] Federal Aviation Administration, “Drone integration: Concept of operations,” 2025, may 2025.
- [28] J. Baca, “A heterogeneous modular robotic system towards the execution of cooperative tasks,” PhD Thesis, Universidad Politécnica de Madrid, Madrid, Spain, 2011.
- [29] J. Baca, M. Ferre, and R. Aracil, “A heterogeneous modular robotic design for fast response to a diversity of tasks,” *Robotics and Autonomous Systems*, 2012.
- [30] H. Zhang, Y. Tao, and W. Zhu, “Global path planning of unmanned surface vehicle based on improved a-star algorithm,” *Sensors*, 2023.
- [31] J. Tian, T. Chao, M. Yang, J. Zhu, and S. Wang, “A path planning algorithm based on improved rrt* for uavs,” in *2022 IEEE International Conference on Unmanned Systems (ICUS)*, 2022.
- [32] X. Chang, J. Wang, K. Li, X. Zhang, and Q. Tang, “Research on multi-uav autonomous obstacle avoidance algorithm integrating improved dynamic window approach and orca,” *Scientific Reports*, 2025.

- [33] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *2011 IEEE international conference on robotics and automation*. Ieee, 2011, pp. 2520–2525.
- [34] C. Richter, A. Bry, and N. Roy, “Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments,” in *Robotics Research: The 16th International Symposium ISRR*. Springer, 2016, pp. 649–666.
- [35] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics research: the 14th international symposium ISRR*. Springer, 2011, pp. 3–19.
- [36] S. Parsons, “Probabilistic robotics by sebastian thrun, wolfram burgard and dieter fox, mit press, 647 pp., \$55.00, isbn 0-262-20162-3,” *The Knowledge Engineering Review*, vol. 21, no. 3, pp. 287–289, 2006.
- [37] X. Zhang and W. Yu, “Research on the application of kalman filter algorithm in aircraft trajectory analysis,” in *2022 7th International Conference on Intelligent Computing and Signal Processing (ICSP)*, 2022.
- [38] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/036402139090002E>
- [39] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [40] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

- [41] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014.
- [42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [43] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On layer normalization in the transformer architecture,” in *Proceedings of the 37th International Conference on Machine Learning*, ser. ICML’20. JMLR.org, 2020.
- [44] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*. Springer, 2006, vol. 4, no. 4.
- [45] C. M. Bishop, “Mixture density networks,” 1994.
- [46] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, “Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data,” in *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVIII*. Springer-Verlag, 2020.
- [47] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, “Social gan: Socially acceptable trajectories with generative adversarial networks,” 03 2018.
- [48] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

- [49] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML’17. JMLR.org, 2017, p. 1321–1330.
- [50] M. Zhou and H. Lee, “Dynamic obstacle avoidance for an mav using optimization-based trajectory prediction with a monocular camera,” *IEEE Access*, 2024.
- [51] C. Dong, Y. Chen, and J. M. Dolan, “Interactive trajectory prediction for autonomous driving via recurrent meta program induction network,” in *Proceedings of (ICRA) International Conference on Robotics and Automation*, 2019.
- [52] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, “Social lstm: Human trajectory prediction in crowded spaces,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.
- [53] X. Mo, Y. Xing, and C. Lv, “Graph and recurrent neural network-based vehicle trajectory prediction for highway driving,” in *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2021.
- [54] A. Wang, C. Mavrogiannis, and A. Steinfeld, “Group-based motion prediction for navigation in crowded environments,” in *Conference on Robot Learning*. PMLR, 2022.
- [55] K. Zhang, X. Feng, L. Wu, and Z. He, “Trajectory prediction for autonomous driving using spatial-temporal graph attention transformer,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 11, 2022.
- [56] S. Eiffert, K. Li, M. Shan, S. Worrall, S. Sukkarieh, and E. Nebot, “Probabilistic crowd gan: Multimodal pedestrian trajectory prediction using a graph vehicle-pedestrian attention network,” *IEEE Robotics and Automation Letters*, 2020.

- [57] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, “Social gan: Socially acceptable trajectories with generative adversarial networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018.
- [58] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, “Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data,” in *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVIII*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 683–700. [Online]. Available: https://doi.org/10.1007/978-3-030-58523-5_40
- [59] N. Lee, W. Choi, P. Vernaza, C. B. Choy, P. H. Torr, and M. Chandraker, “Desire: Distant future prediction in dynamic scenes with interacting agents,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017.
- [60] N. Deo and M. M. Trivedi, “Trajectory forecasts in unknown environments conditioned on grid-based plans,” *ArXiv*, 2020.
- [61] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nuScenes: A Multimodal Dataset for Autonomous Driving,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society, June 2020, pp. 11 618–11 628. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPR42600.2020.01164>
- [62] S. Ettinger, S. Cheng, B. Caine, C. Liu, H. Zhao, S. Pradhan, Y. Chai, B. Sapp, C. Qi, Y. Zhou, Z. Yang, A. Chouard, P. Sun, J. Ngiam, V. Vasudevan, A. McCauley, J. Shlens, and D. Anguelov, “Large scale interactive motion forecasting for autonomous driving : The waymo open motion dataset,” in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 9690–9699.

- [63] Z. Zhang, G. Hess, J. Hu, E. Dean, L. Svensson, and K. Åkesson, “Future-oriented navigation: Dynamic obstacle avoidance with one-shot energy-based multimodal motion prediction,” *IEEE ROBOTICS AND AUTOMATION LETTERS*, 2025.
- [64] H. Zhu and J. Alonso-Mora, “Chance-constrained collision avoidance for mavs in dynamic environments,” *IEEE Robotics and Automation Letters*, 2019.
- [65] Z. Xu, D. Deng, Y. Dong, and K. Shimada, “Dpmc-planner: A real-time uav trajectory planning framework for complex static environments with dynamic obstacles,” in *2022 International Conference on Robotics and Automation (ICRA)*, 2022.
- [66] O. de Groot, B. F. de Brito, L. Ferranti, D. M. Gavrila, and J. Alonso-Mora, “Scenario-based trajectory optimization in uncertain dynamic environments,” *IEEE Robotics and Automation Letters*, 2021.
- [67] L. Heuer, L. Palmieri, A. Rudenko, A. Mannucci, M. Magnusson, and K. O. Arras, “Proactive model predictive control with multi-modal human motion prediction in cluttered dynamic environments,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [68] Q. Quan, *Introduction to multicopter design and control*. Springer, 2017, vol. 10.
- [69] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. USA: Princeton University Press, 2012.
- [70] J. Förster, “System identification of the crazyflie 2.0 nano quadrocopter,” 2015.
- [71] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, “Reaching the limit in autonomous racing: Optimal control versus reinforcement learning,” *Science Robotics*,

vol. 8, no. 82, p. eadg1462, 2023.

- [72] K. J. Åström and T. Hägglund, “Pid control,” *IEEE Control Systems Magazine*, vol. 1066, pp. 30–31, 2006.
- [73] J. A. Preiss*, W. Hönig*, G. S. Sukhatme, and N. Ayanian, “Crazyswarm: A large nano-quadcopter swarm,” in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017.
- [74] T. Qin, P. Li, and S. Shen, “Vins-mono: A robust and versatile monocular visual-inertial state estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [75] R. W. Deters, G. K. Ananda Krishnan, and M. S. Selig, “Reynolds number effects on the performance of small-scale propellers,” in *32nd AIAA applied aerodynamics conference*, 2014, p. 2151.
- [76] R. Busetto, E. Cereda, M. Forgione, G. Maroni, D. Piga, and D. Palossi, “Nonlinear system identification for a nano-drone benchmark,” *Control Engineering Practice*, vol. 172, p. 106871, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0967066126001152>
- [77] J. Park, D. Kim, G. C. Kim, D. Oh, and H. J. Kim, “Online distributed trajectory planning for quadrotor swarm with feasibility guarantee using linear safe corridor,” *IEEE Robotics and Automation Letters*, 2022.
- [78] Z. Liu, B. Chen, H. Zhou, G. Koushik, M. Hebert, and D. Zhao, “Mapper: Multi-agent path planning with evolutionary reinforcement learning in mixed dynamic environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

- [79] F. F. Lizzio, M. Bugaj, J. Rostáš, and S. Primatesta, “Comparison of multiple models in decentralized target estimation by a uav swarm,” *Drones*, 2024.
- [80] M. Matsushima, T. Hashimoto, and F. Miyazaki, “Learning to the robot table tennis task-ball control rally with a human,” in *2003 IEEE International Conference on Systems, Man and Cybernetics*.
- [81] P. Shukla, S. Shukla, and A. Kumar Singh, “Trajectory-prediction techniques for unmanned aerial vehicles (uavs): A comprehensive survey,” *IEEE Communications Surveys Tutorials*, 2025.
- [82] L. Lu, G. Fasano, A. Carrio, M. Lei, H. Bavle, and P. Campoy, “A comprehensive survey on non-cooperative collision avoidance for micro aerial vehicles: Sensing and obstacle detection,” *Journal of Field Robotics*, 2023.
- [83] Z. Xu, X. Zhan, B. Chen, Y. Xiu, C. Yang, and K. Shimada, “A real-time dynamic obstacle tracking and mapping system for uav navigation and collision avoidance with an rgb-d camera,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [84] M. Lu, H. Chen, and P. Lu, “Perception and avoidance of multiple small fast moving objects for quadrotors with only low-cost rgbd camera,” *IEEE Robotics and Automation Letters*, 2022.
- [85] Z. Xu, D. Deng, Y. Dong, and K. Shimada, “Dpmc-planner: A real-time uav trajectory planning framework for complex static environments with dynamic obstacles,” in *International Conference on Robotics and Automation (ICRA)*, 2022.
- [86] M. Zhang, B. Pang, C. Yan, M. Feroskhan, and C. Lv, “Real-time avoidance of obstacles and emergent geo-fences for urban air mobility using deep reinforcement learning,” *IEEE Transactions on Intelligent Transportation Systems*, 2025.

- [87] J. Tordesillas and J. P. How, “Deep-panther: Learning-based perception-aware trajectory planner in dynamic environments,” *IEEE Robotics and Automation Letters*, 2023.
- [88] S. Tanaka, A. Asignacion, T. Nakata, S. Suzuki, and H. Liu, “Review of biomimetic approaches for drones,” *Drones*, 2022.
- [89] S. Poudel, M. Y. Arafat, and S. Moh, “Bio-inspired optimization-based path planning algorithms in unmanned aerial vehicles: A survey,” *Sensors*, 2023.
- [90] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, “Vision-aided uav navigation and dynamic obstacle avoidance using gradient-based b-spline trajectory optimization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [91] E. Aldao, L. M. González-deSantos, H. Michinel, and H. González-Jorge, “Uav obstacle avoidance algorithm to navigate in dynamic building environments,” *Drones*, 2022.
- [92] W. Xia, F. Song, and Z. Peng, “Dynamic obstacle perception technology for uavs based on lidar,” *Drones*, 2025.
- [93] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, “Vision-aided uav navigation and dynamic obstacle avoidance using gradient-based b-spline trajectory optimization,” in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2023.
- [94] Z. F. J. S. . Q. S. Shiji Hai, Xitai Na, “Penc: a predictive-estimative nonlinear control framework for robust target tracking of fixed-wing uavs in complex urban environments,” *Scientific Reports*, 2025.
- [95] W. Dai, Z. Wang, and C. Ding, “Biomimetic swarm-based uav collision avoidance control,” in *International Conference on Computing and Communication Networks*. Springer, 2024.

- [96] T. Wakabayashi, Y. Suzuki, and S. Suzuki, “Dynamic obstacle avoidance for multi-rotor uav using chance-constraints based on obstacle velocity,” *Robotics and Autonomous Systems*, 2023.
- [97] J. Park, Y. Lee, I. Jang, and H. J. Kim, “Decentralized trajectory planning for quadrotor swarm in cluttered environments with goal convergence guarantee,” *The International Journal of Robotics Research*, 2025.
- [98] P. Chen, Y. Jiang, Y. Dang, T. Yu, and R. Liang, “Real-time efficient trajectory planning for quadrotor based on hard constraints,” *Journal of Intelligent & Robotic Systems*, 2022.
- [99] T. Wang, L. Yang, Y. Chang, Z. Huang, H. Jiang, and Y. Zheng, “A review of dynamic obstacle avoidance for unmanned aerial vehicles (uavs),” in *7th International Symposium on Autonomous Systems (ISAS)*, 2024.
- [100] K. Tamanakijprasart, S. Mondal, and A. Tsourdos, “Multi-agent path planning in dynamic environment with communication disruptions,” in *International Conference on Control, Automation and Diagnosis (ICCAD)*, 2025.
- [101] L. Ren, M. Li, S. Fan, Y. Zhang, F. Yu, and J. Yang, “Cooperative control method of multi-agent formation and obstacle avoidance,” in *IEEE International Conference on Mechatronics and Automation (ICMA)*, 2025.
- [102] C. Toumieh and A. Lambert, “Decentralized multi-agent planning using model predictive control and time-aware safe corridors,” *IEEE Robotics and Automation Letters*, 2022.
- [103] B. Luders, M. Kothari, and J. How, “Chance constrained rrt for probabilistic robustness to environmental uncertainty,” *AIAA Guidance, Navigation, and Control Conference*, 2010.

- [104] M. Garvanova, I. Garvanov, D. Borissova, N. Kerimbayev, and Z. Menlibay, “Uav detection and recognition technologies,” in *International Conference on Telecommunications and Remote Sensing*. Springer, 2024.
- [105] H.-C. Chen, “Monocular vision-based obstacle detection and avoidance for a multicopter,” *IEEE Access*, 2019.
- [106] J. Lin, H. Zhu, and J. Alonso-Mora, “Robust vision-based obstacle avoidance for micro aerial vehicles in dynamic environments,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [107] J. Zhong, M. Li, Y. Chen, Z. Wei, F. Yang, and H. Shen, “A Safer Vision-Based Autonomous Planning System for Quadrotor UAVs with Dynamic Obstacle Trajectory Prediction and Its Application with LLMs ,” in *IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*. IEEE Computer Society, Jan. 2024.
- [108] P. Kumar, D. Makwana, O. Susladkar, A. Mittal, and P. K. Kalra, “Moves: Movable and moving lidar scene segmentation in label-free settings using static reconstruction,” *Pattern Recognition*, 2024.
- [109] T. Eppenberger, G. Cesari, M. Dymczyk, R. Siegwart, and R. Dubé, “Leveraging stereo-camera data for real-time dynamic obstacle detection and tracking,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [110] X. Liu, G. V. Nardari, F. Cladera, Y. Tao, A. Zhou, T. Donnelly, C. Qu, S. W. Chen, R. A. F. Romero, C. J. Taylor, and V. Kumar, “Large-scale autonomous flight with real-time semantic slam under dense forest canopy,” *IEEE Robotics and Automation Letters*, 2022.

- [111] S. I. Ullah and A. Muhammad, “Autonomous navigation and mapping of water channels in a simulated environment using micro-aerial vehicles,” in *2023 International Conference on Robotics and Automation in Industry (ICRAI)*, 2023.
- [112] G. Chen, W. Dong, P. Peng, J. Alonso-Mora, and X. Zhu, “Continuous occupancy mapping in dynamic environments using particles,” *IEEE Transactions on Robotics*, 2024.
- [113] G. Chen, W. Dong, X. Sheng, X. Zhu, and H. Ding, “An active sense and avoid system for flying robots in dynamic environments,” *IEEE/ASME Transactions on Mechatronics*, 2021.
- [114] S. Amutha, S. Ramanan, S. Ganesan, B. Leena, A. J. M. D. Choudhry, and M. Sundarrajan, “Crow search algorithm for energy-efficient uav path planning in large-scale aerial surveys,” in *3rd International Conference on Device Intelligence, Computing and Communication Technologies (DICCT)*, 2025.
- [115] Y. K. Nakka, A. Liu, G. Shi, A. Anandkumar, Y. Yue, and S.-J. Chung, “Chance-constrained trajectory optimization for safe exploration and learning of nonlinear systems,” *IEEE Robotics and Automation Letters*, 2021.
- [116] E. Olcay, H. Meeß, and G. Elger, “Dynamic obstacle avoidance for uavs using mpc and gp-based motion forecast,” in *European Control Conference (ECC)*, 2024.
- [117] J. Zhou, Y. Gao, O. Johansson, B. Olofsson, and E. Frisk, “Robust predictive motion planning by learning obstacle uncertainty,” *IEEE Transactions on Control Systems Technology*, 2025.
- [118] S. Argiliana, E. Ekawati, and F. Mukhlis, “Adaptive strategies for dynamic obstacle avoidance and formation control in multi-agent drone systems: A review,” *Journal of Robotics and Control (JRC)*, 2025.

- [119] J. Tordesillas and J. P. How, “Minvo basis: Finding simplexes with minimum volume enclosing polynomial curves,” *Computer-Aided Design*, 2022.
- [120] C. Gao, Z. Wang, J. Hu, Y. Liu, and X. He, “Consensus-based distributed state estimation over sensor networks with encoding-decoding scheme: Accommodating bandwidth constraints,” *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 6, pp. 4051–4064, 2022.
- [121] M. M. Rashid, M. Seyedi, and S. Jung, “Simulation of pedestrian interaction with autonomous vehicles via social force model,” *Simulation Modelling Practice and Theory*, 2024.
- [122] T. T. Zhang, P. J. Jin, S. T. McQuade, A. Bayen, and B. Piccoli, “Car-following models: A multidisciplinary review,” *IEEE Transactions on Intelligent Vehicles*, 2025.
- [123] I. H. Lee and C. G. Park, “An improved interacting multiple model algorithm with adaptive transition probability matrix based on the situation,” *International Journal of Control, Automation and Systems*, vol. 21, no. 10, pp. 3299–3312, 2023.
- [124] Y. Huang, J. Du, Z. Yang, Z. Zhou, L. Zhang, and H. Chen, “A survey on trajectory-prediction methods for autonomous driving,” *IEEE Transactions on Intelligent Vehicles*, 2022.
- [125] P. Kothari, S. Kreiss, and A. Alahi, “Human trajectory forecasting in crowds: A deep learning perspective,” *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [126] M.-F. Chang, J. Lambert, P. Sangkloy, J. Singh, S. Bak, A. Hartnett, D. Wang, P. Carr, S. Lucey, D. Ramanan, and J. Hays, “Argoverse: 3d tracking and forecasting with rich maps,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

- [127] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, “Social lstm: Human trajectory prediction in crowded spaces,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [128] N. S. H. Al-Zadjali, S. Balasubaramanian, C. Savarimuthu, and E. O. Rances, “Bio-inspired motion detection models for improved uav and bird differentiation: a novel deep learning framework,” *Scientific Reports*, 2025.
- [129] H. Zhou, X. Yang, M. Fan, L. Qi, X. Li, M.-H. Yang, and F. Luo, “Three-dimensional trajectory prediction with 3dmotraj dataset,” in *Forty-second International Conference on Machine Learning*, 2025.
- [130] J. Zhang, Z. Meng, S. Liu, J. Ji, and J. He, “A novel trajectory prediction method for uav air combat based on qcnet-3d,” *Defence Technology*, 2025.
- [131] H. Liang, S. Yuan, F. Liu, Y. Yang, B. Wang, Z. Huang, C. Shi, and J. Jin, “Label-free long-horizon 3d uav trajectory prediction via motion-aligned rgb and event cues,” 2025.
- [132] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, “Propagating state uncertainty through trajectory forecasting,” in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 2351–2358.
- [133] A. Chaudhari, M. Treiber, and O. Okhrin, “Mitra: A drone-based trajectory data for an all-traffic-state inclusive freeway with ramps,” *Scientific Data*, 07 2025.
- [134] S. Yuan, Y. Yang, T. H. Nguyen, T.-M. Nguyen, J. Yang, F. Liu, J. Li, H. Wang, and L. Xie, “Mmaud: A comprehensive multi-modal anti-uav dataset for modern miniature drone threats,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

- [135] H. Naik, J. Yang, D. Das, M. C. Crofoot, A. Rathore, and V. H. Sridhar, “Bucktales: a multi-uav dataset for multi-object tracking and re-identification of wild antelopes,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS, 2025.
- [136] S. Shah, D. Dey, C. Lovett, and A. Kapoor, “Airsim: High-fidelity visual and physical simulation for autonomous vehicles,” in *Field and service robotics: Results of the 11th international conference*. Springer, 2017.
- [137] G. Rizzoli, F. Barbato, M. Caligiuri, and P. Zanuttigh, “Syndrone-multi-modal uav dataset for urban scenarios,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.
- [138] S.-L. Hsu, E. Tung, J. Krumm, C. Shahabi, and K. Shafique, “Trajgpt: Controlled synthetic trajectory generation using a multitask transformer-based spatiotemporal model,” in *Proceedings of the 32nd ACM International Conference on Advances in Geographic Information Systems*. USA: Association for Computing Machinery, 2024.
- [139] Y. Zhu, Y. Ye, S. Zhang, X. Zhao, and J. J. Yu, “Difftraj: generating gps trajectory with diffusion probabilistic model,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS, 2023.
- [140] N. K. Chandran, M. T. H. Sultan, A. Lukaszewicz, F. S. Shahr, A. Holovatyy, and W. Giernacki, “Review on type of sensors and detection method of anti-collision system of unmanned aerial vehicle,” *Sensors*, vol. 23, no. 15, 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/15/6810>
- [141] S. Izzat Ullah, J. Baca, P. Rangel, T. Chu, and C. Rubio-Medrano, “Pof+mader: Trajectory planner in dynamic environments with improved collision avoidance,” *IEEE Access*, vol. 13,

- pp. 215 533–215 549, 2025.
- [142] O. Nacar, M. Abdelkader, L. Ghouti, K. Gabr, A. Al-Batati, and A. Koubaa, “Vector: Velocity-enhanced gru neural network for real-time 3d uav trajectory prediction,” *Drones*, vol. 9, no. 1, 2025.
 - [143] Y. Yuan, X. Weng, Y. Ou, and K. Kitani, “Agentformer: Agent-aware transformers for socio-temporal multi-agent forecasting,” in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 9793–9803.
 - [144] Z. Zhou, L. Ye, J. Wang, K. Wu, and K. Lu, “Hivt: Hierarchical vector transformer for multi-agent motion prediction,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 8813–8823.
 - [145] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6405–6416.
 - [146] A. Kendall and Y. Gal, “What uncertainties do we need in bayesian deep learning for computer vision?” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 5580–5590.
 - [147] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.
 - [148] S. Pellegrini, A. Ess, K. Schindler, and L. van Gool, “You’ll never walk alone: Modeling social behavior for multi-target tracking,” in *2009 IEEE 12th International Conference on Computer Vision*, 2009, pp. 261–268.

- [149] S. I. Ullah and J. Baca, “SynTraG: A synthetic trajectory generator for non-cooperative dynamic obstacles in UAV navigation,” in *Advances in Automation and Robotics Research*. Springer, 2026.
- [150] S. I. Ullah and J. Baca, “Nanobench: A multi-task benchmark dataset for nano-quadrator system identification, control, and state estimation,” *arXiv preprint arXiv:2603.09908*, 2026.
- [151] D. Schwab, S. M. O’Rourke, and B. L. Minnehan, “Combining lstm and mdn networks for traffic forecasting using the argoverse dataset,” in *2021 IEEE 24th International Conference on Information Fusion (FUSION)*, 2021, pp. 1–6.
- [152] Y. Zhi, Z. Bao, S. Zhang, and R. He, “Bigru based online multi-modal driving maneuvers and trajectory prediction,” *Proceedings of the institution of mechanical engineers, part d: journal of automobile engineering*, vol. 235, no. 14, pp. 3431–3441, 2021.
- [153] W. Guo, Y. Du, X. Shen, V. Lepetit, X. Alameda-Pineda, and F. Moreno-Noguer, “Back to mlp: A simple baseline for human motion prediction,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, January 2023, pp. 4809–4819.
- [154] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [155] I. Loshchilov and F. Hutter, “Sgdr: tochastic gradient descent with warm restarts,” *International Conference on Learning Representations*, 2017.
- [156] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, Y. W. Teh and

- M. Titterton, Eds., vol. 9. Chia Laguna Resort, Sardinia, Italy: PMLR, 13–15 May 2010, pp. 249–256. [Online]. Available: <https://proceedings.mlr.press/v9/glorot10a.html>
- [157] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Mueller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature*, vol. 620, pp. 982–987, 08 2023.
- [158] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-fly enables rapid learning for agile flight in strong winds,” *Science Robotics*, 2022.
- [159] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, “Data-driven mpc for quadrotors,” *IEEE Robotics and Automation Letters*, 2021.
- [160] D. Palossi, N. Zimmerman, A. Burrello, F. Conti, H. Muller, L. M. Gambardella, L. Benini, A. Giusti, and J. Guzzi, “Fully onboard ai-powered human-drone pose estimation on ultralow-power autonomous flying nano-uavs,” *IEEE Internet of Things Journal*, 2022.
- [161] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, “Safe learning in robotics: From learning-based control to safe reinforcement learning,” *Annual Review of Control, Robotics, and Autonomous Systems*, 2022.
- [162] L. Pichierri, A. Testa, and G. Notarstefano, “Crazychoir: Flying swarms of crazyfly quadrotors in ros,” *IEEE Robotics and Automation Letters*, 2023.
- [163] R. W. Deters, G. K. Ananda Krishnan, and M. S. Selig, “Reynolds number effects on the performance of small-scale propellers,” in *32nd AIAA applied aerodynamics conference*, 2014, p. 2151.

- [164] G. Shi, X. Shi, M. O’Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural lander: Stable drone landing control using learned dynamics,” in *international conference on robotics and automation (ICRA)*. IEEE, 2019.
- [165] D. Palossi, “On the autonomous navigation of nano-uavs,” Ph.D. dissertation, ETH Zurich, 2019.
- [166] L. Bauersfeld, E. Kaufmann, P. Foehn, S. Sun, and D. Scaramuzza, “Neurobem: Hybrid aerodynamic quadrotor model,” *arXiv preprint arXiv:2106.08015*, 2021.
- [167] J. Eschmann, D. Albani, and G. Loianno, “Learning to fly in seconds,” *IEEE Robotics and Automation Letters*, 2024.
- [168] N. El-Sheimy and A. Youssef, “Inertial sensors technologies for navigation applications: State of the art and future trends,” *Satellite navigation*, vol. 1, no. 1, p. 2, 2020.
- [169] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, “The euroc micro aerial vehicle datasets,” *The International Journal of Robotics Research*, 2016.
- [170] J. Delmerico, T. Cieslewski, H. Rebecq, M. Faessler, and D. Scaramuzza, “Are we ready for autonomous drone racing? the uzh-fpv drone racing dataset,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6713–6719.
- [171] T.-M. Nguyen, S. Yuan, M. Cao, Y. Lyu, T. H. Nguyen, and L. Xie, “Ntu viral: A visual-inertial-ranging-lidar dataset, from an aerial vehicle viewpoint,” *The International Journal of Robotics Research*, 2022.

- [172] C. Brommer, A. Fornasier, M. Scheiber, J. Delaune, R. Brockers, J. Steinbrener, and S. Weiss, “Insane: Cross-domain uav data sets with increased number of sensors for developing advanced and novel estimators,” *arXiv preprint arXiv:2210.09114*, 2022.
- [173] A. Antonini, W. Guerra, V. Murali, T. Sayre-McCord, and S. Karaman, “The blackbird dataset: A large-scale dataset for uav perception in aggressive flight,” in *2018 International Symposium on Experimental Robotics (ISER)*, 2018. [Online]. Available: https://doi.org/10.1007/978-3-030-33950-0_12
- [174] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, “Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight,” *Science Robotics*, 2022.
- [175] M. Wang, J. Zhou, X. Duan, D. Zhao, P. Cai, J. Zhai, X. Liu, and C. Ren, “Nonlinear system identification for quadrotors with neural ordinary differential equations,” in *2023 IEEE International Conference on Unmanned Systems (ICUS)*, 2023, pp. 317–322.
- [176] K. Y. Chee, T. Z. Jiahao, and M. A. Hsieh, “Knode-mpc: A knowledge-based data-driven predictive control framework for aerial robots,” *IEEE Robotics and Automation Letters*, 2022.
- [177] C. Llanes, Z. Kakish, K. Williams, and S. Coogan, “Crazysim: A software-in-the-loop simulator for the crazyflie nano quadrotor,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [178] A. Gleave, M. Taufeeque, J. Rocamonde, E. Jenner, S. H. Wang, S. Toyer, M. Ernestus, N. Belrose, S. Emmons, and S. Russell, “imitation: Clean imitation learning implementations,” *arXiv:2211.11972v1 [cs.LG]*, 2022.

- [179] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic mpc for model-based reinforcement learning,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

ProQuest Number: 32788635

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2026).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license
or other rights statement, as indicated in the copyright statement or in the metadata
associated with this work. Unless otherwise specified in the copyright statement
or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA